National University of Computer and Emerging Sciences

Department of Artificial Intelligence



SwiftRescue

Project Team

Muhammad Sajjad	**22I-0590**
Usama Ahmad	**22I-0577**
Ali Raza	**22I-0627**

Supervised by

Dr. Muhammad Noman Noor

Co-Supervised by

Dr. Ahmad Din

Session 2022–2026

Islamabad, Pakistan

January, 2026

Contents

Chapter 1 Introduction **7**

Abstract **7**

1.1 1.1 Problem Statement 8
1.2 1.2 Motivation 8
1.3 1.3 Problem Solution 9
1.4 1.4 Scope 9
1.5 1.5 Modules 9
1.5.1 1.5.1 Victim Detection and Simulation Setup 10
1.5.2 1.5.2 Simulation and Autonomous UAV Coordination 10
1.5.3 1.5.3 GCS Development and Drone Assembly 11
1.5.4 1.5.4 Ground Control and Hardware Integration 11
1.6 1.6 Related Work 11
1.6.1 1.6.1 UAV-Based Disaster Response 12
1.6.2 1.6.2 Deep Learning for Aerial Object Detection 12
1.6.3 1.6.3 Multi-UAV Coordination 12
1.6.4 1.6.4 UAV Simulation Frameworks 13
1.7 1.7 Tools and Technologies 13
1.8 1.8 Entities and Their Descriptions 14

Chapter 2 Project Requirements **16**

2.1 2.1 Use-case / Event Response Table / Storyboarding 16
2.2 2.2 Functional Requirements 22
2.2.1 2.2.1 Module 1 – Victim Detection and Simulation 22
2.2.2 2.2.2 Module 2 – Simulation and UAV Swarm Coordination . . . 22
2.2.3 2.2.3 Module 3 – Ground Control Station (GCS) 23
2.2.4 2.2.4 Module 4 – Ground Control and Hardware Integration . . . 23
2.3 2.3 Non-Functional Requirements 24
2.3.1 2.3.1 Performance 24
2.3.2 2.3.2 Reliability 24
2.3.3 2.3.3 Usability 25
2.3.4 2.3.4 Scalability 25
2.3.5 2.3.5 Security 25

Chapter 3 System Overview **26**

3.1 3.1 Architectural Design 26
3.1.1 3.1.1 Disaster Environment Layer 26
3.1.2 3.1.2 UAV Swarm Layer 27
3.1.3 3.1.3 Communication Layer 27
3.1.4 3.1.4 Data Flow and Processing Layer 28
3.1.5 3.1.5 Ground Control System (GCS) 28
3.1.6 3.1.6 Architectural Alignment with System Modules 28
3.2 Domain Model 31
3.3 Data Design 31
3.4 Design Models 32
3.4.1 Activity Diagram 33

3.4.2 Class Diagram 36
3.4.3 State Diagram 38

Chapter 4: Implementation and Testing **39**
4.1 Simulation Implementation 39
4.1.1 Simulation Framework Overview 39
4.1.2 Gazebo Harmonic (LTS Version) 41
4.1.3 ROS 2 Jazzy Integration 42
4.1.4 ArduPilot Simulation (SITL) Overview 44
4.1.5 UAV and Sensor Simulation 46
4.1.6 Multi-UAV Swarm Simulation Setup 47
4.1.7 Mission Execution and Operator Interaction Workflow 48
4.1.8 Simulation Testing and Validation 49
4.1.9 Summary of Simulation Implementation 51
4.2 Algorithm Design and Implementation 52
4.2.1 Perception Algorithm (YOLOv8s-Based Detection) 52
4.2.2 Detection Pipeline and Processing Workflow 53
4.2.3 Navigation and Flight Control Logic (ArduPilot) 54
4.2.4 Multi-UAV Coordination Strategy (Mission Planner-Based) 55
4.2.5 Data Communication and Processing Architecture 56
4.2.6 Secure Data Transmission (HTTPS-Based Encryption) 57
4.2.7 End-to-End System Pipeline and Workflow 58
4.3 Ground Control Station (GCS) 60
4.3.1 Overview of GCS Architecture 60
4.3.2 Mission Planner Integration (Telemetry and Control) 61
4.3.3 Real-Time Video and Detection Interface 61
4.3.4 User Interface and Visualization Design 62
4.3.5 System Monitoring and Operator Interaction 63
4.4 Hardware Integration and System Deployment 64
4.4.1 Hardware Components and Platform Description 64
4.4.1.1 Core Flight and Control Hardware 64
4.4.1.2 Power System and Energy Management 66
4.4.1.3 Navigation and Sensor Suite 67
4.4.1.4 Onboard Computing and Autonomy Platform 67
4.4.1.5 Communication and Networking Hardware 68
4.4.1.6 Safety and Auxiliary Components 69
4.4.1.7 Storage and Peripheral Accessories 69
4.4.2 Hardware Integration and Configuration (Mission Planner) 70
4.4.2.1 Flight Controller Firmware Setup 70
4.4.2.2 Sensor Calibration and Verification 70
4.4.2.3 Power System Configuration 71
4.4.2.4 Motor, ESC, and Frame Configuration 72
4.4.2.5 Safety, Failsafe, and Arming Configuration 73
4.4.2.6 Companion Computer Interface Configuration 74
4.4.3 System Deployment Workflow 75
4.4.3.1 Hardware Setup and Power Initialization 75
4.4.3.2 Flight Controller Initialization and Sensor Verification 75
4.4.3.3 Onboard Processing System Setup (Jetson Orin Nano) 76

4.4.3.4 Communication Link Establishment 76
4.4.3.5 Mission Planning and Configuration (GCS) 77
4.4.3.6 Autonomous Mission Execution 77
4.4.3.7 Real-Time Monitoring and Operator Interaction 77
4.4.3.8 Mission Termination and System Shutdown 78
4.4.3.9 Final System Deployment and Hardware Overview 78
4.5 Testing Details 80
4.5.1 Simulation-Based Testing and Validation 80
4.5.2 Hardware-Oriented Testing Considerations and System Readiness . 99

Chapter 5: Conclusions and Future Work **120**
5.1 Conclusion 120
5.2 Future Work 121

List of Figures

1 Use Case Diagram of the SwiftRescue UAV Swarm System 21
2 System Architecture of the SwiftRescue UAV Swarm System 30
3 Domain Model of the SwiftRescue UAV Swarm System 31
4 Data Design Diagram of the SwiftRescue UAV Swarm System 32
5 Activity Diagram of SwiftRescue Mission Execution 34
6 Activity Diagram of UAV Autonomous Operation 35
7 Class Diagram of the SwiftRescue UAV Swarm System 37
8 State Diagram of the UAV Mission Lifecycle in the SwiftRescue System . 38
9 Gazebo-Based Flood Disaster Simulation Environment with UAV and Static Obstacles 40
10 Complex Flooded Urban Disaster Scenario Simulated in Gazebo for Multi-UAV Operations 41
11 ROS 2 Jazzy Architecture Illustrating Node Execution and DDS-Based Middleware Layers 43
12 ROS 2 Communication Flow Showing Node Interaction via Topics and Services 44
13 ArduPilot Autopilot Architecture and Control Flow in Software-in-the-Loop (SITL) Simulation 46
14 Simulated UAV Camera View Capturing Flooded Disaster Environment in Gazebo 47
15 Multi-UAV Swarm Simulation Setup in Gazebo Flood Scenario 48
16 Operator-to-UAV interaction via GCS using high-level commands such as “Fly To” 49
17 Ground Control Station View Showing Mission Execution and Detected Points in Simulation 51
18 Ground Control Station interface showing multi-UAV detection streams and Mission Planner-based telemetry and control visualization 63
19 Core flight and control hardware components used in the SwiftRescue UAV platform, including motors, ESCs, propellers, and quadcopter frame . . . 65
20 Jetson Orin Nano Developer Kit used as the onboard computing platform for real-time perception and detection processing 68
21 Final Assembled SwiftRescue UAV Platform 79

List of Tables

1 Entities and Their Descriptions 15
2 Functional Testing: Mission Configuration (TC-FUNC-01) 81
3 Functional Testing: Autonomous Navigation (TC-FUNC-02) 81
4 Functional Testing: YOLOv8s Victim Detection Pipeline (TC-FUNC-03) 82
5 Functional Testing: MAVLink Telemetry Transmission (TC-FUNC-04) . 82
6 Functional Testing: Mission Termination (TC-FUNC-05) 83
7 Integration Testing: Gazebo–ROS 2 Sensor Integration (TC-INT-01) . . 84
8 Integration Testing: ROS 2 and ArduPilot SITL Communication (TC-INT-02) 84
9 Integration Testing: YOLOv8s Perception Module and GCS Detection Dashboard (TC-INT-03) 85
10 Integration Testing: Multi-UAV Telemetry Aggregation at GCS (TC-INT-04) 85
11 Integration Testing: End-to-End Mission Execution (TC-INT-05) 86
12 Multi-UAV Swarm Testing: Multi-UAV Deployment (TC-SWARM-01) . 87
13 Multi-UAV Swarm Testing: Independent UAV Control (TC-SWARM-02) 87
14 Multi-UAV Swarm Testing: Distributed Area Coverage (TC-SWARM-03) 88
15 Multi-UAV Swarm Testing: Multi-UAV Telemetry Aggregation (TC-SWARM-04) 88
16 Multi-UAV Swarm Testing: Swarm Scalability Observation (TC-SWARM-05) 89
17 Communication Reliability: Continuous MAVLink Telemetry Streaming (TC-COMM-01) 89
18 Communication Reliability: YOLOv8s Detection Data Transmission (TC-COMM-02) 90
19 Communication Reliability: Operator Command Responsiveness via MAVLink (TC-COMM-03) 90
20 Communication Reliability: Multi-UAV MAVLink Telemetry Stability (TC-COMM-04) 91
21 Communication Reliability: Extended Mission Duration Stability (TC-COMM-05) 91
22 Failure and Safety Testing: Mission Abort Command (TC-SAFE-01) . . 92
23 Failure and Safety Testing: Partial UAV Failure (TC-SAFE-02) 92
24 Failure and Safety Testing: Communication Interruption Recovery (TC-SAFE-03) 93
25 Failure and Safety Testing: ArduPilot Emergency Mode Switching (TC-SAFE-04) 93
26 Failure and Safety Testing: Mission Termination Event Logging (TC-SAFE-05) 94
27 Performance Observation: MAVLink Telemetry Update Rate (TC-PERF-01) 94
28 Performance Observation: UAV Position Visualization Smoothness (TC-PERF-02) 95
29 Performance Observation: YOLOv8s Detection Output Latency (TC-PERF-03) 95
30 Performance Observation: Multi-UAV Simulation System Stability (TC-PERF-04) 96

31 Performance Observation: Operator Command Response Time via MAVLink (TC-PERF-05) 96
32 User Interaction Testing: Mission Configuration via Mission Planner GCS (TC-UI-01) 97
33 User Interaction Testing: Mission Planner Telemetry Clarity (TC-UI-02) 97
34 User Interaction Testing: YOLOv8s Detection Result Interpretation (TC-UI-03) 98
35 User Interaction Testing: Operator Intervention During Autonomous Mission (TC-UI-04) 98
36 User Interaction Testing: Overall Dual-Interface GCS Usability (TC-UI-05) 99
37 Power System Test Case TC-UAV-PS-01: Battery Voltage Verification . . 100
38 Power System Test Case TC-UAV-PS-02: Voltage Stability Under Motor Load 101
39 Power System Test Case TC-UAV-PS-03: Power Distribution Verification 101
40 Power System Test Case TC-UAV-PS-04: ESC Electrical Response Testing 102
41 Power System Test Case TC-UAV-PS-05: Jetson Orin Nano Power Integration 103
42 Flight Controller Test Case TC-UAV-FC-01: IMU Calibration Verification 104
43 Flight Controller Test Case TC-UAV-FC-02: Compass Calibration 105
44 Flight Controller Test Case TC-UAV-FC-03: GPS Signal Acquisition and Sensor Validation 106
45 Propulsion Test Case TC-UAV-MP-01: Motor Rotation Direction Verification 107
46 Propulsion Test Case TC-UAV-MP-02: ESC Calibration and Throttle Response 108
47 Propulsion Test Case TC-UAV-MP-03: Thrust Generation and Hover Readiness 109
48 Communication Test Case TC-UAV-CM-01: MAVLink Telemetry Link Verification 110
49 Communication Test Case TC-UAV-CM-02: MAVLink Command Transmission Testing 110
50 Communication Test Case TC-UAV-CM-03: YOLOv8s Detection Stream Transmission via HTTPS 111
51 Flight Test Case TC-UAV-FS-01: Initial Hover Stability Assessment . . . 112
52 Flight Test Case TC-UAV-FS-02: Attitude Stability and Recovery Response 113
53 Flight Test Case TC-UAV-FS-03: Low-Altitude Short-Duration Stability 114
54 Flight Mode Test Case TC-UAV-AF-01: STABILIZE Mode Verification . 115
55 Flight Mode Test Case TC-UAV-AF-02: ALTITUDE HOLD Mode Testing 115
56 Flight Mode Test Case TC-UAV-AF-03: LOITER Mode (GPS Position Hold) Testing 116
57 Safety Test Case TC-UAV-SF-01: Pre-Flight Safety Check Verification . 117
58 Safety Test Case TC-UAV-SF-02: ArduPilot Failsafe Response Verification 118
59 Safety Test Case TC-UAV-SF-03: Emergency Stop and Disarm Testing . 119

Abstract

Natural disasters such as floods pose severe threats to human life, particularly in densely populated and developing regions such as Pakistan, where delayed situational awareness and limited aerial coverage significantly reduce rescue effectiveness. Existing UAV-based disaster response systems predominantly rely on single-drone operation or manual control, limiting their scalability, coverage, and operational efficiency in large-scale and time-critical rescue scenarios. This work presents **SwiftRescue**, a modular, AI-enabled multi-UAV framework designed for coordinated autonomous search and rescue operations in flood-affected environments.

SwiftRescue integrates a fine-tuned YOLOv8s deep learning model, trained on a custom dataset of 472 flood-related aerial images using a two-phase training strategy, for real-time onboard victim detection deployed on the NVIDIA Jetson Orin Nano (8GB) edge computing platform. Autonomous flight control and waypoint-based navigation are managed by ArduPilot firmware on the Pixhawk flight controller, with multi-UAV coordination achieved through Ground Control Station-based mission planning via Mission Planner, where each UAV executes an independently assigned mission plan enabling systematic and non-overlapping area coverage without requiring inter-UAV communication. A dual-channel communication architecture separates MAVLink-based flight telemetry from HTTPS-encrypted detection streaming, ensuring that perception pipeline operations do not interfere with flight control stability.

System validation was conducted through two complementary phases. Simulation-based validation was performed using Gazebo Harmonic, ROS 2 Jazzy, and ArduPilot SITL across 35 structured test cases covering functional behavior, integration, multi-UAV coordination, communication reliability, failure handling, performance, and operator usability — all confirming correct system behavior. The fine-tuned YOLOv8s baseline model achieved a mean Average Precision of 0.893 at mAP@0.5, with precision of 0.836 and recall of 0.810 on the validation set, while the UAV-optimized deployment model demonstrated improved generalization under real-world aerial conditions. Hardware-oriented testing across 23 test cases on the physical F450 quadrotor platform confirmed stable power distribution, correct sensor calibration, reliable propulsion behavior, and effective ArduPilot failsafe mechanisms, with results categorized as Pass, Pass with Observation, or Conditional Pass to reflect honest system readiness.

The results demonstrate the practical feasibility of integrating autonomous multi-UAV operation, onboard deep learning inference, and real-time operator monitoring within a unified and tested disaster response framework. SwiftRescue establishes a technically grounded foundation for UAV-based rescue operations, with identified directions for future work including outdoor field deployment, PID gain tuning, RTK-GPS integration for improved positional accuracy, dataset expansion, and decentralized swarm coordination.

Keywords: Unmanned Aerial Vehicle, Multi-UAV Systems, Search and Rescue, YOLOv8s, Object Detection, ArduPilot, Gazebo Simulation, Disaster Response, Edge Computing, NVIDIA Jetson Orin Nano, Ground Control Station, MAVLink

Chapter 1

Introduction

Natural disasters such as floods, earthquakes, and large-scale structural collapses pose serious threats to human life and critical infrastructure, particularly in densely populated and developing regions. These events often create hazardous and rapidly changing environments where timely rescue operations are difficult due to damaged transportation networks, unstable structures, and limited situational awareness. In Pakistan, recent flood disasters have resulted in widespread displacement, economic loss, and significant casualties, where delayed access to affected areas and lack of real-time information severely reduced rescue efficiency [1, 2]. Unmanned Aerial Vehicles (UAVs) have emerged as an effective tool for disaster response due to their ability to rapidly survey affected areas and operate without exposing human responders to danger [6]. However, most existing UAV-based systems rely on single-drone operation or manual control, limiting scalability and coverage in large disaster zones [3, 4]. Recent advancements in autonomous control, onboard computing, and coordinated multi-UAV operation enable multiple UAVs to operate collaboratively, providing improved coverage, adaptability, and operational reliability [8]. Motivated by these advancements, this project presents **SwiftRescue**, a modular UAV-based framework designed for simulation-based validation, perception-driven victim detection using a fine-tuned YOLOv8s model, and coordinated autonomous multi-UAV search and rescue operations managed through a unified Ground Control Station.

1.1 Problem Statement

Disaster-affected environments are often characterized by collapsed structures, flooded terrain, debris, and hazardous conditions that severely restrict access for rescue teams. Delays in locating victims and assessing situational conditions significantly increase the risk of fatalities and secondary damage. While UAVs have been introduced to assist in disaster monitoring, many existing systems operate as isolated platforms or rely heavily on manual control, limiting their effectiveness in large-scale and time-critical rescue missions [3, 4]. There remains a critical gap in integrated multi-UAV systems capable of autonomous navigation, real-time victim detection, and coordinated operation within a unified and scalable framework. Addressing this gap requires practical multi-UAV solutions that combine deep learning-based perception, autonomous navigation, and reliable communication while ensuring robustness, adaptability, and operational reliability in uncertain and dynamic environments [8, 7].

1.2 Motivation

Pakistan's recurring exposure to floods and other natural disasters underscores the urgent need for advanced technological solutions that enhance emergency response capabilities. Analysis of past flood events indicates that delayed situational awareness, limited aerial coverage, and constrained human resources significantly reduce rescue effectiveness [1, 2]. Autonomous UAV systems offer a promising solution by enabling rapid exploration of large and inaccessible disaster areas, continuous real-time monitoring, and coordinated

operation of multiple aerial agents [5]. Coordinated multi-UAV deployment improves area coverage while maintaining manageable system complexity and operator supervision. The availability of lightweight deep learning models such as YOLOv8s [11], capable of real-time inference on edge computing platforms such as the NVIDIA Jetson Orin Nano [19], further enables onboard victim detection without reliance on external servers. These advantages motivated the development of **SwiftRescue**, which integrates perception-based detection, autonomous navigation, and simulation-based validation to improve the efficiency, safety, and scalability of disaster response operations.

1.3 Problem Solution

SwiftRescue proposes a modular multi-UAV system designed to support autonomous disaster monitoring and coordinated search-and-rescue operations. Each UAV operates autonomously using ArduPilot-based navigation [29] while performing real-time victim detection using a fine-tuned YOLOv8s model [11] deployed on the NVIDIA Jetson Orin Nano onboard computing platform [19]. Coordination between multiple UAVs is achieved through structured mission planning using the Ground Control Station (GCS), where each UAV executes an independently assigned mission plan under unified operator supervision, enabling systematic and non-overlapping coverage of the disaster area.

Communication between system components is established using ROS 2 [27] and MAVLink-based protocols [29], enabling real-time telemetry exchange and mission monitoring. The Ground Control Station provides centralized supervision, visualization, and control, allowing operators to monitor multiple UAVs simultaneously and intervene when necessary to ensure safe and reliable operation.

1.4 Scope

The scope of **SwiftRescue** encompasses the design and implementation of a simulation-driven UAV framework for disaster response. This includes realistic disaster modeling, autonomous navigation, perception-based victim detection using a custom-trained YOLOv8s model, and coordinated multi-UAV operation [26, 27]. While simulation-based validation forms the core of the current implementation, the system architecture is designed to support transition to real-world deployment through hardware integration on physical UAV platforms.

Limitations:

- Simulation results depend on the fidelity of environmental models and sensor abstractions [26].
- Certain real-world factors such as extreme weather conditions, signal interference, and hardware constraints are simplified in the current simulation setup [27, 28].
- Advanced swarm intelligence mechanisms such as dynamic task allocation, inter-UAV communication, and learning-based coordination are not implemented in the current prototype and are identified as directions for future work.

1.5 Modules

SwiftRescue is organized into four major modules to ensure modularity, scalability, and clear separation of responsibilities. Each module addresses a specific functional aspect

of the system, allowing independent development, testing, and future extension. This modular design improves system reliability and enables seamless integration between simulation, perception, coordination, and control components.

1.5.1 Victim Detection and Simulation Setup

This module focuses on creating a realistic simulation environment and enabling reliable victim detection within that environment. Virtual sensors and disaster scenarios are used to safely evaluate perception algorithms under diverse and challenging conditions. The module ensures that UAVs are exposed to realistic terrain variations, occlusions, and environmental constraints, which is essential for validating detection performance before real-world deployment. A custom flood-related image dataset comprising 472 images was collected and used to fine-tune a YOLOv8s model [11, 12] for victim detection in aerial flood scenarios.

Key responsibilities include:

- Modeling disaster environments and terrain using Gazebo Harmonic simulation tools [26]
- Simulating onboard sensors such as cameras, GPS, and IMU
- Implementing and deploying a fine-tuned YOLOv8s-based victim detection pipeline [11]
- Evaluating detection performance under varying environmental and lighting conditions

1.5.2 Simulation and Autonomous UAV Coordination

This module governs the coordinated operation of multiple UAVs within the simulated environment. It enables autonomous navigation using ArduPilot [29] while supporting coordinated multi-UAV operation through structured mission planning via the Ground Control Station. Each UAV executes an independently assigned waypoint-based mission plan, enabling systematic and distributed coverage of the disaster area under unified GCS supervision [35].

Key responsibilities include:

- Autonomous UAV navigation and motion control using ArduPilot SITL [30]
- Multi-UAV coordination using GCS-based mission planning and waypoint assignment [35]
- Distributed area coverage through independently assigned mission plans per UAV
- Safe and stable UAV operation during coordinated multi-UAV missions

1.5.3 GCS Development and Drone Assembly

This module provides a centralized interface for monitoring and supervising UAV operations. It enables operators to visualize UAV positions, system status, and mission progress in real time. Additionally, this module supports mission planning, telemetry analysis, and system diagnostics, facilitating structured testing and performance evaluation. The GCS integrates Mission Planner for flight telemetry and control alongside a web-based detection dashboard for real-time visualization of victim detection outputs streamed from the onboard Jetson Orin Nano [19].

Key responsibilities include:

- Real-time visualization of UAV telemetry and status via Mission Planner [35]
- Mission planning, waypoint configuration, and command execution
- Monitoring system health and communication links across multiple UAVs
- Supporting performance analysis, detection visualization, and mission logging

1.5.4 Ground Control and Hardware Integration

This module bridges the gap between simulation-based validation and real-world deployment. It focuses on integrating onboard computing platforms, sensors, and communication protocols to enable execution of validated system components on physical UAVs. The module ensures architectural consistency so that perception, navigation, and coordination mechanisms developed in simulation can be transferred to hardware with minimal modification.

Key responsibilities include:

- Integration of the NVIDIA Jetson Orin Nano [19], Pixhawk flight controller [20], and onboard sensors
- Configuration of MAVLink [29] and ROS 2 [27] communication protocols
- Deployment of perception and navigation modules on physical hardware
- Ensuring consistency between simulation and hardware execution environments

1.6 Related Work

This section reviews existing work across four areas directly relevant to SwiftRescue: UAV-based disaster response, deep learning for aerial victim detection, multi-UAV coordination, and simulation frameworks. Each area is examined to identify the limitations of existing approaches and to position the design decisions of SwiftRescue relative to the current state of the field.

1.6.1 UAV-Based Disaster Response

The use of UAVs in disaster response has been extensively studied. Erdelj et al. [5] survey UAV applications in disaster management and demonstrate consistent improvements in response time and situational awareness compared to ground-based approaches, while noting that most deployed systems at the time relied on single-UAV platforms under manual operator control. Shakhatreh et al. [49] identify limited onboard processing capability and the absence of coordinated multi-agent operation as the primary technical barriers to effective autonomous disaster response. Silvagni et al. [10] demonstrate a multipurpose single-UAV rescue platform for avalanche scenarios that, while operationally useful, lacks the area coverage scalability provided by multi-UAV deployment. In the Pakistani context specifically, Malik and Khan [1] establish that delayed aerial coverage and limited autonomous operation were primary factors reducing rescue effectiveness during the 2022 flood disaster, directly motivating the need for a coordinated multi-UAV system such as SwiftRescue. Queralta et al. [3] present a comprehensive multi-robot search and rescue framework demonstrating that multi-agent systems significantly outperform single-agent approaches in coverage and mission completion time, while Ahmad and Waseem [4] identify that practical deployment of autonomous multi-UAV systems with onboard intelligence on real hardware remains an open challenge, with most reported systems validated in simulation only.

1.6.2 Deep Learning for Aerial Object Detection

Real-time object detection from aerial imagery presents challenges including small object size, variable scale, and complex backgrounds. The YOLO family of single-stage detectors [12, 13, 16] established the real-time detection paradigm, with each iteration improving accuracy and robustness for general-purpose detection. EfficientDet [14] proposed compound scaling for competitive accuracy with reduced computational cost, though its embedded inference latency remains a concern for real-time UAV deployment. Work targeting UAV-specific detection has also advanced: Zhu et al. [18] proposed TPH-YOLOv5 with transformer-based prediction heads for improved small object detection in drone-captured scenarios, though the additional computational overhead limits direct deployment on resource-constrained edge platforms. The Ultralytics YOLOv8 framework [11] represents the current state of the art in single-stage detection, offering an anchor-free detection head, improved backbone architecture, and integrated support for CUDA-optimized edge inference. SwiftRescue adopts YOLOv8s fine-tuned on a custom 472-image flood aerial dataset, achieving a baseline mAP@0.5 of 0.893 with precision of 0.836 and recall of 0.810, with the UAV-optimized model selected for deployment due to its improved generalization under real-world aerial conditions.

1.6.3 Multi-UAV Coordination

Multi-UAV coordination approaches range from biologically-inspired decentralized algorithms to centralized mission planning. Vicsek et al. [43] established theoretical foundations for emergent flocking behavior, while Parker [44] proposed the ALLIANCE architecture for fault-tolerant multi-robot cooperation with adaptive behavior selection. Schilling et al. [45] demonstrated learning-based swarm flight by imitation, though learned policies are difficult to verify for safety-critical rescue applications. Zhang et al. [8] review swarm intelligence algorithms for UAV coordination and identify that decentralized approaches

are constrained by the requirement for reliable inter-UAV communication, which is often unavailable in disaster environments. Mozaffari et al. [51] further analyze wireless channel variability in multi-UAV deployments and confirm that communication reliability in disaster environments supports centralized rather than decentralized coordination. SwiftRescue adopts GCS-based mission planning via Mission Planner [35], where each UAV executes an independently assigned waypoint plan without inter-UAV communication, prioritizing operational reliability and human-in-the-loop safety over theoretical coordination optimality.

1.6.4 UAV Simulation Frameworks

Simulation-based validation is essential for safe UAV system development. The Gazebo Harmonic environment [26] provides physics-based UAV simulation with tight ROS 2 integration [27], enabling sensor data to be published as standard topics consumed by perception and navigation nodes. Sardinha et al. [33] demonstrate the use of an extended ArduPilot-Gazebo plugin for multi-UAV simulation and validate that this combination effectively closes the gap between simulation and physical deployment. ArduPilot SITL [30] is distinguished by running identical firmware in simulation and on physical hardware, meaning behaviors validated in SITL transfer directly to the Pixhawk flight controller without firmware modification. Mittal [17] surveys deep learning optimization techniques for embedded platforms and identifies the NVIDIA Jetson as particularly suited for aerial robotics, supporting the selection of the Jetson Orin Nano [19] as the onboard inference platform in SwiftRescue. Collectively, the Gazebo Harmonic, ROS 2 Jazzy, and ArduPilot SITL stack adopted in SwiftRescue represents the most hardware-consistent simulation environment currently available for multirotor UAV development.

1.7 Tools and Technologies

The implementation of SwiftRescue relies on a carefully selected set of tools and technologies that collectively support realistic simulation, autonomous control, coordinated multi-UAV operation, and perception-based detection. These tools were chosen based on their robustness, maturity, community support, and compatibility with simulation-driven development workflows, ensuring reliable experimentation and reproducible results.

- **Gazebo Harmonic** is used to create realistic and physics-based simulation environments that model disaster scenarios, terrain variations, and environmental constraints. It enables safe evaluation of UAV behavior under controlled yet dynamic conditions [26].
- **ROS 2 Jazzy** serves as the middleware for internal simulation communication, UAV-to-UAV data exchange, and UAV-to-GCS integration during simulation. It provides a scalable and reliable framework for message passing, service coordination, and system integration across distributed UAV components [27, 28].
- **ArduPilot SITL** is utilized for autopilot simulation and flight control validation. It allows UAV navigation, control logic, and mission execution to be tested without physical hardware, ensuring safety and repeatability during development [29, 30, 31].

- **YOLOv8s** is employed for real-time victim detection using a custom-trained deep learning model fine-tuned on a flood-specific aerial image dataset. The UAV-optimized model, trained for 80 epochs at 960px resolution with advanced augmentation strategies, was selected for deployment due to its improved generalization under real-world aerial conditions [11, 12, 14].
- **NVIDIA Jetson Orin Nano (8GB)** is used as the primary onboard computing platform for running the YOLOv8s perception model and handling real-time inference and data streaming tasks, leveraging its GPU-accelerated computing capabilities for efficient edge deployment [19, 17].

1.8 Entities and Their Descriptions

To clarify the system architecture, the following entities describe the core physical and logical components of SwiftRescue and their interactions in enabling coordinated and autonomous operation. These entities define the main functional blocks of the system and the flow of information between perception, control, communication, and user interaction layers, supporting modularity and system-level understanding.

Entity	Description
UAV (Drone)	Autonomous aerial agent equipped with sensors, onboard processing, and communication modules. Each UAV performs navigation using ArduPilot [29] and executes real-time victim detection using the onboard YOLOv8s model [11] while transmitting telemetry and detection data to the GCS.
Coordination Mechanism	Defines multi-UAV coordination through GCS-based mission planning and independent waypoint assignment per UAV [35], enabling distributed and non-overlapping area coverage without requiring inter-UAV communication.
Ground Control Station	Centralized interface for mission monitoring and control. Integrates Mission Planner for telemetry and flight control with a web-based detection dashboard for real-time visualization of victim detection outputs [35].
Victim	Detected human subject identified by the onboard YOLOv8s model [11], associated with spatial location and confidence score, used for monitoring and rescue-related decision-making.
Environment	Disaster area containing terrain variations, flood water, obstacles, and hazards, defining the operational context for UAV navigation and perception. Modeled in Gazebo Harmonic for simulation-based evaluation [26].
Communication Module	Enables data exchange between UAVs and GCS using MAVLink for flight telemetry and HTTPS-encrypted streaming for detection outputs, with ROS 2 used for internal simulation communication [27, 29, 48].
Detection Module	Processes camera data using the fine-tuned YOLOv8s model deployed on the NVIDIA Jetson Orin Nano [19] to identify victims and generate detection outputs with associated confidence scores.
Mission	Represents a search operation with defined objectives, UAV assignment, and waypoint-based execution state including initialization, active flight, and completion, managed through the GCS [35].
Data Logger	Stores telemetry, detection data, and system events locally at the GCS for monitoring, analysis, and post-mission evaluation.

Table 1: Entities and Their Descriptions

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the SwiftRescue UAV Swarm System. The requirements define the expected behavior of the system, the interactions between users and system components, and the quality attributes that the system must satisfy. These requirements are derived based on the project scope, use-case analysis, and the limitations of a simulation-based implementation. The goal of this chapter is to clearly specify what the system is expected to do and how well it should perform under defined conditions.

2.1 Use-case / Event Response Table / Storyboarding

This section presents the key use cases of the SwiftRescue system. The use cases describe how different actors interact with the system to perform mission planning, coordinated multi-UAV operation, monitoring, and data analysis. Each use case outlines the actors involved, the main flow of events, and the resulting system behavior. These use cases reflect practical system functionality based on autonomous UAV operation, perception-driven detection using a fine-tuned YOLOv8s model [11], and mission-level coordination through the Ground Control Station [35].

Use Case 1: Configure Mission

- **Actors:** Mission Operator
- **Description:** Allows the Mission Operator to define mission parameters such as search area, UAV deployment configuration, and mission objectives using the Ground Control Station [35]. This establishes the operational context for coordinated UAV execution, including waypoint assignment per UAV and area partitioning.
- **Preconditions:** GCS is operational, Mission Planner is running, and the system is initialized.
- **Main Flow:**
  1. Operator opens the Mission Planner mission configuration interface.
  2. Mission parameters are entered, including search area boundaries and UAV-specific waypoint plans.
  3. System validates configuration against defined mission constraints.
  4. Configuration is saved and mission plans are uploaded to respective UAVs via MAVLink [39].
- **Postconditions:** Mission is ready for execution with each UAV assigned an independent waypoint plan.

Use Case 2: Launch UAV Swarm

- **Actors:** Mission Operator, UAV Swarm
- **Description:** Initiates deployment of multiple UAVs based on the configured mission. UAVs operate autonomously using ArduPilot-based flight control [29] while following independently assigned mission plans uploaded through the GCS, collectively providing distributed coverage of the disaster area.
- **Preconditions:** Mission is configured and all UAVs are armed and ready.
- **Main Flow:**
  1. Operator issues launch command via Mission Planner.
  2. Each UAV receives its assigned mission waypoints via MAVLink [39].
  3. UAVs perform pre-flight checks and begin autonomous flight.
- **Postconditions:** UAV swarm becomes active with all UAVs executing their assigned mission plans.

Use Case 3: Navigate Environment

- **Actors:** UAV Swarm
- **Description:** Enables UAVs to autonomously navigate the disaster environment using predefined waypoints managed by the ArduPilot autopilot [29]. Each UAV independently follows its assigned waypoint sequence, collectively ensuring systematic and distributed area coverage.
- **Preconditions:** UAV swarm is active and mission waypoints have been uploaded.
- **Main Flow:**
  1. Each UAV follows its individually assigned waypoint sequence.
  2. UAVs navigate through the disaster environment maintaining assigned altitude and speed.
  3. Navigation state and position data are continuously updated and transmitted to the GCS via MAVLink [29].
- **Postconditions:** UAVs continue systematic mission traversal across their assigned coverage areas.

Use Case 4: Maintain Flight Stability

- **Actors:** UAV Swarm
- **Description:** Supports safe flight behavior through ArduPilot autopilot-based attitude stabilization and path control [29], allowing UAVs to maintain stable navigation and recover from minor disturbances during mission execution. Flight stability is managed internally by the Pixhawk flight controller [20] and does not rely on active obstacle detection algorithms in the current implementation.

- **Preconditions:** UAVs are navigating and the ArduPilot autopilot is active.
- **Main Flow:**
  1. ArduPilot continuously monitors attitude, altitude, and position state.
  2. Flight controller adjusts motor control inputs to maintain stable trajectory.
  3. UAV recovers from minor disturbances and continues waypoint navigation.
- **Postconditions:** Safe and stable navigation is maintained throughout mission execution.

Use Case 5: Detect Victims

- **Actors:** UAV Swarm
- **Description:** Detects potential victims in flood-affected areas using the onboard fine-tuned YOLOv8s perception model [11] running on the NVIDIA Jetson Orin Nano [19] and generates detection outputs with confidence scores for real-time monitoring.
- **Preconditions:** Perception module is active and the YOLOv8s model is loaded on the Jetson Orin Nano.
- **Main Flow:**
  1. UAV captures real-time image frames via the onboard RGB camera.
  2. YOLOv8s model [11] processes each frame on the Jetson Orin Nano GPU [19].
  3. Detected persons are identified with associated bounding boxes and confidence scores.
  4. Detection results are packaged and streamed to the GCS via HTTP.
- **Postconditions:** Detection results are generated and transmitted to the GCS for operator review.

Use Case 6: Send Telemetry Data

- **Actors:** UAV Swarm
- **Description:** Transmits flight telemetry data such as UAV position, altitude, battery level, and system status to the GCS using the MAVLink communication protocol [39, 29]. Detection results and live video streams are transmitted separately via HTTP from the Jetson Orin Nano [19] to the GCS detection dashboard.
- **Preconditions:** MAVLink communication link is active between Pixhawk and GCS.
- **Main Flow:**
  1. Pixhawk flight controller collects real-time telemetry data including position, altitude, and battery status.

2. Telemetry data is encoded and transmitted to GCS via MAVLink protocol [39].
3. Jetson Orin Nano streams detection results and video via HTTP to the GCS detection dashboard.

- **Postconditions:** Telemetry data and detection outputs reach the GCS for visualization.

Use Case 7: Receive Telemetry

- **Actors:** Ground Control Station
- **Description:** Receives and processes flight telemetry data from all active UAVs via MAVLink through Mission Planner [35], and receives detection streams via HTTP from the Jetson Orin Nano, preparing all data for unified visualization and monitoring.
- **Preconditions:** UAV swarm is active and communication links are established.
- **Main Flow:**
  1. Mission Planner receives MAVLink telemetry from all connected UAVs [35].
  2. GCS detection dashboard receives HTTP video and detection streams.
  3. Both data streams are processed and prepared for operator visualization.
- **Postconditions:** Combined telemetry and detection data are available for real-time visualization.

Use Case 8: Visualize UAV Positions

- **Actors:** Mission Operator, Ground Control Station
- **Description:** Displays real-time UAV positions, flight paths, and mission progress on the Mission Planner map interface to support operator situational awareness and mission supervision [35].
- **Preconditions:** Telemetry data is being received by the GCS.
- **Main Flow:**
  1. Position data from MAVLink telemetry is processed by Mission Planner.
  2. UAV locations and flight trajectories are rendered on the GCS map view.
- **Postconditions:** Operator can track all active UAV positions and mission progress in real time.

Use Case 9: Display Detection Results

- **Actors:** Mission Operator, Ground Control Station

- **Description:** Displays detected victims on the GCS detection dashboard with annotated video frames, bounding boxes, confidence scores, and victim counts received via HTTP from the onboard YOLOv8s model [11], supporting operator monitoring and rescue decision-making.
- **Preconditions:** Detection data is being received from the Jetson Orin Nano via HTTP.
- **Main Flow:**
  1. Detection results and annotated video frames are received via HTTP stream.
  2. GCS detection dashboard renders bounding boxes, confidence scores, and victim count.
- **Postconditions:** Detection information is clearly visible to the operator for rescue coordination.

Use Case 10: Store Mission Data

- **Actors:** Ground Control Station
- **Description:** Stores telemetry logs, detection outputs, and mission events locally at the GCS for post-mission analysis, performance evaluation, and reporting. Mission data is retained without dependency on external database systems [35].
- **Preconditions:** Mission data is actively being generated during execution.
- **Main Flow:**
  1. Telemetry and detection data are continuously collected during mission execution.
  2. Data is stored locally at the GCS in structured log format.
- **Postconditions:** Mission data is archived locally and available for post-mission review.

Use Case 11: Review Mission Logs

- **Actors:** Mission Operator
- **Description:** Allows review and analysis of stored mission data including telemetry logs, detection records, and system events for performance evaluation, system validation, and iterative improvement [35].
- **Preconditions:** Mission logs exist from previously executed missions.
- **Main Flow:**
  1. Operator selects a completed mission from the GCS log interface.
  2. Stored telemetry, detection events, and system logs are displayed.

- **Postconditions:** Mission data is reviewed and available for analysis and reporting.

Use Case 12: Abort Mission

- **Actors:** Mission Operator
- **Description:** Terminates the active mission in case of emergency, hardware anomaly, or operator decision, triggering safe UAV behavior such as return-to-launch or controlled landing via ArduPilot failsafe mechanisms [29] to ensure system and environmental safety.
- **Preconditions:** Mission is active and GCS command link is operational.
- **Main Flow:**
  1. Operator issues abort command via Mission Planner [35].
  2. Command is transmitted to all UAVs via MAVLink [39].
  3. UAVs execute return-to-launch or controlled landing and disarm safely.
- **Postconditions:** Mission is safely terminated, all UAVs land, and mission data is preserved locally.

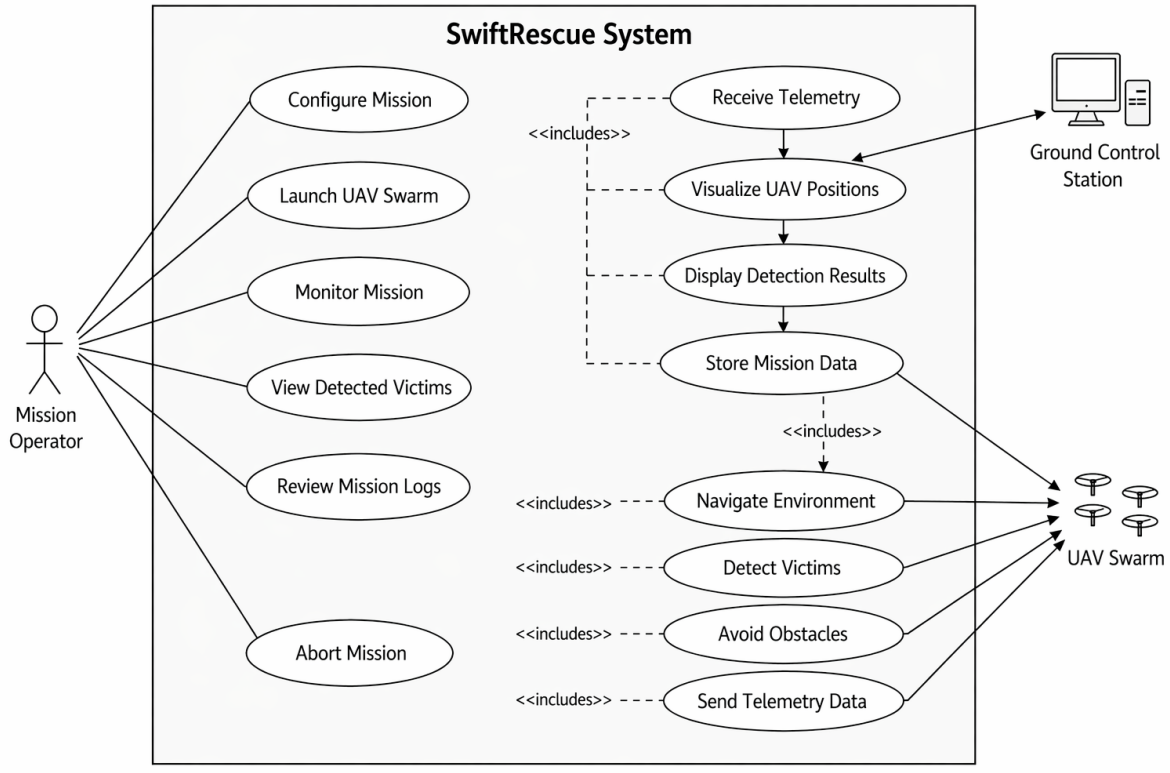


Figure 1: Use Case Diagram of the SwiftRescue UAV Swarm System

Figure 1 illustrates the complete use case diagram of the SwiftRescue UAV Swarm System, summarizing the interactions between the Mission Operator, UAV Swarm, and Ground Control Station across all twelve use cases described above.

2.2 Functional Requirements

This section defines the functional requirements of the SwiftRescue system. The requirements are organized module-wise to clearly describe the expected behavior of each system component and how they collectively support autonomous multi-UAV swarm-based search and rescue operations using deep learning-based perception, mission planning, and coordinated control. Each module represents a key functional layer of the system, ensuring that sensing, navigation, coordination, and monitoring are performed in an integrated and consistent manner [3, 8].

2.2.1 Module 1 – Victim Detection and Simulation

This module focuses on enabling perception and environment realism within the simulation framework [26]. It ensures that UAVs operate in realistic disaster scenarios where the fine-tuned YOLOv8s detection model [11] can be evaluated under varying environmental conditions and constraints. The detection model was trained on a custom dataset of 472 flood-related aerial images using a UAV-optimized configuration to improve generalization in real-world scenarios [11, 12].

- The system shall generate realistic simulated disaster environments using Gazebo Harmonic [26].
- The system shall simulate UAV onboard sensors including RGB camera, GPS, and IMU.
- The system shall support real-time vision-based victim detection using the fine-tuned YOLOv8s model [11] deployed on the NVIDIA Jetson Orin Nano [19].
- The system shall associate detections with approximate spatial locations and confidence scores.
- The system shall forward detection results and annotated video streams to the GCS via HTTP for visualization.
- The system shall allow repeated simulation runs for systematic performance evaluation.

2.2.2 Module 2 – Simulation and UAV Swarm Coordination

This module governs the coordinated operation of multiple UAVs within the system. It ensures that UAVs operate autonomously while achieving distributed area coverage through GCS-based mission planning and independent waypoint assignment per UAV [35, 29]. Coordination is managed at the operator level through Mission Planner, where each UAV is assigned a non-overlapping mission plan enabling systematic collective coverage without requiring inter-UAV communication [8].

- The system shall deploy multiple UAVs as a coordinated swarm under unified GCS supervision [35].
- The system shall enable autonomous waypoint navigation within mission boundaries using ArduPilot [29].

- The system shall support independent waypoint-based mission execution per UAV for distributed area coverage.
- The system shall achieve swarm coordination through GCS-managed mission planning and area partitioning [35], ensuring non-overlapping coverage across all active UAVs.
- The system shall maintain stable and safe swarm operation throughout mission execution, with each UAV independently managed via MAVLink [39].

2.2.3 Module 3 – Ground Control Station (GCS)

This module provides the human–system interface required for mission supervision and control [35]. It enables operators to monitor UAV behavior, visualize system outputs, and interact with the system in real time through a dual-interface architecture combining Mission Planner for flight telemetry with a web-based dashboard for detection visualization.

- The system shall provide a Mission Planner-based GCS interface for mission configuration, waypoint planning, and real-time monitoring [35].
- The system shall display real-time UAV telemetry including position, altitude, battery status, and flight mode via MAVLink [39].
- The system shall visualize detected victims through annotated video streams received from the Jetson Orin Nano via HTTP [19].
- The system shall log all mission telemetry, detection events, and operator actions locally at the GCS.

2.2.4 Module 4 – Ground Control and Hardware Integration

This module ensures seamless integration between software components and the underlying UAV hardware [20, 19]. It supports communication, control, and deployment of system functionality on both simulated and physical platforms, ensuring architectural consistency between simulation-based validation and real-world hardware execution.

- The system shall integrate ArduPilot autopilot firmware on the Pixhawk flight controller [29, 20] for both SITL simulation and hardware deployment.
- The system shall transmit flight control commands to UAVs via the MAVLink communication protocol [39].
- The system shall integrate the NVIDIA Jetson Orin Nano (8GB) [19] as the onboard computing platform for running the YOLOv8s perception model [11].
- The system shall support controlled mission termination including return-to-launch and safe landing via ArduPilot failsafe mechanisms [29].

2.3 Non-Functional Requirements

This section describes the quality attributes that the SwiftRescue system must satisfy to ensure robust, reliable, and secure operation under simulation-based and hardware-supported conditions. Non-functional requirements define how the system performs its functions rather than what functions it performs. These requirements are critical for evaluating system effectiveness in realistic disaster-response scenarios, where performance, stability, and usability directly impact mission success [5, 3]. In addition, these attributes ensure that the system remains consistent, scalable, and responsive when handling multiple UAVs operating simultaneously.

2.3.1 Performance

These requirements define acceptable performance behavior during mission execution and monitoring. Performance considerations are particularly important in time-critical disaster scenarios, where delays in telemetry updates or detection visualization can reduce situational awareness and affect operator decision-making [51]. The YOLOv8s model achieves an inference speed of approximately 10ms per frame on the Jetson Orin Nano [19, 11], supporting near real-time detection output.

- The system shall support near real-time telemetry updates via MAVLink at a minimum rate of 5 Hz [39].
- The system shall visualize UAV movements and mission progress smoothly on the Mission Planner GCS map interface [35].
- The system shall process and stream detection outputs from the Jetson Orin Nano within onboard computational constraints, targeting a minimum of 15 frames per second [19].

2.3.2 Reliability

These requirements ensure consistent system operation during normal and degraded conditions. Given the unpredictable nature of disaster environments, the system must continue functioning correctly even when individual components experience failures or communication disruptions [3]. The ArduPilot failsafe framework [29] provides built-in support for battery failsafe, RC signal loss, and return-to-launch behaviors.

- The system shall tolerate individual UAV failures within swarm operation, with remaining UAVs continuing their assigned missions independently.
- The system shall prevent loss of mission data during execution through continuous local logging at the GCS.
- The system shall handle temporary communication interruptions, with ArduPilot failsafe mechanisms [29] triggering safe UAV behavior upon signal loss.

2.3.3 Usability

These requirements focus on ease of use and operator interaction. Since disaster response often involves high cognitive load and time pressure, the system interface must be designed to support quick understanding and efficient interaction [5]. The dual-interface GCS architecture, combining Mission Planner [35] with a web-based detection dashboard, ensures that flight control and perception outputs are accessible simultaneously without requiring context switching.

- The GCS interface shall be intuitive and easy to use, requiring minimal prior UAV operational experience.
- The system shall require minimal operator training through the use of established ground control software [35].
- The system shall present mission data, UAV status, and detection results in a clear and organized format to minimize operator cognitive load during rescue operations.

2.3.4 Scalability

These requirements describe the system's ability to adapt to increased operational scale. Scalability is essential for extending the system from small-scale simulations to larger swarm deployments without architectural redesign [8]. The GCS-based coordination approach allows additional UAVs to be integrated by assigning new mission plans without modifying the underlying system architecture [35].

- The system shall support multiple UAVs operating concurrently under unified GCS supervision, each identified by a unique MAVLink system identifier [39].
- The system architecture shall allow addition or removal of UAVs within mission planning without requiring architectural changes, supporting scalable swarm deployment [35].

2.3.5 Security

These requirements ensure protection of system access and mission data. Secure access control and data integrity are necessary to prevent unauthorized manipulation of mission operations or stored results [48]. Detection video streams are transmitted using HTTPS-based encryption to protect perception data in transit between UAVs and the GCS.

- The system shall restrict mission control access to authorized operators through GCS authentication [35].
- The system shall protect detection data streams in transit using HTTPS-based encryption with TLS protocol [48].
- The system shall protect stored mission data and telemetry logs from unauthorized modification through local access control at the GCS.

Chapter 3

System Overview

This chapter presents the overall system design of the SwiftRescue UAV Swarm System. It provides a high-level description of the system architecture, core components, data organization, and design models used to represent system behavior and structure. The purpose of this chapter is to explain how different modules interact to support autonomous multi-UAV swarm-based search and rescue operations within a simulated disaster environment [26], while remaining extensible for future real-world deployment.

In addition to describing individual components, this chapter explains how these components form a cohesive and reliable system. Emphasis is placed on modularity, interoperability, and separation of concerns, which are essential for managing coordinated UAV swarm systems [3, 8]. The architectural decisions discussed here support scalability, robustness, and ease of future enhancement.

3.1 Architectural Design

The SwiftRescue system follows a modular, layered architecture designed to support autonomous UAV swarm operations, real-time monitoring, and simulation-based evaluation [26, 27]. The architecture separates perception, coordination, communication, and control components, improving scalability and maintainability while enabling integration between simulation and hardware platforms [29, 19].

The architectural design supports disaster scenarios where multiple UAVs operate concurrently under dynamic conditions [5]. A layered approach ensures that changes in one layer, such as perception or communication, do not affect others. This separation is particularly useful for simulation-based development, where iterative testing is required [26].

The architecture also enables parallel development of subsystems, allowing perception, navigation, and communication components to be refined independently while maintaining system stability. This simplifies debugging and performance evaluation through clearly defined data flow between layers.

Figure 2 illustrates the high-level architecture of the SwiftRescue system and highlights interactions between the disaster environment, UAV swarm, communication layer, onboard processing, and the ground control system.

3.1.1 Disaster Environment Layer

The system operates within a simulated disaster environment modeled using Gazebo Harmonic [26], representing scenarios such as floods and damaged infrastructure. This layer includes elements such as flood water, debris, buildings, vehicles, and victim models, providing realistic spatial and physical conditions for testing UAV navigation, perception, and swarm coordination.

The environment enables controlled experimentation by varying terrain layout and obstacle density, allowing evaluation of system performance under different operational

conditions without real-world risk [26]. Repeated simulation runs support comparative performance analysis and regression testing across different mission configurations.

3.1.2 UAV Swarm Layer

The UAV Swarm Layer consists of multiple autonomous UAVs operating collaboratively within the simulated environment [8]. Each UAV is equipped with sensors including RGB cameras, GPS, and IMU, enabling perception, localization, and waypoint-based navigation [21].

Each UAV performs onboard processing using the NVIDIA Jetson Orin Nano (8GB) [19], where camera data is processed in real time and victim detection is performed using the fine-tuned YOLOv8s model [11]. The UAV-optimized model, trained for 80 epochs at 960px resolution with advanced augmentation, was selected for deployment due to its improved generalization under real-world aerial conditions. Detection results are generated entirely onboard, ensuring low-latency operation without dependency on external processing services.

The outputs, including detection results and telemetry data, are handled through separate communication channels. Telemetry data is transmitted via MAVLink [39, 29], while detection results and live video streams are transmitted via HTTPS from the Jetson Orin Nano to the Ground Control System [19, 48].

Swarm coordination is achieved through GCS-based mission planning where each UAV is assigned an independent waypoint-based mission plan via Mission Planner [35]. This approach ensures distributed and non-overlapping area coverage, predictable UAV behavior, and simplified operator control while maintaining system reliability. The swarm also provides operational redundancy, allowing remaining UAVs to continue their assigned missions independently if one UAV becomes unavailable.

3.1.3 Communication Layer

The Communication Layer enables reliable data exchange between the UAV swarm and the Ground Control System using wireless links such as WiFi or RF [51]. The system utilizes multiple communication mechanisms based on functional requirements, ensuring that control data and perception data are handled through independent channels.

MAVLink [39] is used for flight telemetry and control, enabling structured communication of UAV position, altitude, battery status, and command data between the Pixhawk flight controller [20] and the GCS. This channel supports real-time bidirectional communication between ArduPilot [29] and Mission Planner [35].

In parallel, detection results and live video streams generated on the NVIDIA Jetson Orin Nano [19] are transmitted to the GCS using HTTPS-based communication with TLS encryption [48]. This allows efficient and secure delivery of high-level perception outputs without interfering with flight control communication.

Additionally, ROS 2 Jazzy [27] is used exclusively within the simulation environment for internal communication between system components, supporting topic-based sensor data exchange and modular integration during Gazebo-based testing [28]. ROS 2 is not used in the real-world hardware deployment configuration.

3.1.4 Data Flow and Processing Layer

This layer manages the transmission and processing of real-time data generated by the UAV swarm. Telemetry data from the Pixhawk flight controller [20] is transmitted to the GCS using the MAVLink protocol [39], while detection outputs and live video streams from the NVIDIA Jetson Orin Nano [19] are sent using HTTPS-based communication independently [48].

This separation ensures that low-level flight control data and high-level perception data are handled through distinct channels, improving system efficiency and reliability [29]. The GCS integrates these data streams through its dual-interface architecture — Mission Planner for telemetry visualization and a web-based detection dashboard for victim detection outputs — to provide a unified real-time operational view.

Mission data is stored locally at the GCS for logging and post-mission analysis. This approach simplifies system design while maintaining efficient real-time performance and reliable data handling without requiring external database infrastructure.

3.1.5 Ground Control System (GCS)

The Ground Control System provides the primary interface for mission configuration, monitoring, and visualization. It adopts a dual-interface architecture combining Mission Planner [35] for flight telemetry and control with a web-based detection dashboard for real-time visualization of victim detection outputs streamed from the NVIDIA Jetson Orin Nano [19].

Operators use Mission Planner to supervise missions, track UAV positions, analyze telemetry, and issue commands such as mission start, waypoint updates, and abort via MAVLink [39]. The system supports human-in-the-loop control while maintaining autonomous UAV operation, ensuring that operator oversight is always available [5].

The GCS stores mission logs locally, enabling analysis, validation, and performance evaluation without requiring external database systems. It aggregates telemetry data received from multiple UAVs into a unified Mission Planner interface, improving situational awareness and decision-making during mission execution. Additionally, the GCS ensures continuous feedback between UAVs and operators, allowing timely intervention when required.

Furthermore, the GCS integrates telemetry data, annotated visual detection feeds, and detection metadata into a centralized monitoring platform, enabling operators to correlate spatial UAV information with real-time perception results. This integration enhances mission coordination and reduces response latency in critical rescue scenarios [3].

3.1.6 Architectural Alignment with System Modules

The architecture aligns with the system modules defined in Chapters 1 and 2, ensuring consistency across design and implementation. This alignment ensures that each architectural layer directly supports the functional requirements defined earlier, enabling traceability between system design, implementation, and evaluation.

Additionally, this structured alignment facilitates modular development, allowing individual components to be developed, tested, and integrated independently without affecting overall system functionality. It also supports scalability by enabling the addition of new UAVs or modules with minimal architectural changes [8, 35].

- **Module 1:** Victim Detection and Simulation Setup, implemented within the UAV Swarm and Disaster Environment layers, enabling realistic environment modeling in Gazebo Harmonic [26] and validation of the fine-tuned YOLOv8s perception pipeline [11].

- **Module 2:** Simulation and UAV Swarm Coordination, supported by the UAV Swarm and Communication layers, enabling coordinated swarm behavior through GCS-based mission planning and independent waypoint assignment per UAV [35, 29].

- **Module 3:** Ground Control Station Development, realized through the dual-interface GCS architecture combining Mission Planner [35] and the web-based detection dashboard, providing real-time telemetry monitoring, detection visualization, and operator interaction.

- **Module 4:** Ground Control and Hardware Integration, supported by MAVLink communication protocols [39], the Pixhawk flight controller [20], and the NVIDIA Jetson Orin Nano onboard processing platform [19], ensuring seamless interaction between simulation and physical system components.

The layered architecture described above reflects deliberate engineering decisions that balance system complexity with operational reliability. By assigning clearly defined responsibilities to each layer, the system avoids tight coupling between components, which is a common source of failures in multi-UAV systems where dynamic conditions can affect multiple subsystems simultaneously. The separation between the MAVLink-based flight control channel and the HTTPS-based perception data channel, for example, ensures that network congestion or streaming disruptions in the detection pipeline do not degrade flight control responsiveness or telemetry continuity [39, 48].

From a scalability perspective, the architecture supports incremental deployment. Additional UAVs can be integrated into an active mission by assigning new independent mission plans through Mission Planner [35] without requiring changes to the underlying communication or perception infrastructure. Each new UAV simply requires a unique MAVLink system identifier and an independently assigned coverage zone, after which it operates within the same architectural framework as existing swarm members [8]. This property is particularly valuable in disaster response contexts, where operational scale may need to be increased rapidly as more hardware becomes available on site.

Furthermore, the architectural design maintains a clear boundary between simulation-time and deployment-time components. ROS 2 Jazzy [27], which provides internal middleware during Gazebo-based simulation, is not present in the hardware deployment stack. In hardware deployment, ArduPilot [29] handles all navigation and stabilization autonomously, MAVLink [39] handles telemetry, and the Jetson Orin Nano [19] manages perception and HTTPS streaming independently. This clean boundary reduces the risk of simulation-to-hardware transition failures caused by middleware dependencies that do not exist in real deployment environments.

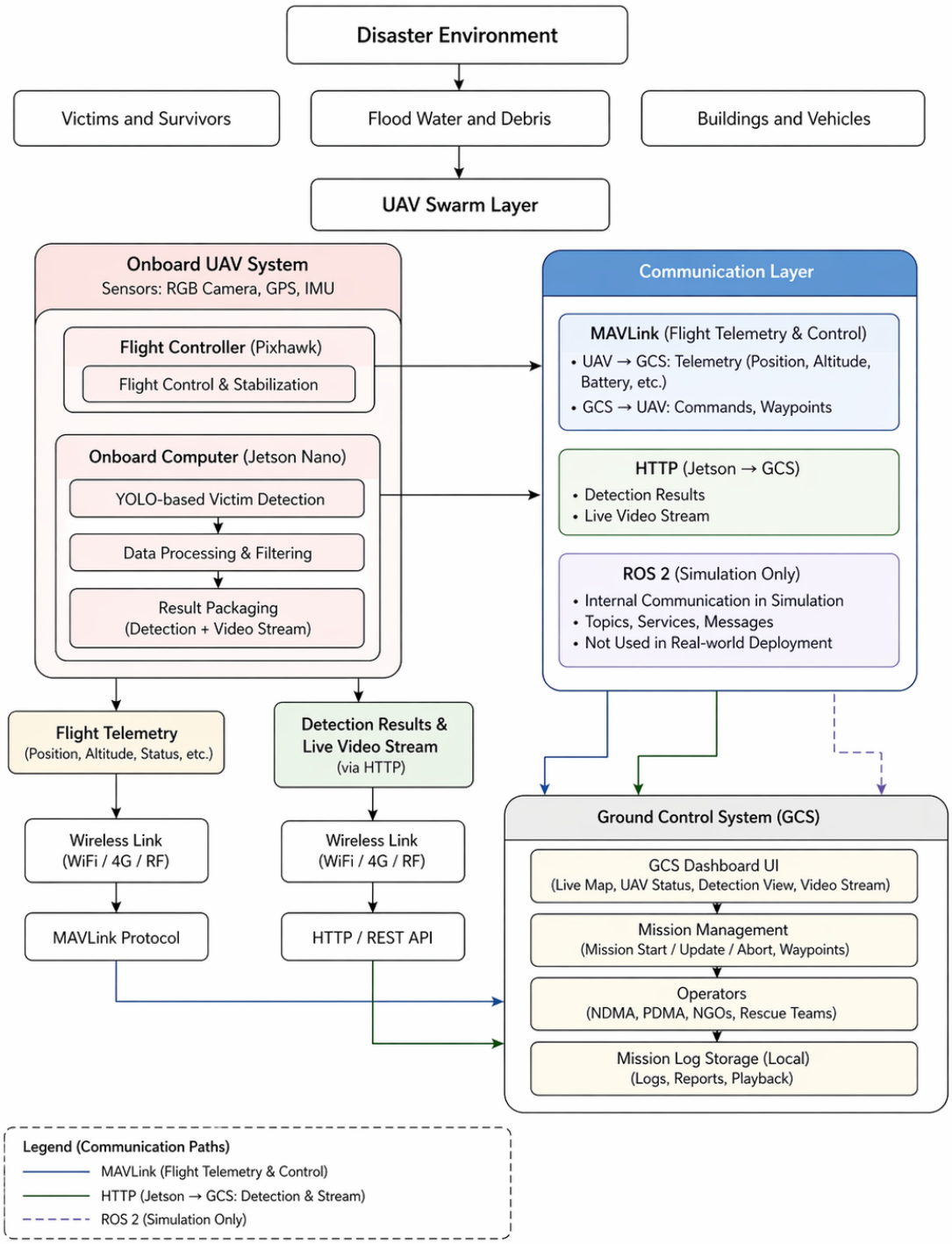


Figure 2: System Architecture of the SwiftRescue UAV Swarm System

3.2 Domain Model

The domain model of the SwiftRescue UAV Swarm System represents the core physical and logical entities involved in autonomous multi-UAV search and rescue operations and their relationships [3]. It captures key system components such as the Mission, UAVs, Ground Control Station (GCS), Environment, Detection Results, Telemetry Data, and Communication Module. The model illustrates how missions are defined and coordinated through the GCS, how UAVs interact with the environment to perform navigation and victim detection, and how system data is generated and transmitted in real time.

In the proposed system, UAVs perform onboard processing using the NVIDIA Jetson Orin Nano (8GB) [19], where victim detection is carried out using the fine-tuned YOLOv8s model [11]. Detection outputs and live video streams are transmitted to the GCS using HTTP-based communication, while flight telemetry data such as position, altitude, and status is communicated using the MAVLink protocol [39, 29]. ROS 2 Jazzy [27] is used within the simulation environment exclusively for internal data exchange between Gazebo [26] and system components. This separation of communication mechanisms ensures efficient handling of both control and perception data.

Swarm coordination is achieved through mission planning and independent waypoint-based execution managed by the GCS via Mission Planner [35]. UAVs operate autonomously while following their individually assigned mission plans, enabling structured and predictable distributed coverage without requiring inter-UAV communication. This abstraction supports understanding of system structure, data flow, and entity relationships while ensuring consistency with the architectural design and functional requirements described in previous sections.

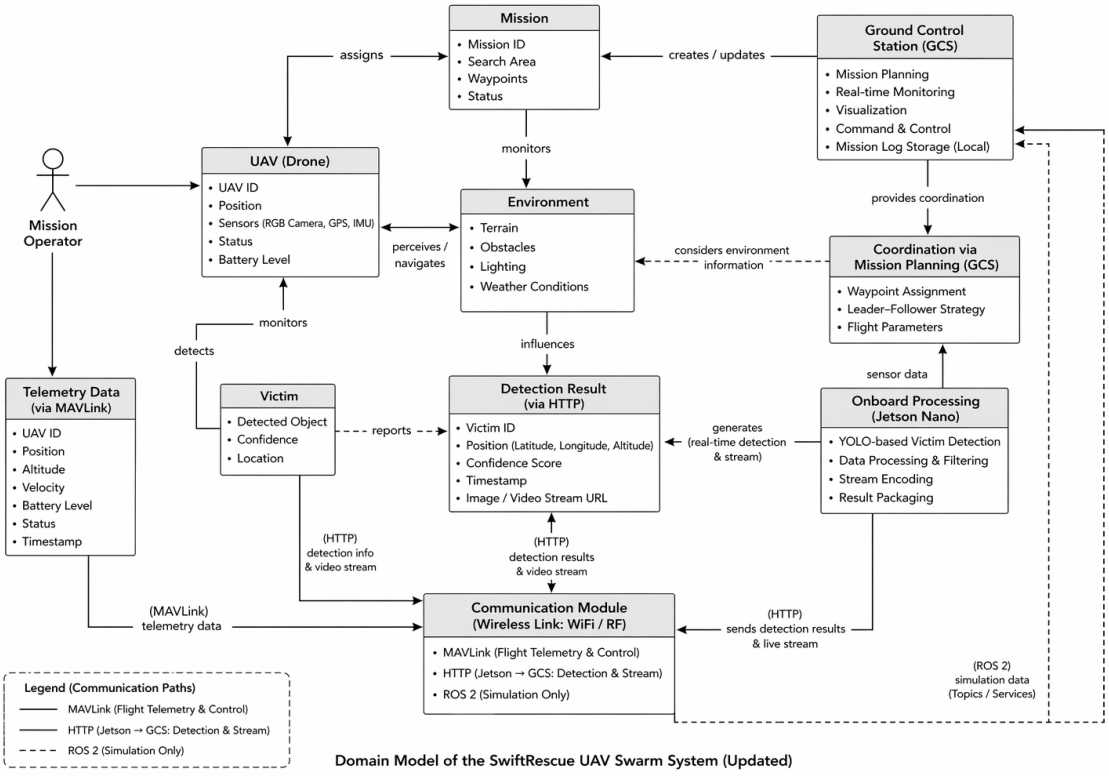


Figure 3: Domain Model of the SwiftRescue UAV Swarm System

3.3 Data Design

The data design of the SwiftRescue UAV Swarm System defines how information is generated, processed, transmitted, and utilized during mission execution [39]. The system handles multiple independent data streams including MAVLink telemetry, HTTP-based detection results, and mission state information, ensuring reliable real-time monitoring and efficient system operation.

As illustrated in Figure 4, data generation begins at the UAV level through onboard sensors such as RGB cameras, GPS, and IMU [21]. These sensors provide raw inputs including image frames and position coordinates, which are processed by onboard modules. The perception module performs victim detection using the NVIDIA Jetson Orin Nano [19] running the fine-tuned YOLOv8s model [11], while the navigation module within the Pixhawk flight controller [20] generates telemetry data such as UAV position, altitude, and system status.

The system applies a strict separation of communication channels based on data type [29]. Telemetry data is transmitted using MAVLink [39], enabling structured and reliable communication between the Pixhawk flight controller and the Ground Control Station via Mission Planner [35]. In parallel, detection results and live annotated video streams generated on the Jetson Orin Nano are transmitted to the GCS detection dashboard using HTTP-based communication [19], with HTTPS encryption applied for data security [48].

The Ground Control Station integrates these independent data streams through its dual-interface architecture to provide a unified view of mission execution. It displays UAV positions, detected victims, and system status in real time, while also storing all mission logs locally for post-mission analysis and validation. This data design ensures low latency, reliable communication, and consistency with the overall system architecture.

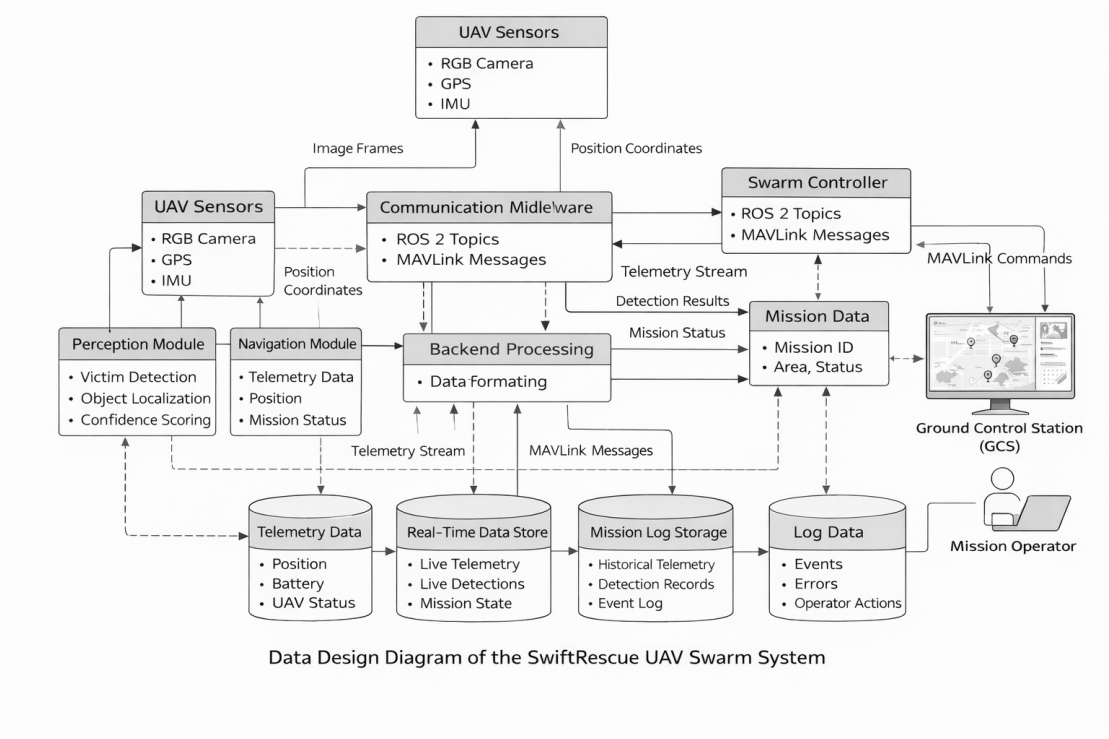


Figure 4: Data Design Diagram of the SwiftRescue UAV Swarm System

3.4 Design Models

Design models are used to visually represent the behavior and structure of the SwiftRescue UAV Swarm System. These models support understanding of system workflows, decision-making logic, and interactions between system components [3]. The design models presented here are aligned with the system requirements and architectural design discussed in earlier chapters. Activity diagrams describe mission execution workflows and individual UAV autonomy. Class and state diagrams capture structural and lifecycle dimensions respectively.

3.4.1 Activity Diagram

Activity diagrams model sequences of actions, decision points, and parallel activities within the SwiftRescue system [29]. Two diagrams are presented: the first describes the overall mission execution process involving the Mission Operator, GCS, and UAV swarm; the second focuses on the internal autonomous operation of an individual UAV.

Activity Diagram 1: SwiftRescue Mission Execution Flow

The first diagram represents the high-level workflow of a complete search and rescue mission. The process begins with mission configuration by the Mission Operator through Mission Planner [35]. Once validated, each UAV receives its independently assigned waypoint plan and is deployed into the disaster environment under ArduPilot control [29]. During execution, UAVs transmit telemetry via MAVLink [39] and detection results via HTTPS from the Jetson Orin Nano [19]. The GCS visualizes all outputs in real time. If an abort command is issued, ArduPilot failsafe mechanisms are triggered and all data is stored locally; otherwise the mission continues until all waypoints are completed.

Activity Diagram 2: UAV Autonomous Operation Flow

The second diagram focuses on the internal behavior of an individual UAV. After sensor initialization, the UAV enters a continuous loop of waypoint navigation and image capture under ArduPilot stabilization [29]. The fine-tuned YOLOv8s model [11] running on the Jetson Orin Nano [19] processes each frame onboard, generating detection results with confidence scores. Detection outputs are streamed to the GCS via HTTPS while telemetry is transmitted concurrently via MAVLink [39]. The loop continues while the mission is active, after which the UAV executes return-to-launch and lands safely.

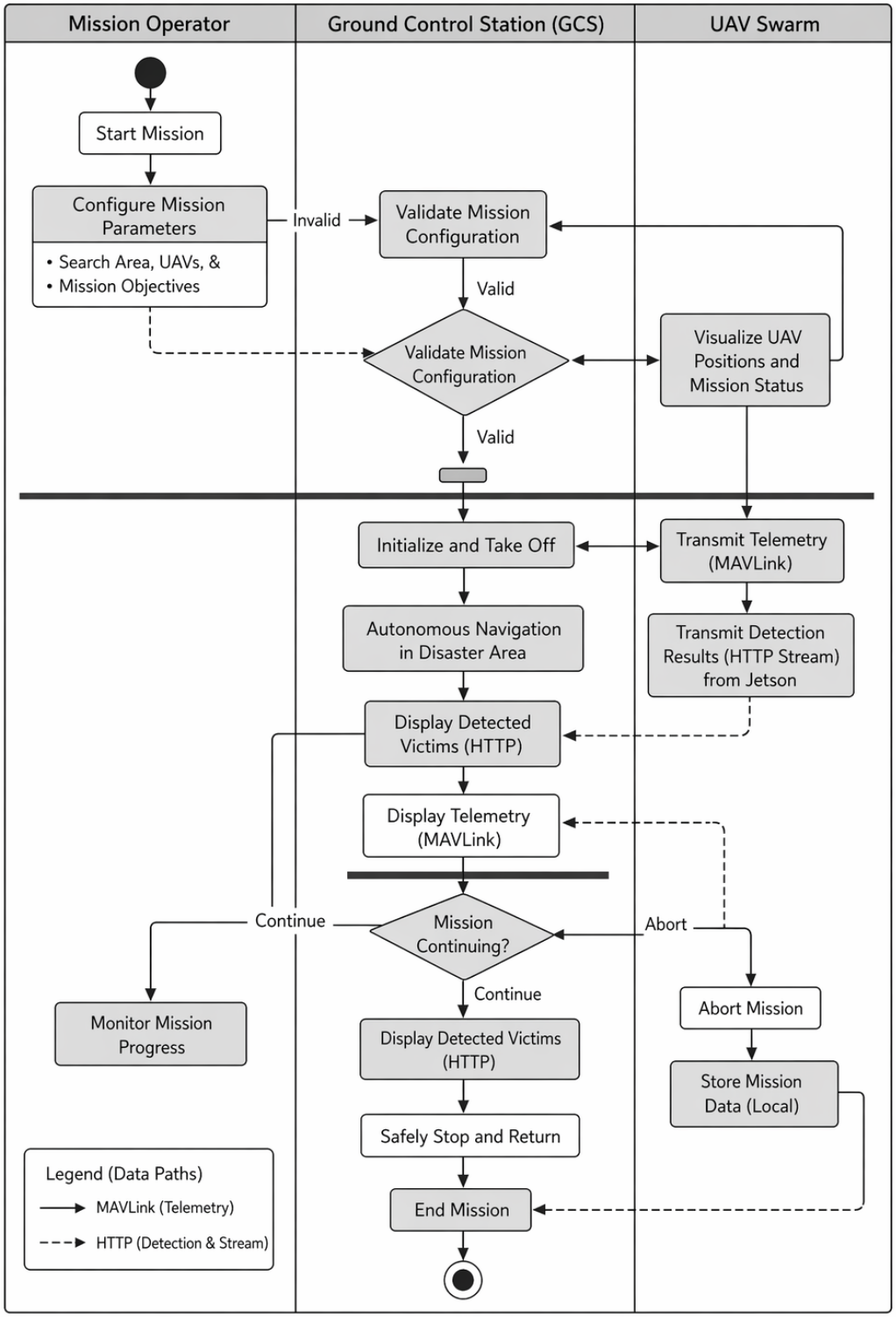


Figure 5: Activity Diagram of SwiftRescue Mission Execution

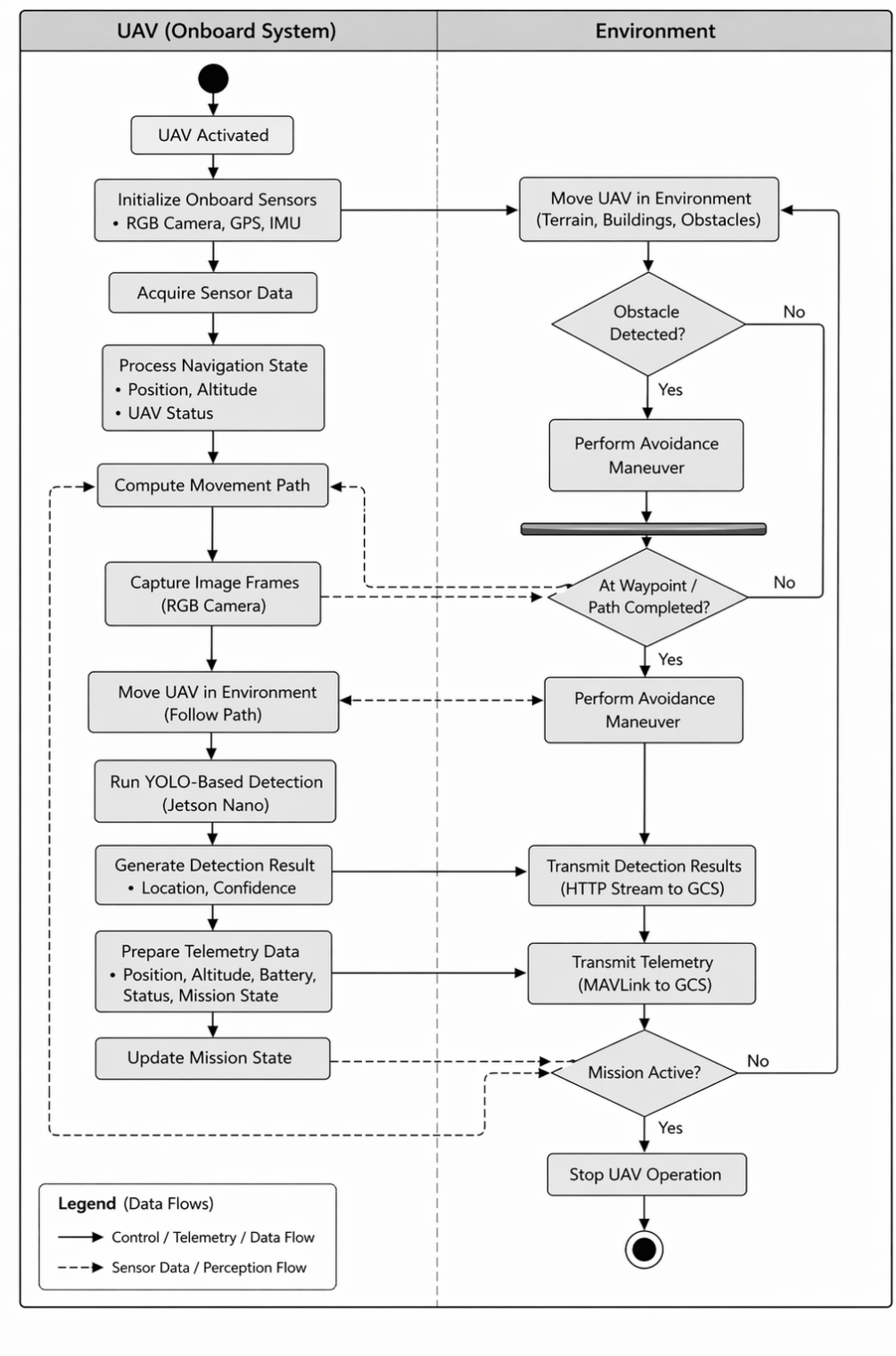


Figure 6: Activity Diagram of UAV Autonomous Operation

3.4.2 Class Diagram

Design models are used to visually represent the behavior and structure of the SwiftRescue UAV Swarm System. These models support understanding of system workflows, decision-making logic, and interactions between system components. They provide a structured way to analyze system behavior during mission execution and autonomous UAV operation [3].

The design models presented in this section are aligned with the system requirements and architectural design discussed in earlier chapters. Activity diagrams describe how missions are executed at the system level and how individual UAVs operate autonomously within a disaster environment. Class and state diagrams capture the structural and lifecycle dimensions of the system respectively.

At the core of the system is the **Mission** class, which represents a search and rescue operation and maintains mission-specific information such as search area, status, and execution timeline. Each mission is associated with multiple **UAV** instances that execute independently assigned waypoints within the mission area, supporting coordinated swarm operation through Mission Planner-based configuration [35].

The **UAV** class models an autonomous drone in the swarm and stores operational state information including position, battery level, and status. UAVs are composed of multiple **Sensor** objects representing onboard sensing devices such as RGB cameras, GPS, and IMU. Autonomous behavior within each UAV is managed by the **NavigationModule**, which handles ArduPilot-based [29] waypoint navigation and flight stability, and the **PerceptionModule**, which handles YOLOv8s-based victim detection [11]. Detection outputs are encapsulated in the **DetectionResult** class.

The **JetsonOrinNano** companion computer class plays a central role in onboard processing by executing the YOLOv8s perception pipeline [11] and encoding live video streams for HTTP transmission. It processes image frames captured from onboard sensors and generates real-time detection results on its integrated GPU, enabling onboard intelligence without relying on external processing systems [19, 17].

The **CommunicationModule** enables data exchange between the UAV and the Ground Control Station using two strictly separated communication channels. Telemetry data such as position, altitude, and battery status is transmitted using MAVLink [39], while detection results and live video streams are transmitted using HTTP-based communication [19]. This separation ensures efficient and non-interfering handling of control and perception data streams.

The **TelemetryData** class represents structured flight data generated during UAV operation, while detection outputs are managed through the **DetectionResult** class. These data structures support real-time monitoring and system feedback at the GCS without requiring external database systems.

Human interaction with the system is facilitated through the **GroundControlStation (GCS)**, which combines Mission Planner [35] for flight telemetry and control with a web-based detection dashboard for victim detection visualization. The **MissionOperator** represents the user supervising mission execution via the GCS interface.

In addition to defining structural relationships, the class diagram highlights a clear separation of concerns between navigation, perception, communication, and user interaction components. This modularity improves system maintainability and simplifies integration between hardware and software subsystems [29, 19]. The class-based design also supports scalability by allowing additional UAVs and sensors to be integrated with min-

imal changes to existing components, which is particularly important for swarm-based operations where multiple UAVs execute concurrently using independently assigned mission plans [8].

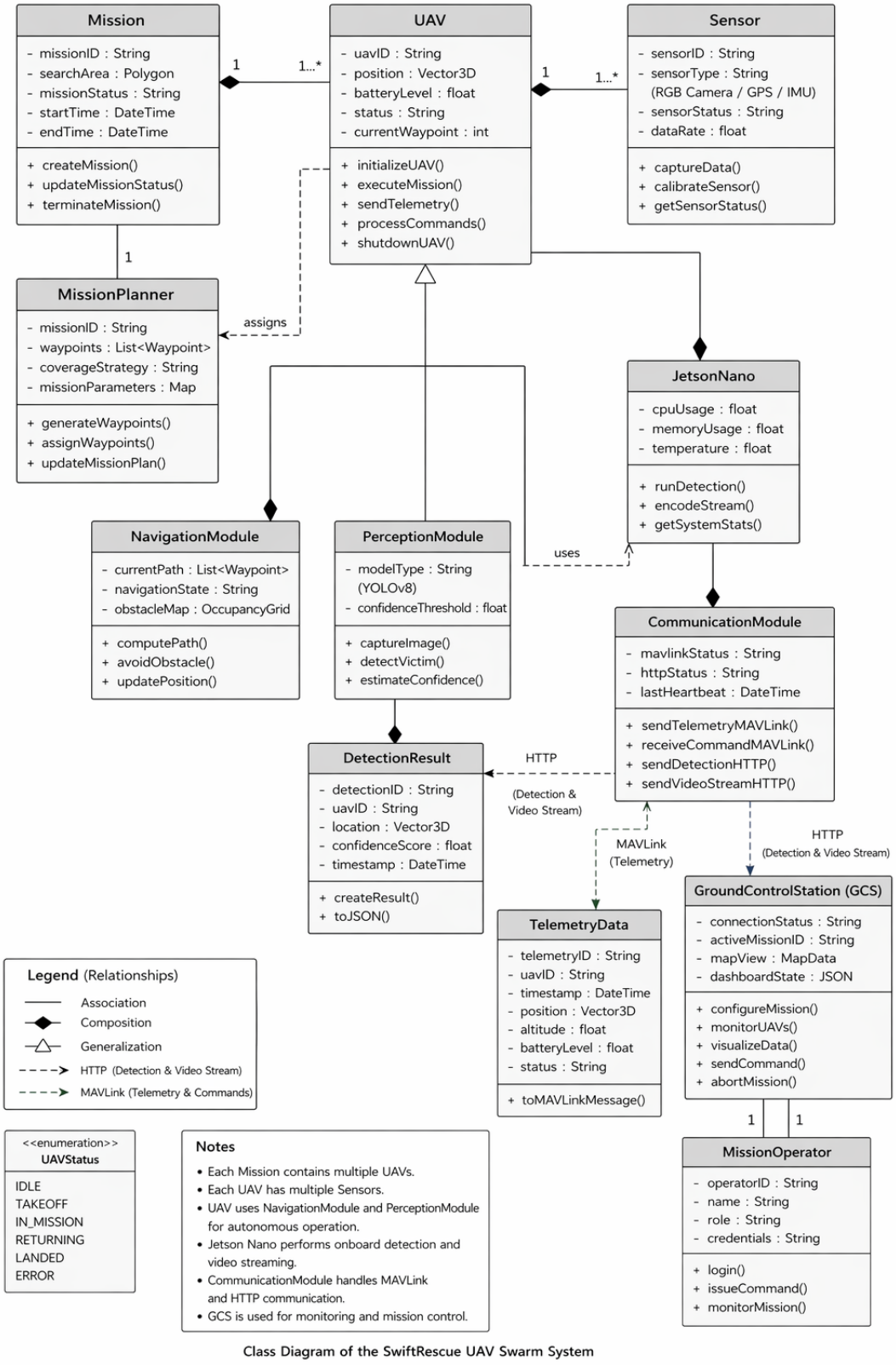


Figure 7: Class Diagram of the SwiftRescue UAV Swarm System

3.4.3 State Diagram

The state diagram represents the lifecycle of a UAV during a search and rescue mission in the SwiftRescue system. It illustrates how the UAV transitions between operational states based on mission commands, onboard processing, and external events [29]. The lifecycle begins in the *Idle* state, where the UAV awaits mission configuration from the Ground Control Station via Mission Planner [35], and progresses through mission activation, pre-flight checks, takeoff, waypoint navigation, victim detection, data transmission, and mission completion. Decision points such as detection triggers and abort conditions capture both normal and exceptional execution paths, ensuring safe and controlled system behavior.

During mission execution, the UAV follows its assigned waypoints under ArduPilot autopilot control [29] while performing continuous onboard perception. Image data from the onboard RGB camera is processed by the NVIDIA Jetson Orin Nano [19] using the fine-tuned YOLOv8s model [11] for real-time victim detection. When a detection event occurs, results are packaged and transmitted to the Ground Control Station via HTTP, while telemetry data such as position, altitude, and battery status is continuously transmitted using MAVLink [39]. This strict separation ensures efficient and non-interfering handling of perception and control data throughout all mission states.

The diagram also highlights structured control flow and clearly defined state transitions. Failure conditions trigger ArduPilot failsafe mechanisms [29], mission termination commands are issued via Mission Planner [35], and return-to-launch procedures are explicitly modeled to ensure operational safety. Overall, the state diagram provides a concise and accurate representation of UAV autonomy and supports validation of reliable mission execution across all defined operational states.

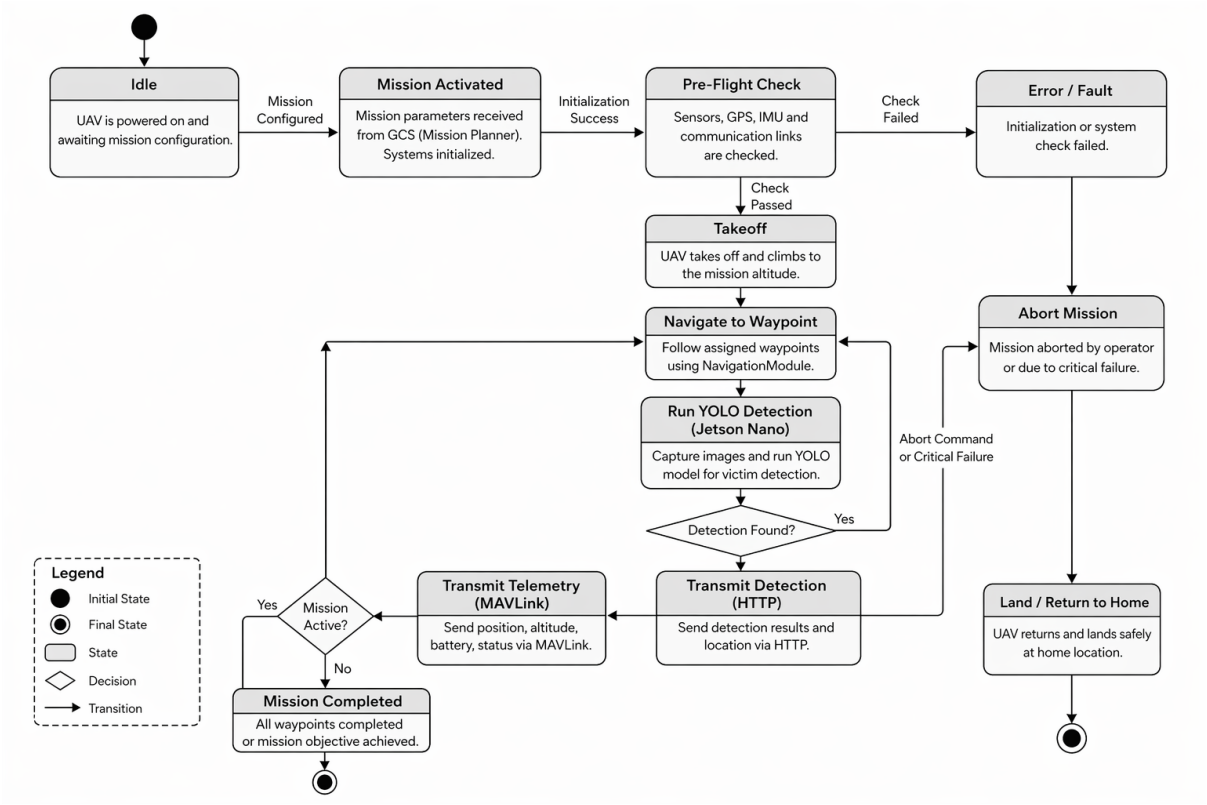


Figure 8: State Diagram of the UAV Mission Lifecycle in the SwiftRescue System

Chapter 4

Implementation and Testing

4.1 Simulation Implementation

This section describes the implementation of the simulation environment used to develop, integrate, and evaluate the SwiftRescue UAV Swarm System. It explains how different simulation components are combined to emulate realistic disaster scenarios and autonomous UAV operations. The simulation implementation serves as the primary validation platform for system architecture, perception pipelines, multi-UAV coordination, and communication mechanisms. By relying on simulation, the system can be tested extensively under controlled conditions before transitioning to hardware-based deployment.

4.1.1 Simulation Framework Overview

Simulation plays a critical role in the development and validation of the SwiftRescue UAV Swarm System [26]. Disaster response scenarios such as floods and collapsed infrastructure are inherently dangerous, unpredictable, and difficult to replicate in real-world testing environments. A simulation-based framework allows the system to be evaluated safely under controlled yet realistic conditions, enabling repeated experimentation without risking hardware damage or human safety [3]. It also allows developers to observe system behavior under extreme and rare conditions that would be impractical or unsafe to test physically. Moreover, simulation significantly reduces development cost and time by eliminating the need for repeated field trials during early system validation phases.

For SwiftRescue, simulation provides a virtual environment where autonomous UAV operation, mission coordination, perception pipelines, and communication mechanisms can be tested and refined before any real-world deployment [29, 26]. It enables rapid prototyping, performance evaluation, and systematic debugging of individual system components as well as their interactions. Simulation also supports scalability testing by allowing multiple UAVs to operate simultaneously in large disaster scenarios [8]. This approach ensures that the system remains consistent with the modular architecture defined in earlier chapters while maintaining flexibility for future extensions and real-world integration.

The simulation framework integrates physics-based environment modeling, realistic sensor emulation, and autonomous control execution to closely mirror real operational conditions [26, 30]. It allows controlled variation of mission parameters such as swarm size, search area, and obstacle density to analyze system behavior under different configurations. The framework also supports repeatable experimentation, which is essential for comparative performance evaluation and regression testing. By isolating system components within the simulation, developers can identify integration issues and performance bottlenecks more efficiently. Overall, the simulation framework provides a robust foundation for validating autonomous UAV behavior and system reliability prior to hardware implementation.

Within the simulation, perception is implemented using the fine-tuned YOLOv8s detection model [11] integrated as ROS 2 nodes [27]. Image data generated from simulated

UAV cameras in Gazebo [26] is processed in real time, allowing detection of victims and points of interest within the environment. This approach ensures that perception logic is validated within the same simulation communication and processing pipeline before transitioning to the HTTP-based streaming used in real hardware deployment.

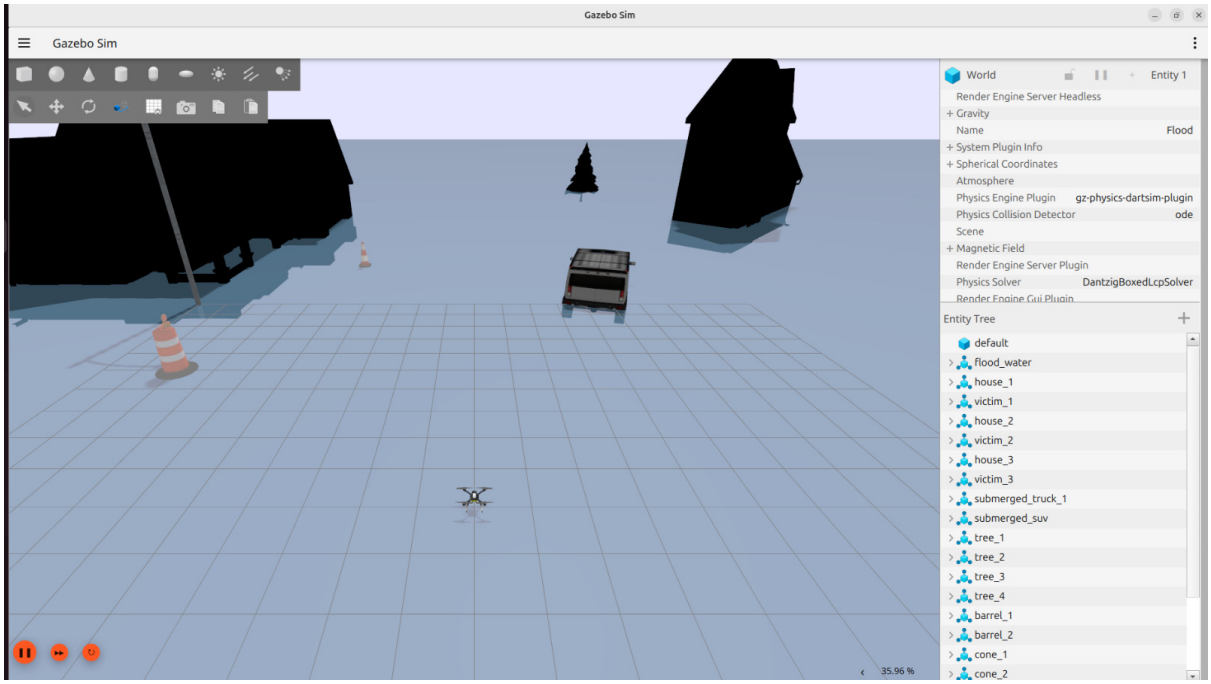


Figure 9: Gazebo-Based Flood Disaster Simulation Environment with UAV and Static Obstacles

The simulation framework integrates multiple software components to replicate real operational conditions. It supports the modeling of complex disaster environments, dynamic obstacles, victim placement, and UAV sensor feedback. This approach allows SwiftRescue to evaluate coordinated multi-UAV operation, navigation robustness, and real-time data exchange in scenarios that closely resemble real disaster situations [5]. It also provides visibility into system behavior under partial failures and communication delays, which are common in disaster response operations.

Additionally, simulation enables systematic testing under varying environmental parameters such as terrain layout, obstacle density, and mission scale. This flexibility is essential for validating system scalability and ensuring that multiple UAVs can operate effectively under different mission requirements without structural changes to the system [8]. By isolating and adjusting individual parameters, developers can analyze performance bottlenecks and improve system reliability in a controlled manner.

The simulation framework further supports repeatable experimentation, allowing identical mission scenarios to be executed multiple times for comparative evaluation. This repeatability is critical for objective performance analysis of navigation strategies, perception accuracy, and communication stability. The framework also enables incremental system integration, where individual components can be validated independently before full system-level testing. Through continuous simulation-based refinement, SwiftRescue achieves improved robustness and preparedness for real-world deployment scenarios.

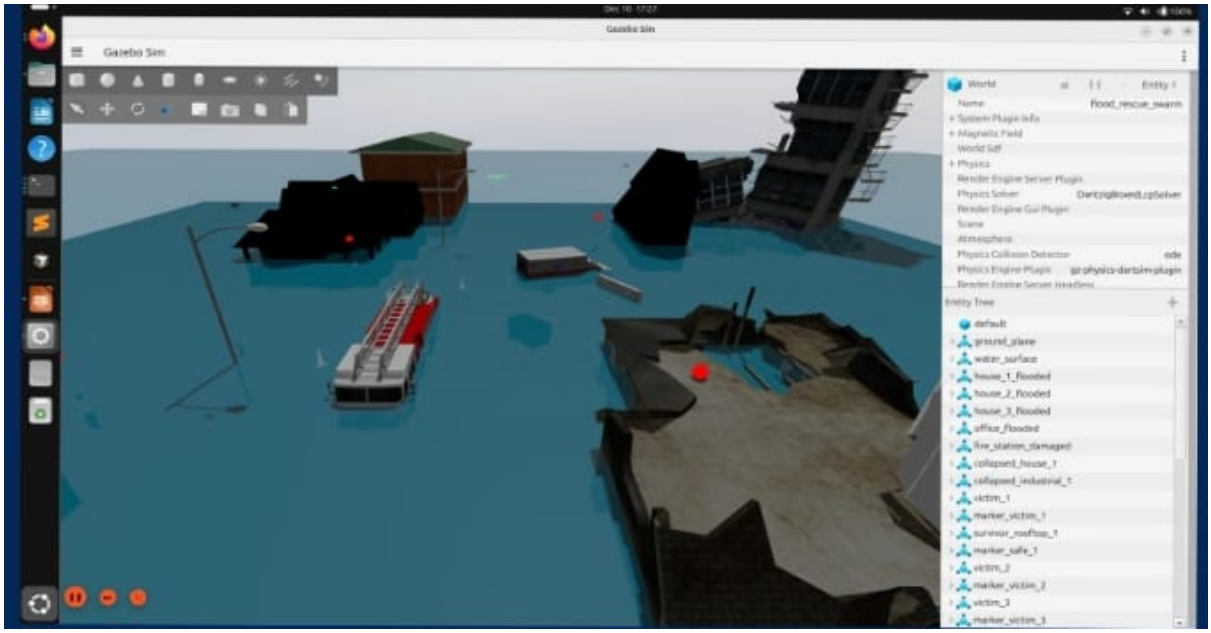


Figure 10: Complex Flooded Urban Disaster Scenario Simulated in Gazebo for Multi-UAV Operations

4.1.2 Gazebo Harmonic (LTS Version)

Gazebo Harmonic (LTS) is used as the primary physics-based simulation environment for the SwiftRescue UAV Swarm System [26]. It provides a stable and long-term supported platform for simulating robotic systems with realistic physics, sensor feedback, and environmental interactions. The selection of the LTS version ensures consistent behavior across simulation runs and compatibility with ROS 2 Jazzy [27] and ArduPilot SITL [30] frameworks used in this project.

Physics-Based Simulation Environment Gazebo Harmonic offers a robust physics engine that enables realistic modeling of UAV motion, gravity effects, collisions, and sensor interactions [26]. Within SwiftRescue, the physics engine is responsible for simulating UAV flight dynamics, obstacle interactions, and environmental constraints such as ground contact and object collisions. This allows UAV navigation and coordinated multi-UAV behavior to be evaluated under conditions that closely resemble real-world physical behavior, without the risks associated with hardware testing [21].

Disaster Scenario Modeling The simulation environment is configured to represent disaster scenarios such as flooded urban areas and partially collapsed infrastructure. Static and dynamic entities including buildings, submerged vehicles, debris, trees, and human victim models are placed within the simulated world. These elements create realistic spatial constraints for UAV navigation and perception. Victim models are strategically distributed across the environment to support evaluation of the YOLOv8s detection pipeline [11] and area coverage performance using multiple UAVs.

Environment Realism and Limitations While Gazebo provides a high degree of environmental realism [26], certain real-world factors are simplified to maintain simula-

tion stability and computational efficiency. Weather effects such as wind turbulence, rain, and water flow dynamics are approximated rather than fully modeled. Sensor noise and latency are represented at an abstract level to enable repeatable testing. These controlled simplifications allow SwiftRescue to focus on validating system behavior, navigation logic, and data flow while acknowledging the gap between simulation and real-world deployment.

Role of Gazebo in the SwiftRescue Project In SwiftRescue, Gazebo Harmonic serves as the foundational environment where all system components are integrated and validated [26, 28]. It acts as the execution layer for UAV operations, enabling interaction between perception modules, ROS 2 communication nodes [27], ArduPilot SITL flight control [30], and the Ground Control Station [35]. The simulation environment supports rapid prototyping, repeatable testing, and performance evaluation before transitioning to hardware-based implementation phases. This integration ensures that perception, control, and communication pipelines remain consistent across simulation experiments.

4.1.3 ROS 2 Jazzy Integration

ROS 2 Jazzy serves as the core middleware layer for internal communication and coordination within the SwiftRescue simulation environment [27]. It enables structured, real-time data exchange between distributed simulation components including UAV nodes, perception processing modules, and GCS visualization services during Gazebo-based testing. The use of ROS 2 ensures modularity, scalability, and reliable inter-process communication within the simulation [27, 28]. It is important to note that ROS 2 is used exclusively within the simulation environment; in real-world hardware deployment, MAVLink [39] handles flight telemetry and control while HTTP handles detection data streaming from the NVIDIA Jetson Orin Nano [19].

Middleware Role in Simulation ROS 2 operates as an abstraction layer between application-level UAV logic and the underlying simulation communication infrastructure [27]. By leveraging a Data Distribution Service (DDS)-based publish–subscribe model, ROS 2 enables decentralized communication without relying on a central broker. This design improves fault tolerance and allows multiple UAV nodes to exchange sensor data, detection results, and mission state information efficiently during simulation [27].

Within the simulation, the YOLOv8s perception module [11] is implemented as ROS 2 nodes, processing simulated camera data from Gazebo in real time for victim detection. This ensures that perception logic is thoroughly validated within the simulation communication pipeline before hardware deployment.

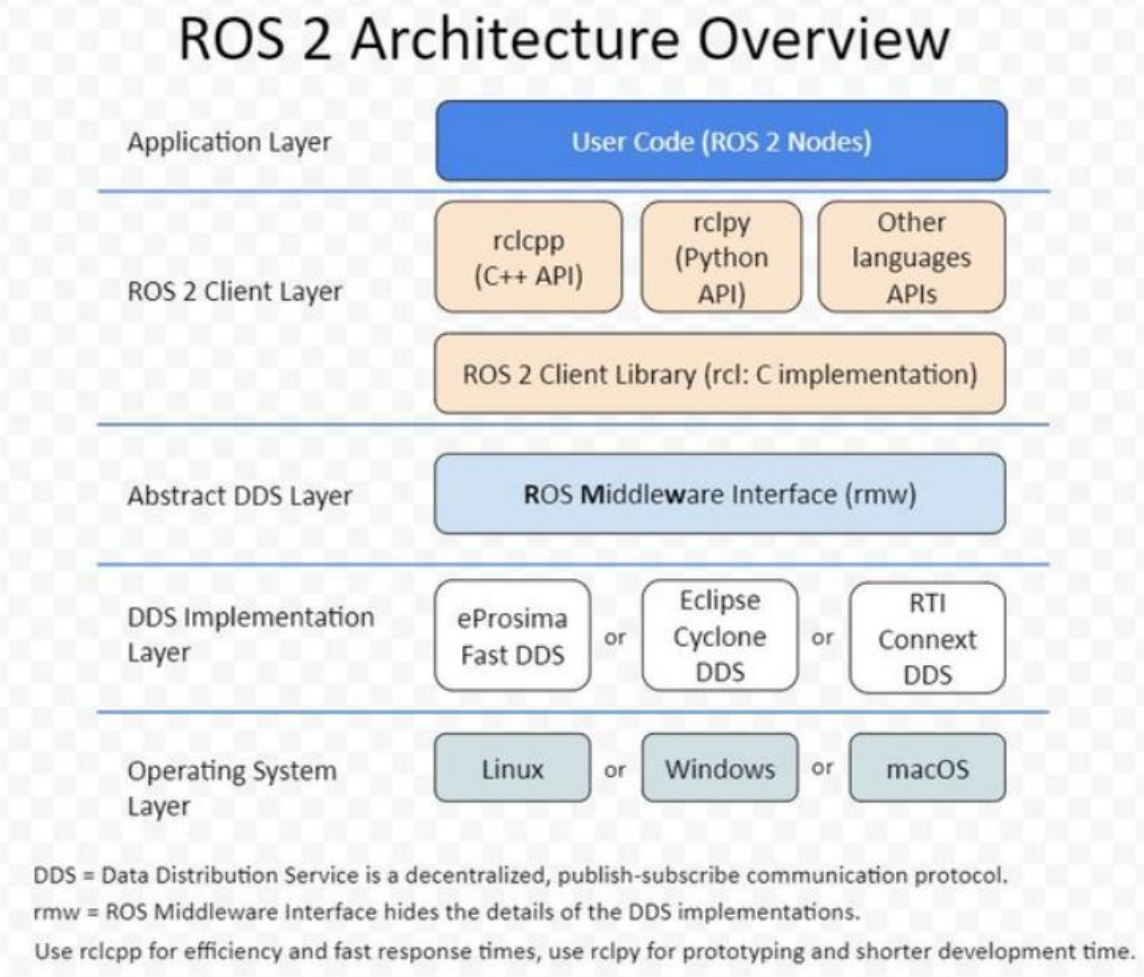


Figure 11: ROS 2 Jazzy Architecture Illustrating Node Execution and DDS-Based Middleware Layers

ROS 2 Nodes, Topics, and Services Within SwiftRescue's simulation environment, ROS 2 nodes represent functional components such as UAV navigation modules, YOLOv8s perception modules [11], multi-UAV coordination logic, and GCS visualization services [27]. These nodes communicate primarily through ROS 2 topics for continuous data streams, including telemetry updates, simulated sensor outputs, and victim detection results. ROS 2 services are used for request–response interactions such as mission configuration, system status queries, and controlled mission termination within the simulation.

This communication model ensures loose coupling between simulation components, enabling independent development and testing of individual modules. It also supports real-time data synchronization across multiple simulated UAVs and the GCS, maintaining consistency in mission state and perception outputs [27].

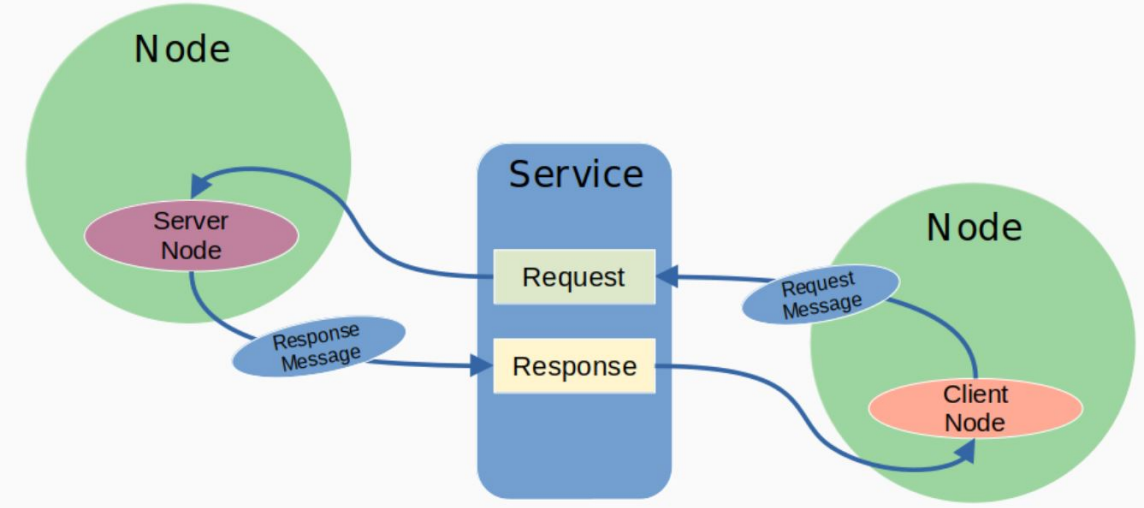


Figure 12: ROS 2 Communication Flow Showing Node Interaction via Topics and Services

Integration with Gazebo ROS 2 is tightly integrated with the Gazebo Harmonic simulation environment through ROS–Gazebo bridges [28, 31]. This integration allows simulated UAV sensors, such as cameras and IMU units, to publish data directly as ROS 2 topics. Similarly, control commands generated by ROS 2 nodes are consumed by the simulated UAV models within Gazebo [26].

This bidirectional integration enables SwiftRescue to evaluate perception, navigation, and multi-UAV coordination logic in real time under realistic simulation conditions. By maintaining consistent ROS 2 interfaces throughout simulation, the system ensures that validated logic can be reliably transitioned to hardware deployment with minimal architectural changes.

4.1.4 ArduPilot Simulation (SITL) Overview

ArduPilot serves as the core autopilot software within the SwiftRescue simulation framework, providing realistic flight control, navigation logic, and vehicle state management for simulated UAVs [29]. It is responsible for translating high-level mission commands into low-level control actions, ensuring stable flight behavior and adherence to mission constraints. By integrating ArduPilot in a Software-in-the-Loop (SITL) configuration [30], SwiftRescue achieves accurate emulation of real UAV dynamics without requiring physical hardware. This integration allows the system to closely mirror real-world UAV behavior while retaining the flexibility and safety benefits of simulation-based development.

Role of ArduPilot in UAV Control Within the SwiftRescue system, ArduPilot acts as the primary flight controller for each simulated UAV [29]. It manages essential control loops including attitude stabilization, altitude control, speed regulation, and waypoint navigation [21]. Sensor inputs such as GPS, IMU, and range data — simulated within Gazebo [26] — are processed by ArduPilot to estimate the UAV's state and generate appropriate control outputs. This enables the UAV system to exhibit realistic flight behavior while remaining fully autonomous during mission execution.

ArduPilot also provides built-in support for safety mechanisms such as failsafe handling, mode switching, and controlled mission termination [29]. These features are critical

for validating emergency behaviors, such as mission abort and return-to-launch scenarios, within a safe simulation environment before hardware deployment.

Software-in-the-Loop (SITL) Concept Software-in-the-Loop (SITL) allows ArduPilot to run as a software process on a host machine while interacting with simulated sensors and actuators instead of real hardware [30]. In this configuration, Gazebo [26] supplies simulated sensor data, while ArduPilot computes control outputs exactly as it would on a physical Pixhawk flight controller [20]. This approach enables rapid testing, debugging, and validation of UAV control logic without the cost or risk associated with real-world flight testing.

Autopilot Behavior in Simulation During simulation, ArduPilot operates in conjunction with ROS 2 [27] and Gazebo [26] to execute autonomous missions. High-level mission commands and telemetry are exchanged through the MAVLink protocol [39], allowing seamless communication between the autopilot and the Ground Control Station via Mission Planner [35]. This layered interaction separates high-level decision-making from low-level flight control while maintaining synchronized system behavior.

Additionally, ArduPilot continuously monitors flight state variables such as position error, velocity, and attitude deviation to maintain control stability throughout mission execution [29]. Feedback control loops operate at high frequency, enabling rapid response to navigation updates and simulated environmental disturbances. This behavior ensures that UAV trajectories remain consistent with planned paths while preserving flight safety and control accuracy.

A key advantage of the SITL configuration within SwiftRescue is its direct consistency with the hardware deployment stack. Since ArduPilot runs identical firmware logic in both simulation and on the physical Pixhawk flight controller [20], behaviors validated in SITL — including waypoint execution, failsafe triggering, and flight mode transitions — transfer directly to hardware without requiring significant reconfiguration. This consistency significantly reduces integration risk during the simulation-to-hardware transition and was confirmed during the hardware testing phase described in Section 4.5.2 [29, 30].

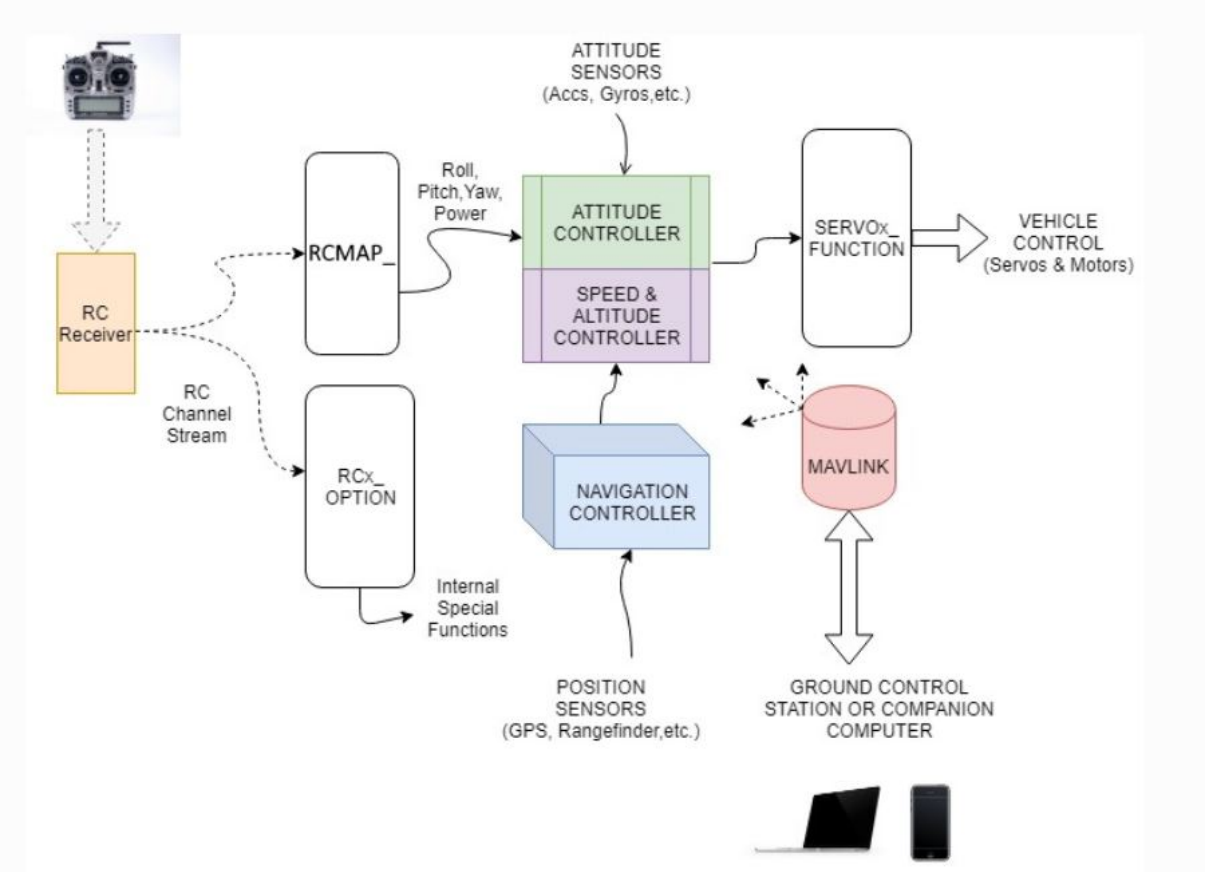


Figure 13: ArduPilot Autopilot Architecture and Control Flow in Software-in-the-Loop (SITL) Simulation

4.1.5 UAV and Sensor Simulation

The SwiftRescue simulation framework includes detailed modeling of UAV platforms and their onboard sensors to emulate realistic operational behavior [26]. Each UAV is represented as a multirotor model within Gazebo, configured with appropriate physical properties such as mass, inertia, and propulsion characteristics [21, 22]. This allows the simulated UAVs to respond realistically to control inputs generated by the ArduPilot autopilot [29] during mission execution.

To support perception and navigation tasks, multiple sensors are simulated on each UAV. An RGB camera is mounted on the UAV to provide visual input for victim detection and environmental awareness. The camera generates continuous image frames that resemble real aerial imagery in flooded and cluttered environments. The image data is processed using the fine-tuned YOLOv8s detection model [11] implemented as ROS 2 nodes within the simulation, enabling real-time identification of victims and validation of the perception pipeline under controlled conditions. In addition, GPS and IMU sensors are simulated to provide position estimates, orientation data, and motion feedback required for ArduPilot-based navigation and stabilization [25].

Sensor data generated within Gazebo is published through ROS 2 topics [27], enabling downstream processing by perception, navigation, and communication modules within the simulation. This sensor simulation pipeline allows SwiftRescue to validate the YOLOv8s perception algorithm [11], telemetry handling, and sensor data flow under re-

alistic conditions before transitioning to onboard processing on the NVIDIA Jetson Orin Nano [19] in hardware deployment.

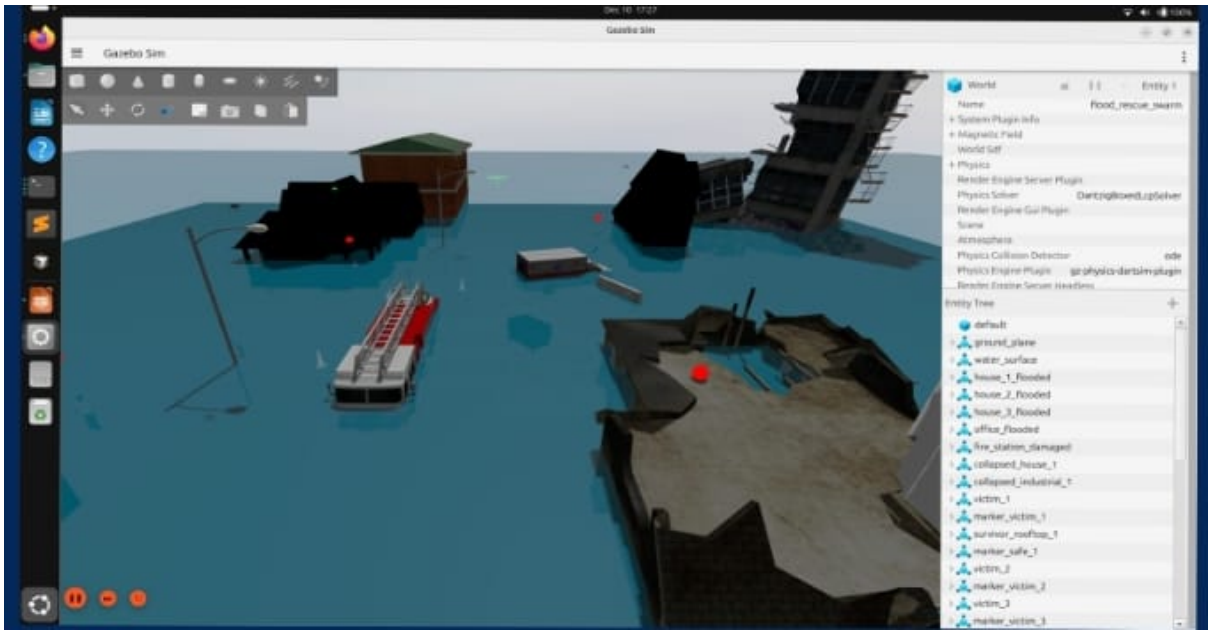


Figure 14: Simulated UAV Camera View Capturing Flooded Disaster Environment in Gazebo

4.1.6 Multi-UAV Swarm Simulation Setup

To evaluate swarm-based operations, SwiftRescue supports the simulation of multiple UAVs operating concurrently within the same disaster environment [8, 26]. Each UAV instance is spawned with a unique namespace and assigned a unique MAVLink system identifier [39], allowing independent sensor streams, control commands, and telemetry topics to be managed simultaneously. This setup enables scalable experimentation with varying swarm sizes and mission configurations.

Coordination among UAVs is achieved through GCS-based mission planning via Mission Planner [35], where each UAV is assigned an independent waypoint-based mission plan enabling distributed and non-overlapping area coverage. Telemetry and detection data from individual UAVs are aggregated at the GCS, providing a unified operational view of all active UAVs and their detection outputs.

The multi-UAV simulation setup allows SwiftRescue to analyze system behavior under different environmental layouts and mission scales, validating coordinated autonomous operation [3]. It enables observation of area coverage patterns and navigation performance under realistic constraints. By simulating multiple UAVs simultaneously, the framework helps identify potential scalability issues related to communication load, data synchronization, and control stability [8]. It also supports fault-tolerance evaluation when individual UAVs experience delays or reduced functionality, confirming that remaining UAVs continue their assigned missions independently.

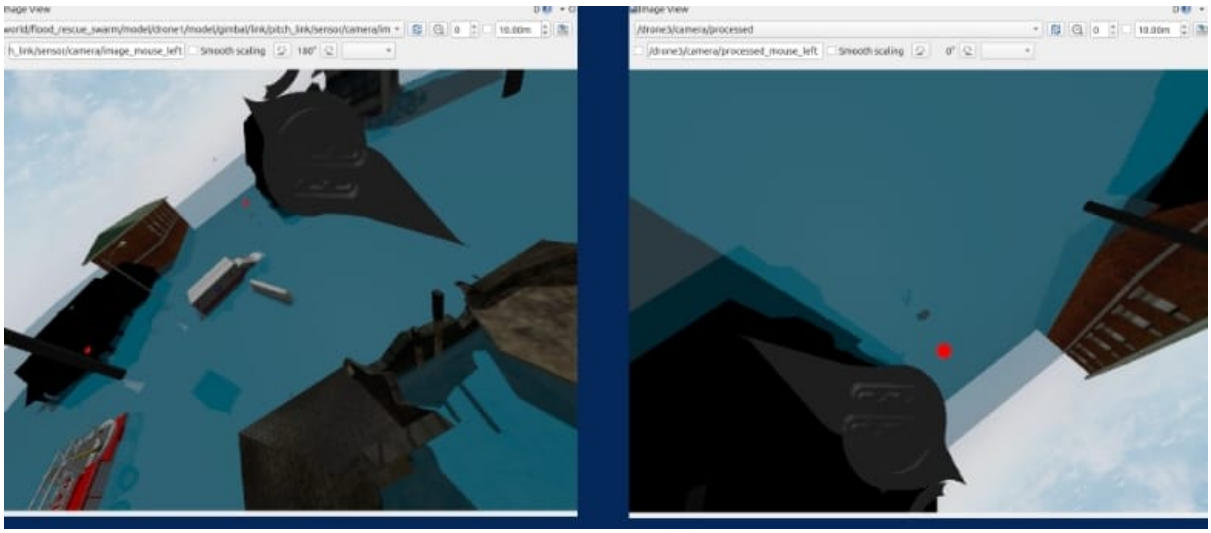


Figure 15: Multi-UAV Swarm Simulation Setup in Gazebo Flood Scenario

4.1.7 Mission Execution and Operator Interaction Workflow

Mission execution in the SwiftRescue simulation framework follows a structured workflow integrating autonomous UAV behavior with human-in-the-loop supervision through the Ground Control Station [35]. Each mission begins with initialization at the GCS, where the operator defines mission parameters such as search area, UAV allocation, and waypoint assignments using Mission Planner [35]. Once confirmed, each UAV receives its independently assigned mission plan and is deployed into the simulated environment, operating autonomously under ArduPilot control [29].

During mission execution, UAVs navigate the environment, collect sensor data, and generate telemetry and detection outputs. ROS 2 topics [27] are used for internal simulation data exchange between Gazebo and processing nodes, while MAVLink [39] handles communication between ArduPilot SITL and the Ground Control Station. The GCS visualizes UAV positions, flight paths, mission status, and detected points of interest in real time through Mission Planner [35].

Operator interaction remains supervisory throughout mission execution, where high-level commands such as waypoint updates, mission adjustments, or abort signals can be issued through the GCS [35]. The mission continues until completion criteria are met or termination is triggered. Upon completion or abort, ArduPilot failsafe mechanisms [29] are engaged, UAVs safely halt operations, and all mission data is preserved locally at the GCS for analysis. This workflow demonstrates reliable integration of autonomous operation with structured operator control in simulation.

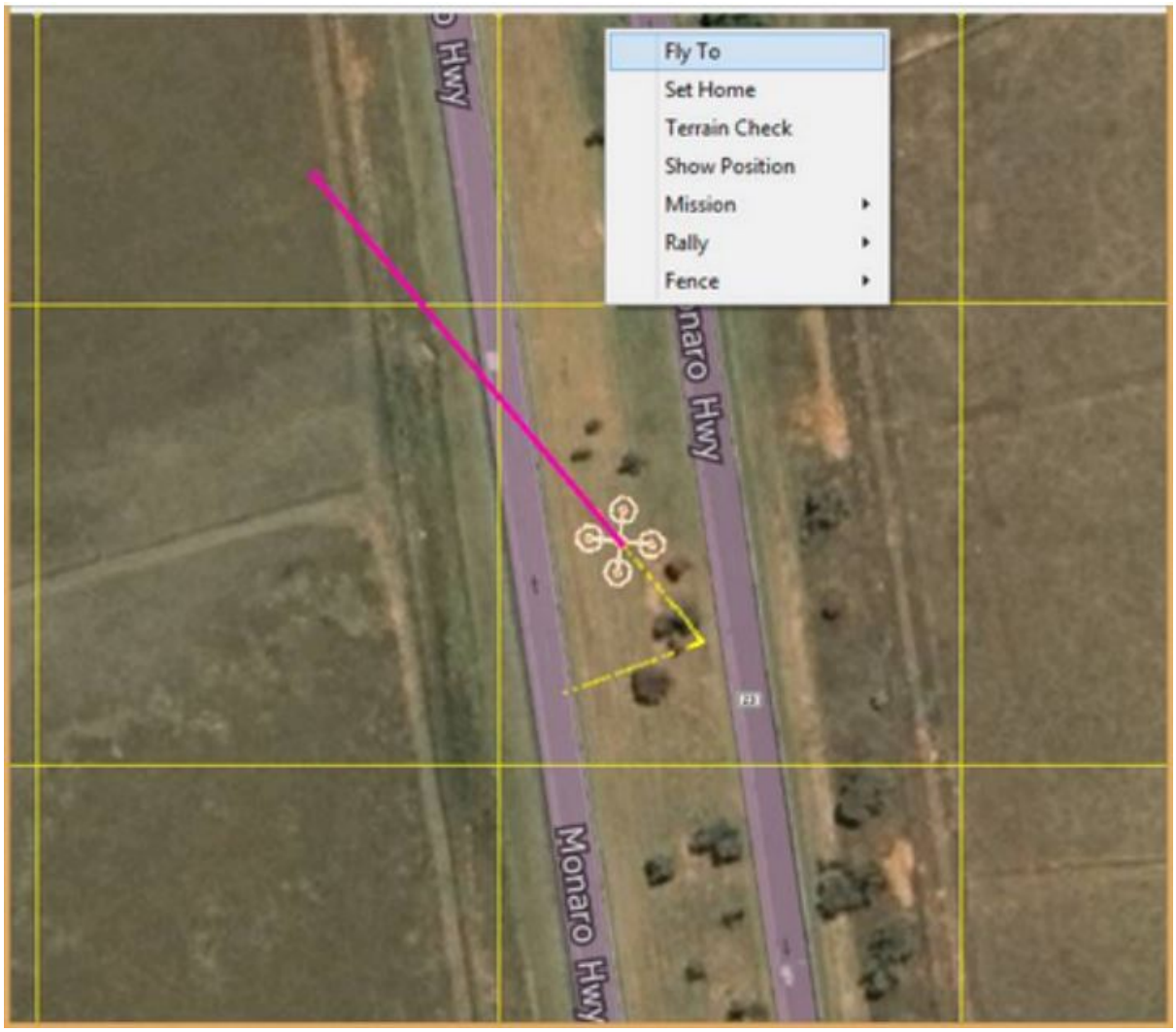


Figure 16: Operator-to-UAV interaction via GCS using high-level commands such as "Fly To"

4.1.8 Simulation Testing and Validation

Simulation testing is a critical component of the SwiftRescue development process, enabling systematic validation of system functionality, robustness, and integration prior to real-world deployment [26, 3]. Given the complexity and safety risks associated with disaster response scenarios, simulation provides a controlled environment where autonomous UAV behavior, coordinated multi-UAV operation using GCS-based mission planning, and data exchange mechanisms can be evaluated repeatedly without risk to hardware or human operators.

Multiple disaster scenarios were executed within the Gazebo simulation environment [26], varying parameters such as environment layout, obstacle density, water coverage, and UAV deployment configurations. These scenarios were designed to reflect realistic flood-affected areas containing partially submerged structures, debris, and limited navigable space. By adjusting these parameters, the system was tested under different operational conditions to observe UAV navigation behavior, sensor data consistency, and communication reliability.

During each simulation run, telemetry data including UAV position, orientation, and battery state was continuously generated and transmitted to the Ground Control Station via MAVLink [39]. Detection outputs produced by the YOLOv8s perception pipeline [11] were also logged and visualized in real time through the GCS detection dashboard. These data streams were used to verify correct message flow between UAVs, onboard processing, and Ground Control Station components, ensuring that no critical data was lost during transmission. ROS 2 topics [27] were used for internal simulation data exchange, while MAVLink [39] handled communication between ArduPilot SITL [30] and the Ground Control Station. Particular attention was given to validating the stability of the communication pipeline when multiple UAVs were active simultaneously.

Mission logs were recorded throughout simulation execution, capturing key events such as UAV launch, navigation updates, detection events, and mission termination. Analysis of these logs allowed verification of mission state transitions and confirmation that UAVs responded correctly to control commands issued via Mission Planner [35], including mission abort scenarios. Extended simulation runs were performed to observe ArduPilot autopilot stability [29] over time and to ensure that UAVs maintained controlled flight behavior without unexpected state transitions.

In addition to quantitative data inspection, qualitative validation was performed through visual observation of camera feeds and GCS dashboards. Live camera views from Gazebo [26] were examined to ensure that sensor perspectives remained consistent with UAV motion and environmental layout. The GCS interface was used to monitor UAV trajectories, mission progress, and detected points of interest, providing confidence that system visualization accurately reflected the underlying simulation state.

Simulation testing also enabled early identification of system limitations and integration challenges. Simplifications in environmental physics, sensor noise modeling, and communication latency were observed when compared to real-world conditions. For example, weather effects such as wind disturbances and signal interference were not fully modeled, and sensor readings were less noisy than expected in physical deployments. These limitations were documented to inform hardware-level testing and system refinement described in Section 4.5.

Despite these constraints, simulation-based testing proved effective for validating the overall system architecture, autonomous operation logic, and data flow consistency of SwiftRescue. The results demonstrate that the system can reliably operate multiple UAVs, process sensor data through the YOLOv8s pipeline [11], and support real-time mission monitoring within a simulated disaster environment. In addition, the simulation results helped identify performance bottlenecks, communication overhead, and coordination delays under increased UAV counts. These observations support informed design decisions and guide future optimization of system parameters. Overall, simulation testing serves as a critical risk-reduction step, ensuring system readiness before transitioning to hardware-based validation.

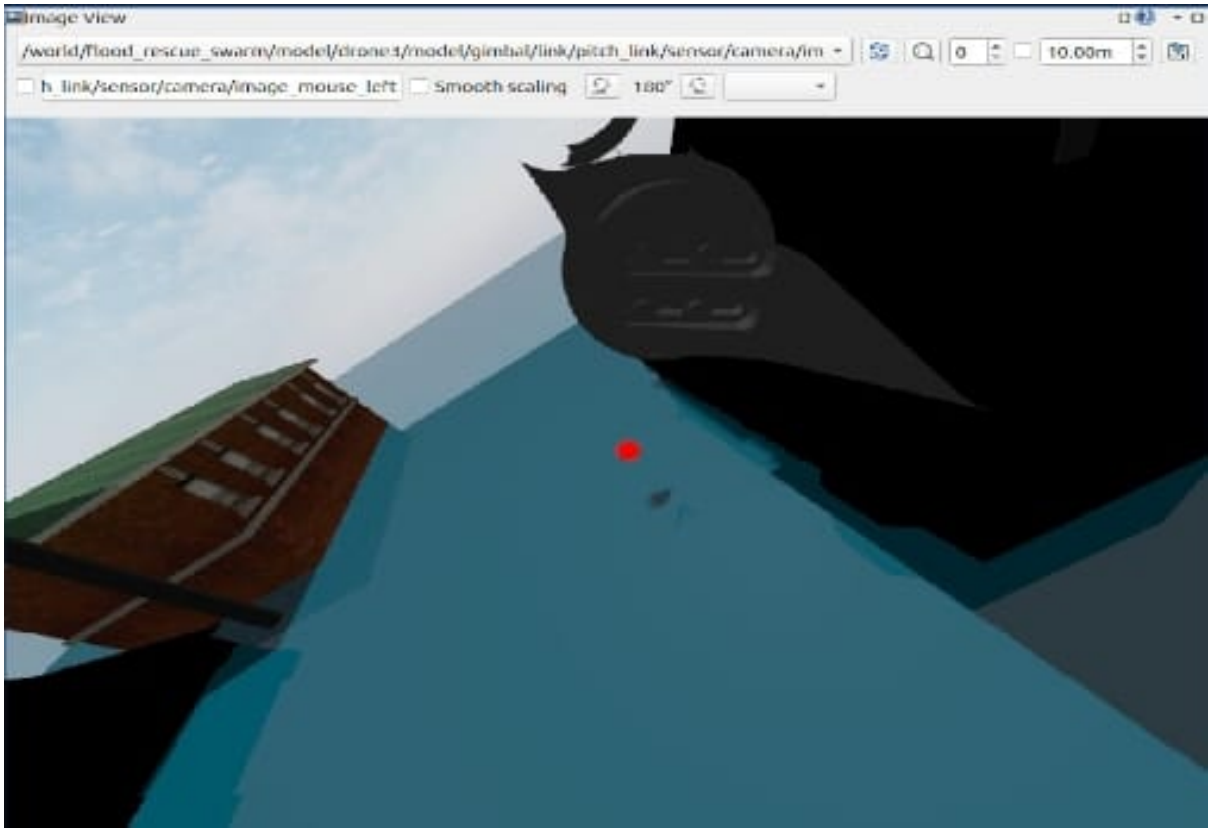


Figure 17: Ground Control Station View Showing Mission Execution and Detected Points in Simulation

4.1.9 Summary of Simulation Implementation

This section presented the simulation implementation of the SwiftRescue UAV Swarm System, covering environment modeling, UAV and sensor simulation, coordinated multi-UAV operation, mission execution, and systematic testing. The simulation framework integrates Gazebo Harmonic [26], ROS 2 Jazzy [27], and ArduPilot SITL [30] to provide a realistic and modular platform for validating autonomous UAV operations in disaster scenarios.

Through simulation, key system components including the YOLOv8s perception pipeline [11], ArduPilot navigation logic [29], MAVLink communication mechanisms [39], and Ground Control Station visualization [35] were evaluated in a safe and repeatable manner. Multi-UAV operation, data flow consistency, and mission-level coordination through GCS-based waypoint assignment were demonstrated across multiple simulated scenarios under varying environmental and operational conditions.

The simulation environment enabled controlled experimentation with different mission parameters and UAV counts, allowing early identification of integration challenges and system limitations while maintaining architectural consistency with earlier design chapters. Although simulation-based testing has inherent limitations compared to real-world deployment [26], it provides a reliable foundation for validating system architecture and autonomous behavior. The insights gained from this implementation directly support the design and evaluation of the algorithms and hardware integration mechanisms discussed in the following sections.

4.2 Algorithm Design and Implementation

This section presents the core algorithms implemented in the SwiftRescue system, covering real-time perception, autonomous flight control, data communication, and multi-UAV coordination. The primary objective of these algorithms is to enable reliable victim detection, efficient data handling, and seamless integration between onboard processing on the NVIDIA Jetson Orin Nano [19] and the Ground Control Station. A fine-tuned YOLOv8s model [11] is utilized for perception tasks, optimized for aerial flood imagery and deployed at the edge to achieve real-time inference without reliance on external servers. The algorithmic design emphasizes real-time performance, modularity, and scalability across both simulation and hardware deployment contexts.

The algorithmic framework is organized into seven components. The first covers the YOLOv8s training pipeline, describing dataset preparation, pseudo-labeling annotation, and two-phase model training. The second presents the real-time detection and HTTPS streaming pipeline executed on the Jetson Orin Nano [19]. The third describes ArduPilot-based [29] navigation and flight control logic including waypoint execution, telemetry transmission via MAVLink [39], and failsafe handling. The fourth covers the multi-UAV coordination strategy through Mission Planner [35], where independent mission plans are assigned per UAV under unified GCS supervision. The fifth addresses the dual-channel communication architecture separating MAVLink telemetry from HTTPS detection streaming [48]. The sixth details HTTPS-secured data transmission using TLS encryption. The seventh presents the complete end-to-end system pipeline integrating all components from initialization through mission execution to post-mission storage.

A central design principle across all components is the strict separation between the flight control subsystem and the perception subsystem. The Pixhawk flight controller [20] running ArduPilot [29] manages autonomous navigation and safety response entirely independently of perception outcomes. The Jetson Orin Nano [19] operates concurrently as an independent computing platform, executing YOLOv8s inference [11] and managing HTTPS detection streaming without imposing any load on the flight controller. This separation ensures that disruptions in the perception pipeline do not affect flight stability or failsafe responsiveness, and vice versa. Since ArduPilot runs identical firmware logic in both SITL simulation [30] and on the physical Pixhawk, behaviors validated in simulation transfer directly to hardware deployment with minimal reconfiguration [29].

The perception algorithm reflects key decisions driven by the operational context of aerial flood disaster response. YOLOv8s [11] was selected for its favorable balance between inference speed and detection accuracy on the Jetson Orin Nano GPU [17]. Training was conducted in two phases: a baseline phase establishing detection performance on the custom 472-image dataset, and a UAV-optimization phase applying aerial-specific augmentations including mosaic composition, HSV adjustment, rotation, and scaling to improve generalization under variable altitude and lighting conditions typical of flood rescue imagery [11, 18]. The UAV-optimized model was selected for deployment due to its improved robustness under real-world aerial conditions. The following subsections present each algorithm in structured pseudocode format with supporting implementation details [3].

4.2.1 Perception Algorithm (YOLOv8s-Based Detection)

Algorithm 1 YOLOv8s-Based Victim Detection Training Pipeline

Initialize Dataset
Initialize dataset D (flood aerial images, 520 total collected)
Extract dataset into working directory

Dataset Preparation
Create YOLO directory structure:
  images/train, images/val
  labels/train, labels/val
Shuffle dataset samples randomly
Split dataset: 416 training / 104 validation (80:20)

Auto-Labeling
Load pre-trained YOLOv8s model [11]
for each image in dataset **do**
  Perform auto-labeling (pseudo-labeling)
  Generate bounding boxes and confidence scores
end for

Label Cleaning
Remove unmatched image-label pairs
Remove confidence values from label files
Keep only class "person"
Remove empty labels
Final dataset: 380 training / 92 validation images

Phase 1 – Baseline Model Training
Generate dataset configuration file (data.yaml)
Train YOLOv8s baseline (epochs = 30, image size = 640, batch = 8)
Result: mAP@0.5 = 0.893, Precision = 0.836, Recall = 0.810

Phase 2 – UAV-Optimized Fine-Tuning
Apply UAV-specific augmentations:
  HSV adjustment, rotation, scaling, translation, flipping
  mosaic and mixup
Increase image size to 960
Train for 80 epochs
Result: mAP@0.5:0.95 = 0.572, Precision = 0.782, Recall = 0.799

Model Selection and Output
Select UAV-optimized model for deployment
Rationale: improved generalization for real-world aerial conditions
Export best weights as `flood_victim_uav.pt`

4.2.2 Detection Pipeline and Processing Workflow

Algorithm 2 Real-Time Detection and Streaming Pipeline (NVIDIA Jetson Orin Nano)

Initialization
Load fine-tuned YOLOv8s model (`flood_victim_uav.pt`) [11]
Initialize CUDA-enabled GPU on Jetson Orin Nano [19]
Configure CSI camera (1280×720, 30 FPS)
Initialize HTTP/HTTPS streaming server [48]

while system is active **do**
Frame Acquisition
Capture frame from onboard CSI camera
Inference
Perform YOLOv8s inference on Jetson Orin Nano GPU [11]
Detection Processing
Extract bounding boxes, labels, and confidence scores
Filter detections (class = person only)
Count detected victims in current frame
Visualization
Annotate frame with bounding boxes and confidence scores
Performance Monitoring
Compute FPS and system uptime
Data Update
Update latest frame and detection statistics
Streaming
Encode annotated frame into JPEG format
Stream via HTTPS endpoint (/stream) [48]
Send detection metadata via API endpoint (/stats)
end while

4.2.3 Navigation and Flight Control Logic (ArduPilot)

Algorithm 3 UAV Navigation and Flight Control using ArduPilot

Initialization
Power on UAV and initialize Pixhawk flight controller [20]
Establish MAVLink connection with GCS (Mission Planner) [39, 35]
Load independently assigned mission waypoints

Pre-Flight Checks
Verify GPS lock and sensor calibration
Check battery status and system health
Arm UAV system via Mission Planner

Takeoff Phase
Execute automatic takeoff to predefined mission altitude

Autonomous Navigation
while mission is active **do**
  Navigate through assigned waypoint sequence [29]
  Continuously update UAV position and altitude
  Maintain flight stability using ArduPilot control loops
  Telemetry Transmission
  Collect real-time data (position, altitude, battery, status)
  Transmit telemetry to GCS via MAVLink protocol [39]
  Command Handling
  if new command received from GCS via MAVLink **then**
    Update mission parameters or execute operator command
  end if
end while

Mission Completion
Execute return-to-launch (RTL) [29]
Perform safe landing
Disarm UAV system

4.2.4 Multi-UAV Coordination Strategy (Mission Planner-Based)

Algorithm 4 Multi-UAV Coordination using Mission Planner

Initialization
Connect multiple UAVs to GCS (Mission Planner) [35]
Assign unique MAVLink system ID to each UAV [39]
Establish independent MAVLink communication link per UAV

Mission Assignment
Partition search area into non-overlapping zones per UAV
Define independent waypoint plan per UAV in Mission Planner [35]
Upload independently assigned mission plan to each UAV

Coordinated Execution
for each UAV in swarm **do**
  Start autonomous mission execution
  Follow independently assigned waypoint sequence [29]
end for

Parallel Operation
while mission is active **do**
  Monitor all UAV positions on GCS map [35]
  Verify safe UAV separation via mission area partitioning
  Each UAV operates independently on its assigned coverage zone
  Data Handling
  Receive telemetry from all UAVs via MAVLink [39]
  Display all UAV states and mission status on GCS
end while

Mission Termination
for each UAV **do**
  Issue return-to-launch or landing command via Mission Planner [35]
end for
Store all mission logs locally at GCS

4.2.5 Data Communication and Processing Architecture

Algorithm 5 UAV Data Communication and Processing Workflow

Initialization
Initialize MAVLink communication between Pixhawk and GCS [39]
Initialize HTTP/HTTPS server on NVIDIA Jetson Orin Nano [19]
Establish wireless network connection (WiFi / 4G)

Telemetry Communication (MAVLink)
while UAV is active **do**
  Collect telemetry from Pixhawk flight controller [20]:
    position, altitude, battery level, UAV status
  Encode using MAVLink protocol [39]
  Transmit telemetry to Mission Planner at GCS [35]
end while

Detection Data Communication (HTTP/HTTPS)
while detection system is active **do**
  Retrieve processed frame from YOLOv8s detection module [11]
  Retrieve detection results (victim count, bounding boxes)
  Encode annotated frame into JPEG format
  Stream via HTTPS endpoint (/stream) [48]
  Package detection metadata
  Send metadata via API endpoint (/stats)
end while

Ground Control Processing
Receive flight telemetry via MAVLink in Mission Planner [35]
Receive detection stream via HTTPS on web-based dashboard
Display UAV status, positions, and detected victims
Update mission monitoring interface in real time

4.2.6 Secure Data Transmission (HTTPS-Based Encryption)

Algorithm 6 Secure Detection Data Transmission using HTTPS

Initialization
Generate self-signed SSL certificate on Jetson Orin Nano [48]:
`cert.pem`, `key.pem` using OpenSSL

Server Configuration
Initialize HTTP server for video streaming
Create SSL context using TLS protocol [48]
Load certificate and private key
Wrap server socket with SSL layer

Secure Communication
while streaming is active **do**
Capture processed frame from YOLOv8s detection pipeline [11]
Encode frame into JPEG format
Encrypt data using TLS protocol [48]
Transmit encrypted stream via HTTPS to GCS dashboard
Provide secure API endpoint for detection metadata
end while

Client Access
Access stream via HTTPS URL on GCS detection dashboard
Handle self-signed certificate warning on first connection
Maintain encrypted communication channel throughout mission

4.2.7 End-to-End System Pipeline and Workflow

Algorithm 7 Complete SwiftRescue UAV System Pipeline

System Initialization
Power on UAV and all onboard systems
Initialize Pixhawk flight controller with ArduPilot firmware [29, 20]
Initialize NVIDIA Jetson Orin Nano with YOLOv8s detection system [19, 11]
Establish MAVLink connection with GCS [39]
Initialize HTTP/HTTPS streaming server [48]

Mission Setup
Define and partition mission area in Mission Planner [35]
Upload independently assigned mission plan to each UAV
Perform pre-flight checks (GPS lock, sensors, battery) [29]
Arm UAV system

Autonomous Operation
while mission is active **do**
  Flight Control
  Execute assigned waypoint navigation using ArduPilot [29]
  Maintain flight stability and altitude control
  Perception Processing
  Capture real-time frames from onboard CSI camera
  Perform YOLOv8s inference on Jetson Orin Nano GPU [11, 19]
  Detect victims (person class) with confidence scores
  Data Communication
  Transmit flight telemetry via MAVLink to GCS [39]
  Stream annotated detection video via HTTPS [48]
  Send detection metadata (victim count, confidence)
  Ground Control Monitoring
  Display UAV positions and mission status in Mission Planner [35]
  Visualize real-time detection results on web-based dashboard
  Monitor system performance and respond to alerts
end while

Mission Completion
Execute return-to-launch (RTL) for all UAVs [29]
Perform safe landing and disarm UAV systems

Post-Mission
Store telemetry logs and detection records locally at GCS
Analyze mission performance and coverage quality

4.3 Ground Control Station (GCS)

The Ground Control Station (GCS) serves as the central interface for monitoring, controlling, and analyzing the SwiftRescue UAV system during mission execution [35]. It integrates flight control, real-time telemetry, and visual perception outputs into a unified operational environment, enabling effective human supervision of autonomous UAV operations. The GCS is designed to support real-time decision-making by providing situational awareness through live UAV positioning, system status updates, and victim detection feedback [3].

Unlike traditional single-interface systems, the SwiftRescue GCS adopts a hybrid dual-interface architecture that combines Mission Planner [35] for flight telemetry and mission control with a web-based detection dashboard for real-time perception visualization. This design ensures a clear separation between flight control and onboard intelligence processing while maintaining seamless integration at the operator level. The GCS receives telemetry data from UAVs via the MAVLink protocol [39] and visual detection data via HTTPS streaming from the onboard NVIDIA Jetson Orin Nano [19]. This dual-channel communication approach enhances system reliability and allows independent optimization of control and perception pipelines.

The GCS plays a critical role in coordinating mission execution, supervising UAV behavior, and enabling operator intervention when required. It supports mission configuration, real-time monitoring, and post-mission analysis, making it a key component in ensuring operational safety, efficiency, and system transparency.

4.3.1 Overview of GCS Architecture

The architecture of the Ground Control Station is designed as a dual-interface system that integrates flight control and perception visualization into a cohesive monitoring platform [35]. It consists of two primary components: Mission Planner for UAV telemetry and flight control management, and a web-based detection interface for real-time video streaming and victim detection visualization.

Mission Planner [35] acts as the core flight control system, responsible for handling UAV communication through the MAVLink protocol [39]. It provides functionalities such as mission planning, waypoint configuration, UAV status monitoring, and command execution. Through this interface, operators define and upload independently assigned mission plans to each UAV, supervise UAV positions in real time using a map-based visualization, and issue high-level commands during mission execution.

In parallel, the web-based detection interface provides a real-time view of the onboard perception system. It receives live video streams from the NVIDIA Jetson Orin Nano [19] deployed on each UAV, where the fine-tuned YOLOv8s model [11] performs victim detection. The interface displays annotated video frames with bounding boxes, detection confidence scores, and victim counts, allowing operators to visually confirm detection results during mission execution.

These two components operate concurrently but are logically separated to ensure modularity and system robustness. Telemetry data and flight control commands are handled exclusively through MAVLink [39], while perception data is transmitted independently via HTTPS [48]. This separation prevents interference between critical flight control operations and high-bandwidth video streaming, improving overall system stability and reliability.

The combined architecture enables operators to monitor both the physical state of

UAVs and the semantic understanding of the environment simultaneously, providing a comprehensive situational awareness framework essential for search and rescue missions [5].

4.3.2 Mission Planner Integration (Telemetry and Control)

Mission Planner [35] is utilized as the primary Ground Control Software for UAV communication, mission planning, and flight supervision within the SwiftRescue system. It interfaces directly with the Pixhawk flight controller [20] using the MAVLink protocol [39], enabling reliable and real-time exchange of telemetry data and flight control commands.

During system operation, Mission Planner continuously receives telemetry data from each active UAV, including position coordinates, altitude, velocity, battery status, and flight mode. This information is visualized through both numerical indicators and map-based representations, allowing operators to track UAV movement and monitor system health throughout the mission [35]. The real-time telemetry stream ensures that operators remain informed about UAV performance and can detect anomalies or failures promptly.

Mission planning is conducted through the waypoint-based interface provided by Mission Planner [35]. Operators define search areas, partition coverage zones, specify independent waypoint sequences per UAV, and configure mission parameters such as altitude and speed. These independently assigned mission plans are uploaded to each respective UAV and executed autonomously using the ArduPilot flight control system [29]. The integration supports multi-UAV operations by allowing multiple vehicles to be connected and monitored simultaneously, each identified by a unique MAVLink system identifier [39].

In addition to monitoring, Mission Planner enables direct operator intervention. Commands such as mission start, pause, waypoint update, and emergency abort can be issued in real time and transmitted to each UAV via MAVLink [39]. This ensures that while the system operates autonomously, human oversight is always maintained, supporting safe and controlled mission execution consistent with the human-in-the-loop design described in earlier chapters.

Overall, the integration of Mission Planner provides a robust and reliable control layer for the SwiftRescue system, enabling precise navigation, continuous telemetry monitoring, and effective coordination of multi-UAV swarm operations.

4.3.3 Real-Time Video and Detection Interface

The real-time video and detection interface is a critical component of the Ground Control Station, providing live visual feedback from UAV-mounted cameras along with onboard perception results. This interface is implemented as a lightweight web-based system that streams processed and annotated video directly from the NVIDIA Jetson Orin Nano [19] deployed on each UAV.

The video stream is generated after real-time processing by the fine-tuned YOLOv8s model [11], which performs victim detection on incoming camera frames at approximately 15–20 frames per second. Each frame is annotated with bounding boxes, class labels, and confidence scores corresponding to detected individuals. These processed frames are encoded into JPEG format and transmitted to the Ground Control Station via HTTPS-encrypted streaming [48]. In addition to the video stream, detection metadata such as

victim count, confidence levels, and system performance metrics are exposed through a dedicated API endpoint.

The interface supports simultaneous streams from multiple UAVs, enabling parallel monitoring of all active drones within the same mission [19]. Each stream is displayed in an independent panel, allowing the operator to observe different coverage zones and detection outputs concurrently. This multi-stream capability enhances coverage efficiency and enables cross-verification of detection results across different UAV perspectives.

To ensure responsiveness, the streaming pipeline on the Jetson Orin Nano is optimized for low latency and efficient data transfer. Frame buffering and JPEG encoding are carefully managed to maintain smooth playback without introducing significant delay. The use of HTTPS protocols [48] allows seamless integration with standard web browsers, eliminating the need for specialized client software and simplifying operator access.

Furthermore, the modular design of the interface allows easy scalability, enabling additional UAV streams to be integrated without significant architectural changes. This makes the system adaptable to larger swarm deployments and extended mission scenarios. As a result, the detection interface provides a scalable, flexible, and real-time visualization platform for monitoring victim detection during mission execution.

4.3.4 User Interface and Visualization Design

The user interface of the Ground Control Station is designed to provide a clear and intuitive visualization of both UAV operations and perception outputs. It follows a split-view layout that integrates the web-based detection dashboard and the Mission Planner interface [35], enabling simultaneous access to visual detection information and flight control data without requiring context switching.

The detection dashboard displays live annotated video feeds from multiple UAVs, with each feed presented in a dedicated panel. Detection results from the YOLOv8s model [11] are overlaid directly on the video frames using bounding boxes and labels, allowing operators to quickly identify victims in the coverage area. Additional indicators such as detection confidence, victim count, and system performance metrics including frames per second and system uptime are also displayed to enhance situational awareness.

On the control side, Mission Planner [35] provides a map-based visualization of all active UAV positions, assigned waypoints, and mission progress. Operators can observe individual UAV trajectories, monitor telemetry data, and interact with mission parameters in real time. The integration of both interfaces ensures that perception outputs and flight control information are simultaneously accessible to the operator.

The interface design prioritizes readability, responsiveness, and ease of use. Visual elements are organized to minimize clutter while highlighting critical information such as detection alerts, UAV battery status, and mission progress indicators. Color coding and structured layouts are used to improve quick interpretation under high-pressure operational scenarios.

Additionally, the interface supports scalability by allowing dynamic addition of UAV detection panels, ensuring adaptability for larger swarm deployments. The design also accounts for network variability, ensuring stable visualization even under fluctuating bandwidth conditions. This enables efficient operator monitoring under time-sensitive rescue conditions.

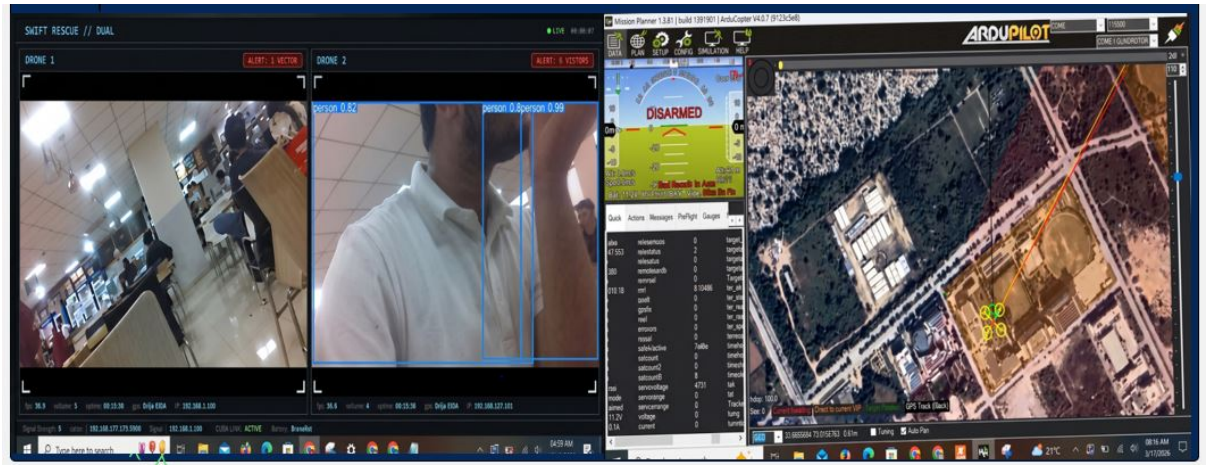


Figure 18: Ground Control Station interface showing multi-UAV detection streams and Mission Planner-based telemetry and control visualization

4.3.5 System Monitoring and Operator Interaction

The Ground Control Station enables continuous system monitoring and facilitates effective operator interaction throughout mission execution [35]. It provides real-time insights into both UAV flight performance and environmental perception, allowing operators to make informed decisions during critical rescue scenarios.

Through Mission Planner [35], operators continuously monitor telemetry data including UAV position, altitude, battery level, velocity, and flight mode for all active UAVs simultaneously. This information is updated in real time and visualized on the GCS map, ensuring that the operator maintains full situational awareness of UAV movement and mission progress. Any anomalies such as low battery, MAVLink communication loss [39], or unexpected flight behavior can be quickly identified and addressed through direct operator intervention.

Simultaneously, the web-based detection dashboard allows operators to observe real-time victim detection results streamed via HTTPS from each UAV's NVIDIA Jetson Orin Nano [19]. Visual confirmation of detected individuals through YOLOv8s-generated bounding boxes [11] enhances operator confidence in the system and supports rescue decision-making. Detection alerts and victim counts help operators prioritize regions of interest and coordinate rescue team deployment effectively.

Operator interaction is supported through control commands including mission start, pause, waypoint modification, and emergency abort, all issued through Mission Planner and transmitted to each UAV via MAVLink [39, 29]. This ensures that while UAVs operate autonomously, the operator retains direct and immediate control at all times.

The system follows a human-in-the-loop design approach [3], where autonomous UAV operation is combined with continuous operator supervision, ensuring safety, accountability, and adaptability in dynamic disaster environments. Mission logs, telemetry records, and detection data are stored locally at the GCS, supporting post-mission analysis, system validation, and iterative performance improvement.

4.4 Hardware Integration and System Deployment

This section describes the hardware components selected for the real-world deployment of the SwiftRescue UAV Swarm System. While the primary validation of the system is performed in simulation, defining the physical hardware platform is essential to demonstrate feasibility and ensure that the proposed architecture can be transitioned to real UAV platforms. The hardware design follows standard multirotor UAV practices and aligns with the control, perception, and communication requirements identified in earlier chapters. All components were selected based on compatibility with ArduPilot-based flight control [29], onboard AI computation on the NVIDIA Jetson Orin Nano [19], and MAVLink-based communication [39], ensuring consistency between simulation results and physical deployment.

4.4.1 Hardware Components and Platform Description

The hardware platform of SwiftRescue is organized into modular subsystems covering flight control, propulsion, structural support, sensing, computation, and communication. This modular design simplifies system integration, improves maintainability, and allows individual components to be upgraded or replaced without redesigning the entire platform. The selected hardware components represent a commonly used and well-documented UAV configuration, making the system reliable, extensible, and suitable for research-oriented swarm deployment scenarios [6, 29].

4.4.1.1 Core Flight and Control Hardware

Core flight and control hardware consists of all flight-critical components required to achieve stable, controlled, and safe UAV operation. These components form the physical foundation of the SwiftRescue aerial platform and directly support autonomous navigation, mission execution, and overall system safety.

The **Pixhawk Flight Controller** [20] serves as the central control unit of the UAV. It executes the ArduPilot autopilot firmware [29] and manages essential flight control tasks such as attitude stabilization, waypoint navigation, sensor fusion, and failsafe handling. Pixhawk is widely adopted in research and industrial UAV platforms due to its reliability, open-source ecosystem, and strong support for autonomous operations and companion computer integration [20, 29].

Brushless motors (1000KV, 4×) are used to generate lift and thrust for quadrotor flight. These motors provide a balanced trade-off between torque and efficiency for medium-sized quadcopters such as the F450 frame, enabling stable hovering, smooth maneuvering, and sufficient payload support for onboard computing and sensors used in SwiftRescue [21].

Electronic Speed Controllers (30A ESCs) regulate motor speed based on control signals received from the Pixhawk flight controller [20]. Each ESC translates low-level control commands into precise motor actuation, ensuring synchronized motor operation and stable flight behavior during autonomous missions. The selected 30A rating ensures reliable current handling under varying load conditions and contributes to overall propulsion stability.

Propellers are configured as clockwise (CW) and counter-clockwise (CCW) pairs to balance rotational torque and maintain yaw stability [22]. Proper propeller pairing

is essential for efficient thrust generation and predictable quadrotor dynamics, and the selected propellers are compatible with both the motors and frame size.

The **F450 quadcopter frame** provides the mechanical structure that integrates all flight components [23]. It is commonly used in research and educational UAV platforms due to its rigid construction, balanced geometry, and ease of component mounting, which collectively support stable flight characteristics and simplified system assembly.

Landing gear is used to protect critical components such as the flight controller, power distribution system, and onboard computer during takeoff and landing. Tall landing gear also provides adequate clearance for downward-facing sensors and cameras required for perception tasks in the SwiftRescue system [6].

Figure 19 illustrates the core flight and control hardware components of the SwiftRescue UAV platform, visually linking the described components to their physical layout and supporting clarity in system integration and deployment planning.



Figure 19: Core flight and control hardware components used in the SwiftRescue UAV platform, including motors, ESCs, propellers, and quadcopter frame

4.4.1.2 Power System and Energy Management

The power system is a critical subsystem of the SwiftRescue UAV platform, responsible for providing stable, regulated, and safe electrical power to all onboard components during flight operations. Proper power management is essential to ensure sustained flight, reliable sensor operation, and uninterrupted execution of autonomous missions, particularly in long-duration and multi-UAV search and rescue scenarios.

A **3S Li-Po battery (5200 mAh)** is used as the primary energy source for the UAV. This battery configuration is selected based on the motor specifications, total system payload, and power requirements of onboard computing and sensing modules. Li-Po batteries are widely adopted in UAV platforms due to their high energy density and ability to supply the high current demanded by brushless motors during takeoff, hovering, and maneuvering [21]. The selected battery capacity ensures sufficient endurance while maintaining safe operating margins.

The **power module (APM)** plays a central role in power regulation and monitoring. It provides a stable and regulated voltage supply to the Pixhawk flight controller [20] while simultaneously measuring battery voltage and current consumption. This telemetry information is continuously reported to the flight controller and Ground Control Station via MAVLink [39], enabling real-time battery status monitoring, low-voltage warnings, and automated failsafe responses during autonomous missions [29].

A **power distribution board (PDB)** is used to distribute electrical power from the battery to the electronic speed controllers (ESCs) and auxiliary subsystems. The PDB simplifies wiring layout, reduces power losses, and ensures uniform voltage delivery to all motors, which is essential for balanced thrust generation and stable flight behavior.

XT60 connectors are employed for secure, high-current battery connections within the UAV power system. These connectors are designed to safely handle the current requirements of quadrotor platforms while minimizing contact resistance and reducing the risk of accidental disconnection or overheating during operation.

Power cables and wiring are selected to support the expected current loads while minimizing voltage drop and electrical noise. Proper insulation and routing are applied to prevent electrical interference and mechanical damage caused by vibration.

The **NVIDIA Jetson Orin Nano companion computer** [19] is powered from the Li-Po battery through a regulated voltage module, ensuring a unified power architecture. This configuration reduces system complexity and allows synchronized operation between flight control and onboard processing components.

A dedicated **Li-Po battery charger** is used to ensure safe and controlled charging of the power source, maintaining battery health and operational reliability.

4.4.1.3 Navigation and Sensor Suite

The navigation and sensor suite of the SwiftRescue UAV platform provides the sensing and state-estimation capabilities required for autonomous navigation, localization, perception, and flight safety [25].

A **GPS module with integrated compass (u-blox NEO-M8N)** is used as the primary global positioning and heading reference for the UAV. It provides reliable position estimates and directional information required for waypoint navigation and return-to-home functionality [25].

The **Inertial Measurement Unit (IMU)**, integrated within the Pixhawk flight controller [20], supplies high-frequency motion data including linear acceleration and angular velocity. These measurements are fused with GPS data to estimate UAV state and ensure stable flight control [21].

An **IMX219 CSI camera module** is used for onboard perception. It captures real-time image data during flight, which is processed by the fine-tuned YOLOv8s model [11] on the NVIDIA Jetson Orin Nano [19] for victim detection. The camera provides high-resolution visual input and is directly interfaced with the onboard computing system, enabling efficient data acquisition and processing for real-time analysis.

Together, the navigation and sensor suite enables reliable autonomous flight, accurate localization, and perception-driven mission execution, supporting the transition from simulation to real-world deployment.

4.4.1.4 Onboard Computing and Autonomy Platform

The onboard computing and autonomy platform provides the high-level processing capabilities required for perception, real-time inference, and data streaming in the SwiftRescue UAV system [19, 17]. This platform acts as a bridge between the low-level flight control handled by the Pixhawk flight controller [20] and the high-level perception and monitoring functionalities required for search and rescue operations. It enables the system to process visual data, generate detection outputs, and stream results to the Ground Control Station during mission execution.

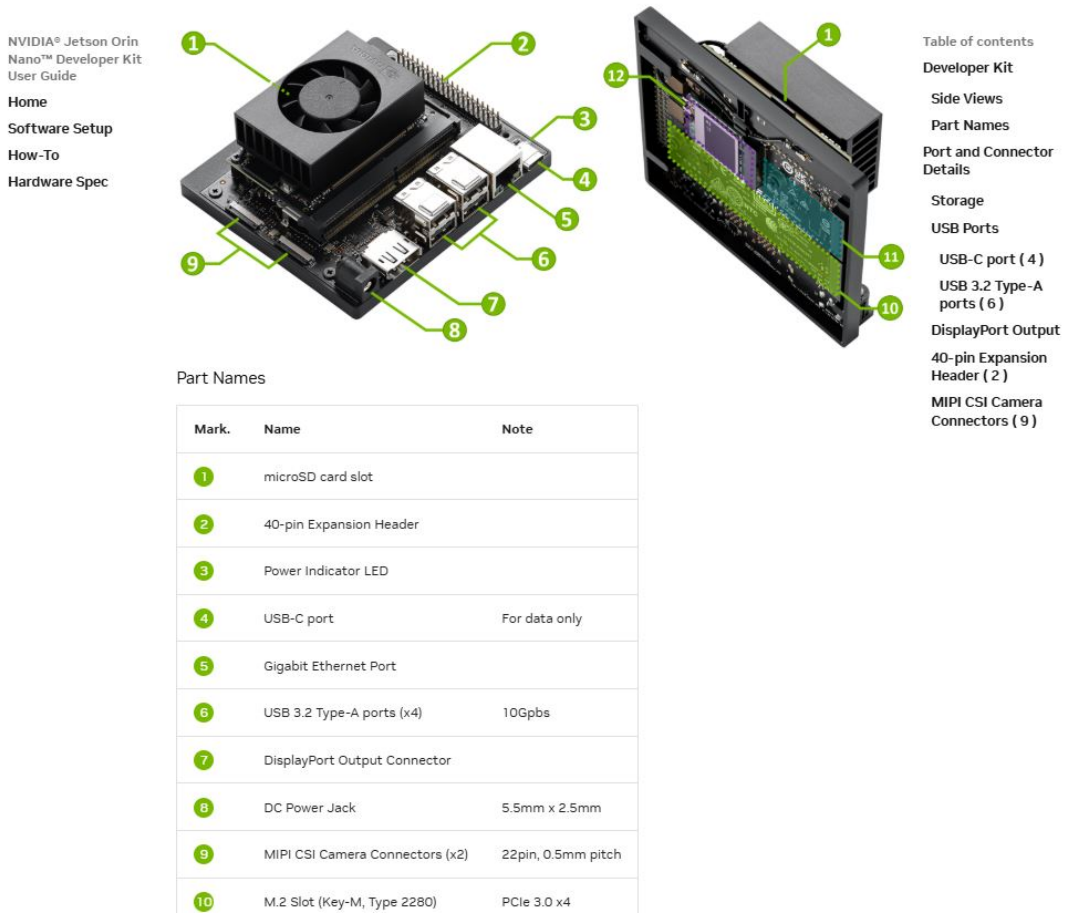
The **Jetson Orin Nano Developer Kit (8GB)** [19] serves as the primary companion computer for the UAV. It is responsible for executing computationally intensive tasks such as real-time image processing and victim detection using the fine-tuned YOLOv8s model [11]. Compared to traditional embedded platforms, the Jetson Orin Nano provides significantly higher GPU performance, making it suitable for deploying deep learning models in aerial systems [17]. In the SwiftRescue system, it processes camera input from the IMX219 CSI camera, performs YOLOv8s inference, and prepares detection outputs for transmission to the Ground Control Station.

The Jetson platform operates on **Ubuntu 22.04 with JetPack 6.2** [19], providing a stable and optimized environment for GPU-accelerated computing. This software stack supports CUDA, cuDNN, and other acceleration libraries required for efficient deep learning execution. The use of JetPack ensures compatibility between hardware and software components and enables reliable deployment of the YOLOv8s perception module [11] in real-time scenarios.

The **fine-tuned YOLOv8s model** [11] is deployed on the Jetson Orin Nano to perform real-time victim detection. The UAV-optimized model, trained for 80 epochs at 960px resolution with advanced augmentation, was selected for deployment due to its improved generalization under real-world aerial conditions. The system processes

incoming image frames from the onboard camera and generates detection outputs with associated confidence scores, which are transmitted to the Ground Control Station via HTTPS-encrypted streaming [48].

Figure 20 illustrates the Jetson Orin Nano Developer Kit and its key interfaces used in the SwiftRescue system, including camera connectivity, power input, and data ports that support onboard processing and communication.



Mark.	Name	Note
1	microSD card slot	
2	40-pin Expansion Header	
3	Power Indicator LED	
4	USB-C port	For data only
5	Gigabit Ethernet Port	
6	USB 3.2 Type-A ports (x4)	10Gpbs
7	DisplayPort Output Connector	
8	DC Power Jack	5.5mm x 2.5mm
9	MIPI CSI Camera Connectors (x2)	22pin, 0.5mm pitch
10	M.2 Slot (Key-M, Type 2280)	PCIe 3.0 x4

Figure 20: Jetson Orin Nano Developer Kit used as the onboard computing platform for real-time perception and detection processing

Together, the onboard computing stack centered around the Jetson Orin Nano [19] enables efficient perception-driven operation, supports real-time YOLOv8s-based detection [11], and integrates seamlessly with the UAV control system. This configuration ensures that high-level processing tasks are handled independently of the flight controller while maintaining reliable data flow to the Ground Control Station, aligning with the overall system architecture defined in earlier chapters.

4.4.1.5 Communication and Networking Hardware

Communication and networking hardware enable reliable command exchange, telemetry transmission, and real-time data streaming within the SwiftRescue UAV system through two independent communication channels [39].

Safety and manual override are provided through an **RC transmitter and receiver**, allowing immediate operator intervention in case of system failure, directly sup-

ported by the Pixhawk flight controller [20] and ArduPilot failsafe configuration [29].

MAVLink [39] handles flight telemetry and control between the Pixhawk and Mission Planner [35], enabling real-time communication of UAV position, altitude, battery status, and flight mode, and supporting mission upload and operator command transmission.

WiFi connectivity is used for high-bandwidth data transmission between the NVIDIA Jetson Orin Nano [19] and the Ground Control Station, supporting HTTPS-encrypted streaming [48] of annotated video and YOLOv8s detection outputs [11], enabling live monitoring of perception results at the GCS.

The communication architecture strictly separates flight control communication (MAVLink) from perception data transmission (WiFi/HTTPS), preventing interference between these two independent channels and supporting both safety and performance requirements of the system.

4.4.1.6 Safety and Auxiliary Components

Safety and auxiliary components enhance the reliability and protection of the SwiftRescue UAV platform during testing and operation, complementing the software-level failsafe mechanisms implemented in ArduPilot [29].

A **buzzer** integrated with the Pixhawk [20] provides audible alerts for arming status, low battery warnings, and failsafe conditions, useful during both setup and emergency situations.

Status LEDs provide visual indicators of system health, GPS lock status, and flight modes, enabling quick assessment of UAV readiness before and during operation.

A **kill switch** provides an additional safety mechanism to immediately cut motor power during critical failures, reducing the risk of damage and ensuring operator safety. **Propeller guards** are used during testing phases to reduce the risk of injury from accidental contact.

A **vibration damping mount** on the Pixhawk flight controller [20] reduces mechanical vibrations from the frame and motors, improving sensor accuracy and ensuring stable IMU readings during flight.

4.4.1.7 Storage and Peripheral Accessories

Storage and peripheral accessories support data logging, system connectivity, and physical integration of hardware components within the SwiftRescue UAV platform.

A **microSD card (64 GB)** on the NVIDIA Jetson Orin Nano [19] stores the operating system, fine-tuned YOLOv8s model weights [11], and mission detection data, supporting continuous data logging during operation.

A **microSD card for the Pixhawk** [20] records flight logs including sensor data, control signals, and telemetry history, essential for post-mission analysis and debugging.

USB connections establish the MAVLink communication interface between the Pixhawk and Jetson Orin Nano [19], enabling telemetry exchange and system synchronization between flight control and onboard processing.

Mounts and brackets securely install all components on the F450 UAV frame, ensuring stability, proper alignment, and protection against vibration during flight.

4.4.2 Hardware Integration and Configuration (Mission Planner)

This section describes the process used to integrate and configure the SwiftRescue UAV hardware platform using Mission Planner [35] as the primary ground control software. Mission Planner serves as the primary configuration and validation tool for the Pixhawk flight controller [20] running ArduPilot firmware [29]. Proper hardware integration at this stage is essential to ensure that the physical UAV platform behaves consistently with the control logic, safety assumptions, and autonomous workflows validated earlier in simulation. The configuration steps outlined in this section establish a stable and reproducible autopilot environment that supports autonomous navigation, sensor fusion, failsafe handling, and reliable interaction with the Jetson Orin Nano companion computer [19] through MAVLink telemetry [39] and HTTPS-based detection streaming [48].

4.4.2.1 Flight Controller Firmware Setup

The flight controller firmware setup forms the foundation of the SwiftRescue hardware integration process. The Pixhawk flight controller [20] is configured to run the ArduPilot autopilot firmware [29], which provides robust support for multirotor platforms, autonomous mission execution, sensor fusion, and companion computer integration. ArduPilot was selected due to its open-source architecture, extensive documentation, and consistency with the SITL configuration [30] used during simulation-based validation.

Using Mission Planner [35], the appropriate ArduPilot firmware version for a quadrotor platform is selected and installed on the Pixhawk flight controller. The firmware upload process initializes the flight controller with a clean and verified software environment, ensuring that default parameters and safety checks are correctly applied. This step aligns the hardware autopilot configuration with the same control logic evaluated in simulation, reducing discrepancies between simulated and physical system behavior.

Following firmware installation, the Pixhawk board configuration is performed within Mission Planner [35]. This includes selecting the correct flight controller board type, verifying sensor availability, and ensuring proper communication interfaces are enabled. Board-level configuration ensures that onboard sensors, power monitoring inputs, and communication ports are correctly recognized and utilized by the autopilot firmware [29].

The UAV frame type is then configured as a quadrotor in X-configuration, matching the physical F450 frame used in the SwiftRescue platform [23]. Correct frame type selection is critical, as it determines motor mixing logic, control allocation, and stability behavior during flight. In addition, frame orientation parameters are verified to ensure that the Pixhawk's physical mounting orientation matches the autopilot's coordinate frame assumptions. Accurate orientation configuration is essential for reliable sensor fusion, attitude estimation, and navigation control [21].

Collectively, these firmware setup and configuration steps establish a consistent and reliable autopilot environment on the Pixhawk flight controller. By aligning firmware selection, board configuration, and frame setup with the system design validated in simulation, SwiftRescue ensures that the hardware platform is correctly prepared for subsequent sensor calibration, safety configuration, and autonomous mission deployment.

4.4.2.2 Sensor Calibration and Verification

Sensor calibration and verification is a critical step in the hardware integration process of the SwiftRescue UAV platform. Accurate sensor measurements are essential for reliable

state estimation, stable flight control, autonomous navigation, and safe mission execution [25]. This stage ensures that all onboard sensors provide consistent and trustworthy data to the Pixhawk flight controller [20] and the higher-level autonomy modules validated earlier in simulation.

The **accelerometer calibration** process is performed using Mission Planner [35] to align the sensor readings with the UAV's physical orientation. During calibration, the UAV is placed in multiple predefined orientations to allow the flight controller to accurately determine gravity vectors and remove sensor bias. Correct accelerometer calibration is fundamental for precise attitude estimation, directly affecting roll, pitch, and overall stability during hover and maneuvering [21].

The **gyroscope calibration** is conducted to eliminate bias and drift in angular rate measurements. The gyroscope plays a key role in detecting rapid rotational movements and supporting attitude stabilization loops within ArduPilot [29]. Calibration ensures that the gyroscope outputs zero angular velocity when the UAV is stationary, enabling smooth and responsive control behavior during both manual and autonomous flight operations.

The **compass calibration**, performed for the integrated GPS and magnetometer module, establishes accurate heading estimation and global orientation [25]. This process accounts for magnetic interference caused by onboard electronics, wiring, and structural components. Through Mission Planner's compass calibration routine [35], the system maps magnetic field measurements across multiple orientations to compensate for distortion effects. Accurate compass data is essential for reliable waypoint navigation, yaw control, and coordinated swarm movement in outdoor environments.

Radio calibration is carried out for the RC receiver to ensure correct interpretation of control signals used for arming, failsafe recovery, and emergency override. Calibration verifies that all control channels operate within expected ranges and that neutral, minimum, and maximum values are correctly detected by the flight controller [29]. Proper radio calibration is a critical safety requirement, ensuring that manual intervention and failsafe mechanisms function reliably during unexpected system behavior.

Upon completion of all calibration procedures, sensor outputs are verified through Mission Planner's real-time monitoring tools [35]. This verification step confirms stable sensor readings, correct orientation alignment, and consistent data fusion within the flight controller. By thoroughly calibrating and validating all sensors, the SwiftRescue hardware platform establishes a dependable sensing foundation that supports autonomous navigation, perception-driven decision-making, and safe multi-UAV mission execution.

4.4.2.3 Power System Configuration

Power system configuration is a fundamental step in preparing the SwiftRescue UAV platform for safe and reliable autonomous operation. Correct power configuration ensures that the flight controller and onboard systems receive stable electrical input, while also enabling continuous monitoring of battery health during mission execution [29]. This stage directly supports energy safety, flight endurance management, and failsafe operation, which are critical for both single-UAV and swarm-based missions.

The **power module voltage and current calibration** is performed using Mission Planner [35] to ensure accurate measurement of battery parameters. The power module provides real-time voltage and current readings to the Pixhawk flight controller [20], which are used for energy estimation and system health monitoring. Calibration aligns

the measured values with the actual battery characteristics, compensating for sensor offsets and ensuring that reported telemetry [39] reflects true power consumption during flight.

Battery monitoring configuration is then established for the installed **3S Li-Po battery (5200 mAh)**. This includes defining the battery type, number of cells, and nominal voltage parameters within the ArduPilot firmware [29]. Accurate battery configuration allows the flight controller to compute remaining capacity and estimate flight time based on real-time current draw. This information is continuously transmitted to the Ground Control Station via MAVLink [39], enabling operators to monitor energy status throughout autonomous missions.

The **low-voltage failsafe mechanism** is configured to protect the UAV from deep battery discharge and sudden power loss [29]. Voltage threshold levels are defined within Mission Planner [35] to trigger predefined safety actions when battery voltage drops below safe limits. These actions may include controlled landing, return-to-launch behavior, or mission termination, depending on the operational context. Proper failsafe configuration is essential for preventing in-flight shutdowns, protecting onboard electronics, and ensuring safe recovery of the UAV during extended missions.

In addition, the same power system is used to support the NVIDIA Jetson Orin Nano [19], which is supplied through a regulated power interface derived from the main battery. This unified power design ensures synchronized system operation while maintaining electrical stability for both flight control and onboard processing units.

Following configuration, power system behavior is verified through real-time telemetry observation during ground testing and low-risk flight scenarios. Voltage stability, current readings, and failsafe triggers are monitored to confirm correct system response under varying load conditions. Through comprehensive power system configuration and validation, the SwiftRescue hardware platform ensures reliable energy management, operational safety, and uninterrupted execution of autonomous search and rescue tasks.

4.4.2.4 Motor, ESC, and Frame Configuration

Motor, ESC, and frame configuration is a critical phase in preparing the SwiftRescue UAV for stable and predictable flight behavior. This configuration ensures that propulsion components operate in a coordinated manner, directly influencing flight stability, maneuverability, and safety during autonomous mission execution [29]. Proper setup at this stage is essential before any higher-level autonomy or swarm coordination functions can be reliably deployed.

The **motor order and direction setup** is performed within Mission Planner [35] to match the physical motor layout on the quadcopter frame. Each motor is assigned a specific index corresponding to the ArduPilot motor mapping scheme [29], ensuring that control outputs from the Pixhawk flight controller [20] are applied to the correct motors. Motor rotation directions are configured according to the quadcopter X-configuration, with alternating clockwise (CW) and counter-clockwise (CCW) rotation patterns [21]. Correct motor ordering and direction are essential for achieving balanced thrust and stable yaw control.

Electronic Speed Controller (ESC) calibration (30A ESCs) is then carried out to synchronize throttle response across all motors. ESC calibration aligns the minimum and maximum throttle values recognized by each ESC, ensuring uniform motor speed response to control inputs. This process prevents uneven thrust generation, reduces the risk

of instability during takeoff, and improves overall flight smoothness during autonomous navigation and maneuvering [22].

The **frame type selection** is configured in ArduPilot [29] to match the physical UAV structure. For SwiftRescue, the quadcopter X-configuration is selected, defining the geometric arrangement of motors and control mixing logic. Accurate frame configuration allows the flight controller to correctly translate attitude and navigation commands into appropriate motor outputs, forming the basis for reliable stabilization and control [23].

Finally, **propeller direction verification** is conducted to ensure that each propeller is mounted in accordance with its designated rotation direction. CW and CCW propellers are checked to confirm proper installation, as incorrect propeller orientation can severely degrade lift generation and compromise flight safety [22]. Verification of propeller direction ensures efficient thrust production and predictable response to control commands.

Through careful motor, ESC, and frame configuration, the SwiftRescue UAV platform achieves consistent propulsion behavior, stable flight dynamics, and reliable response to autonomous control inputs, establishing a robust foundation for safe flight testing, perception integration, and multi-UAV swarm operations.

4.4.2.5 Safety, Failsafe, and Arming Configuration

Safety, failsafe, and arming configuration is a fundamental aspect of the SwiftRescue UAV hardware integration process, ensuring that the system operates reliably under both normal and abnormal conditions [29]. Given the autonomous and swarm-based nature of the platform, robust safety mechanisms are essential to mitigate operational risks, protect hardware, and ensure safe interaction between the UAV, operators, and the surrounding environment.

Arming checks are configured within Mission Planner [35] to enforce mandatory pre-flight conditions before the UAV can be armed. These checks validate critical system states such as sensor health, GPS availability, battery voltage, and flight mode readiness [29]. By enforcing arming checks, the system prevents takeoff under unsafe or undefined conditions, reducing the likelihood of flight instability or immediate mission failure.

RC failsafe setup is implemented to ensure safe recovery in the event of loss of control signal between the RC receiver and the flight controller. Even though SwiftRescue primarily operates autonomously, the RC link remains an essential safety channel for manual override and emergency intervention. The RC failsafe configuration defines explicit system behavior — such as controlled landing or motor disarm — when signal loss is detected, ensuring predictable and safe UAV response [29].

Battery failsafe behavior is configured using real-time battery voltage and current data provided by the power module. Thresholds are defined for low-battery and critical-battery conditions, triggering automated responses such as warning alerts, return-to-launch behavior, or controlled landing [29]. This configuration protects the UAV from deep battery discharge, prevents sudden power loss during flight, and supports safe mission termination during extended autonomous operations.

Emergency disarm configuration provides a mechanism for immediate motor shutdown in critical situations. This functionality allows operators to rapidly disable propulsion in the event of severe malfunction, collision risk, or loss of situational control. Emergency disarm settings are carefully configured to balance responsiveness with protection against accidental activation, ensuring that this mechanism is available when

required without introducing additional operational risk.

Together, these safety, failsafe, and arming configurations establish a comprehensive risk mitigation framework for the SwiftRescue UAV platform. By integrating multiple layers of protection at the flight controller level [29], the system ensures safe autonomous operation, supports reliable mission execution, and provides essential safeguards during both testing and future real-world deployment.

4.4.2.6 Companion Computer Interface Configuration

The companion computer interface configuration establishes the hardware–software bridge between the low-level flight control system and the high-level perception and monitoring components of the SwiftRescue UAV platform. This integration enables coordination between the Pixhawk flight controller [20] and the NVIDIA Jetson Orin Nano companion computer [19], allowing real-time detection outputs and system monitoring to be incorporated into mission execution.

A **MAVLink communication interface** [39] is configured to enable bidirectional data exchange between the Pixhawk and the Jetson Orin Nano. This connection is established using a USB interface and supports continuous transmission of telemetry data, flight status, and system information. MAVLink [39] ensures compatibility between ArduPilot [29] and Mission Planner [35], allowing real-time monitoring of UAV parameters such as position, altitude, and battery status.

The **Pixhawk–Jetson Orin Nano communication setup** ensures that flight data, including position estimates, attitude information, and system state, is streamed in real time to the Jetson. This enables the onboard system to synchronize YOLOv8s perception outputs [11] with UAV state information. Swarm coordination is not performed onboard but is managed externally through the Ground Control Station via Mission Planner [35], consistent with the GCS-based coordination architecture defined in earlier chapters.

Parameter configuration for companion computer interaction is performed within the ArduPilot firmware [29] to allow controlled data exchange without interfering with flight stability. These parameters define safe communication boundaries, ensuring that telemetry is available to the Jetson Orin Nano [19] while maintaining the autonomy of the flight control system.

The Jetson Orin Nano transmits **real-time YOLOv8s detection results and annotated video streams to the Ground Control Station using HTTPS-encrypted communication** [48]. This design strictly separates high-bandwidth perception data from MAVLink telemetry [39], ensuring efficient and non-interfering data flow during mission execution.

The interface configuration also supports continuous monitoring of flight status and system health by the companion computer, enabling real-time awareness of UAV conditions and facilitating logging of synchronized telemetry and detection data for debugging, validation, and performance analysis.

Overall, the companion computer interface configuration provides a reliable integration between flight control and onboard perception systems. By combining MAVLink-based telemetry [39] with HTTPS-based detection streaming [48], the SwiftRescue system achieves efficient communication, real-time monitoring, and consistent alignment with the implemented hardware architecture.

4.4.3 System Deployment Workflow

The system deployment workflow of the SwiftRescue UAV platform defines the structured process through which the integrated hardware and software components are transitioned into an operational autonomous system. This workflow builds upon the hardware assembly described in Section 4.4.1 and the configuration procedures outlined in Section 4.4.2, ensuring that all subsystems operate cohesively during real-world deployment. It outlines the sequence of steps required to prepare the UAV for operation, initialize onboard systems and communication links, configure mission parameters, execute autonomous missions, and safely terminate system operation. By following this structured workflow, SwiftRescue ensures operational reliability, safety, and repeatability while maintaining consistency with the simulation-based validation framework discussed earlier in this chapter.

4.4.3.1 Hardware Setup and Power Initialization

The deployment process begins with the physical setup and power initialization of the UAV platform. All hardware components, including the Pixhawk flight controller [20], NVIDIA Jetson Orin Nano [19], ESCs, brushless motors, telemetry module, and IMX219 camera, are verified for proper connection and secure mounting within the F450 UAV frame. Particular attention is given to ensuring correct power distribution from the LiPo battery to both the flight controller and the onboard processing unit.

The UAV is powered using the 5200 mAh LiPo battery, which supplies energy to the propulsion system as well as the onboard electronics. A regulated power supply is used to provide stable voltage to the Jetson Orin Nano [19], preventing fluctuations during operation. Upon powering the system, the Pixhawk initializes its ArduPilot firmware [29] while the Jetson Orin Nano begins its boot sequence.

During this phase, basic hardware checks are performed, including LED status verification, motor readiness (with propellers handled under safe conditions), and confirmation of camera connectivity. The system is allowed to stabilize briefly to ensure all components are functioning correctly before proceeding to flight controller initialization and sensor verification.

4.4.3.2 Flight Controller Initialization and Sensor Verification

Following hardware setup and power initialization, the Pixhawk flight controller [20] undergoes system initialization and sensor verification. During this phase, the flight controller boots its ArduPilot firmware [29] and performs internal checks on critical onboard sensors, including the accelerometer, gyroscope, compass, and GPS module [25]. These sensors are essential for stable flight control, orientation estimation, and navigation.

The UAV is connected to the Ground Control Station using Mission Planner [35] via the MAVLink telemetry link [39]. Through this interface, the operator verifies system status indicators such as sensor health, battery level, and GPS lock. A minimum GPS satellite threshold is ensured to enable reliable positioning and waypoint navigation during mission execution [25]. Calibration parameters configured during earlier setup are also validated to confirm correct sensor behavior.

Additionally, pre-flight safety checks are performed to ensure that the system is ready for operation. These include verifying flight modes, confirming proper arming conditions, and ensuring that no critical errors are reported by the flight controller [29]. The UAV

remains stationary during this process to avoid incorrect sensor readings. Successful completion of this stage confirms that the flight control system is stable, calibrated, and ready for mission configuration and execution.

4.4.3.3 Onboard Processing System Setup (Jetson Orin Nano)

After successful flight controller initialization, the onboard processing system based on the NVIDIA Jetson Orin Nano [19] is initialized to enable real-time perception and data streaming. The Jetson system boots into Ubuntu 22.04 with JetPack 6.2, where all required dependencies including CUDA libraries, deep learning frameworks, and the fine-tuned YOLOv8s model [11] are pre-installed.

Once the system is fully operational, the perception pipeline is launched. The IMX219 CSI camera connected to the Jetson is activated and a continuous video stream is captured for processing. The fine-tuned YOLOv8s model [11] is loaded into GPU memory on the Jetson Orin Nano [19], enabling efficient real-time inference on incoming frames. The system processes each frame to detect human presence, generating bounding box outputs and detection metadata with associated confidence scores.

In addition to detection, the Jetson initializes the HTTPS-based streaming service [48] responsible for transmitting processed and annotated video to the Ground Control Station. This allows the operator to access the live detection stream through a standard web browser or monitoring interface. The strict separation of perception processing from flight control ensures that YOLOv8s detection tasks do not interfere with UAV stabilization and navigation.

Before proceeding to mission execution, the operator verifies that the detection pipeline is functioning correctly by confirming the availability of the live HTTPS stream and the presence of detection outputs. This stage ensures that the onboard AI system is fully operational and ready to support real-time mission monitoring.

4.4.3.4 Communication Link Establishment

Following initialization of both the flight controller and onboard processing system, communication links are established to enable data exchange between the UAV and the Ground Control Station. The SwiftRescue system utilizes a dual-channel communication architecture, separating flight control from perception data transmission to ensure reliability and modular operation [39, 48].

The first communication channel is based on MAVLink telemetry [39], connecting the Pixhawk flight controller [20] to Mission Planner [35] at the GCS. This link enables real-time transmission of critical flight data including GPS position, altitude, battery status, and system health. It also allows the operator to send control commands such as arming, flight mode switching, and mission waypoint uploads [29]. The telemetry connection is verified by confirming continuous data updates within the Mission Planner interface.

The second communication channel is established between the NVIDIA Jetson Orin Nano [19] and the GCS using the HTTPS-based streaming service [48]. This channel is responsible for transmitting the real-time annotated video feed along with YOLOv8s detection outputs [11]. The operator accesses the stream through a web browser, enabling visualization of detected victims during mission execution.

This separation of communication channels ensures that flight control operations via MAVLink remain stable and unaffected by perception data processing or network fluctuations. Before proceeding to mission configuration, both communication links are verified

to ensure continuous telemetry updates and successful access to the live detection stream, confirming that the system is fully connected and operational.

4.4.3.5 Mission Planning and Configuration (GCS)

After establishing communication between the UAV and the Ground Control Station, mission planning and configuration are performed using Mission Planner [35]. This stage defines the operational behavior of the UAV during deployment and ensures that all mission parameters are correctly set before flight execution.

The operator uses Mission Planner [35] to configure mission waypoints, flight altitude, and navigation paths based on the target search area. For multi-UAV deployments, the search area is partitioned into non-overlapping coverage zones, with an independently assigned waypoint plan uploaded to each UAV via MAVLink [39]. Additional parameters such as flight speed, loiter time, and return-to-launch behavior are also configured to ensure controlled and safe mission execution [29].

Once the mission plan is defined and uploaded, the operator verifies that all waypoints are accurately displayed within the Mission Planner interface [35]. Prior to arming the UAV, final pre-flight checks are performed, including verification of flight mode configuration, battery status, GPS lock, and system readiness [29].

This stage ensures that the UAV is properly configured for autonomous navigation while maintaining operator supervision through the GCS. Successful mission configuration confirms that the system is ready to transition into autonomous mission execution.

4.4.3.6 Autonomous Mission Execution

Upon successful mission configuration, the UAV transitions into autonomous mission execution. The Pixhawk flight controller [20] manages waypoint navigation based on the independently assigned mission plan uploaded from Mission Planner [35]. Using onboard sensor data and GPS positioning [25], the UAV follows the planned trajectory while ArduPilot [29] maintains stable flight and altitude control throughout.

Concurrently, the NVIDIA Jetson Orin Nano [19] operates as the onboard perception unit. The IMX219 camera continuously captures visual data, which is processed in real time using the fine-tuned YOLOv8s model [11]. The system detects human presence within captured frames and generates bounding box outputs and detection metadata with confidence scores. These detection results are integrated into the annotated video stream and transmitted to the GCS via the HTTPS communication channel [48].

The strict separation between MAVLink-based flight control [39] and HTTPS-based detection streaming [48] ensures that navigation remains stable while perception operates independently. The operator simultaneously monitors both UAV flight status through Mission Planner [35] and detection outputs through the web-based dashboard, maintaining full situational awareness throughout mission execution.

This stage represents the core operational phase of the system, where autonomous navigation and real-time YOLOv8s-based perception [11] function together to support search and rescue objectives. The UAV continues execution until the mission is completed or manually interrupted by the operator.

4.4.3.7 Real-Time Monitoring and Operator Interaction

During mission execution, real-time monitoring and operator interaction are performed through the Ground Control Station. The operator continuously observes UAV status using Mission Planner [35], which provides telemetry data such as GPS position, altitude, battery level, flight mode, and overall system health via the MAVLink communication link [39].

In parallel, the operator monitors the live annotated video stream transmitted via HTTPS from the NVIDIA Jetson Orin Nano [19]. This stream includes real-time detection outputs generated by the YOLOv8s model [11], allowing visualization of identified human targets within the operational environment. The integration of MAVLink telemetry and HTTPS visual feedback enables the operator to maintain comprehensive situational awareness throughout the mission.

The system supports human-in-the-loop supervision [3], allowing the operator to intervene when necessary. In case of anomalies such as MAVLink communication instability, unexpected UAV behavior, or low battery conditions, corrective actions can be initiated through Mission Planner [35]. These actions may include adjusting flight modes, pausing the mission, or triggering return-to-launch procedures via ArduPilot failsafe mechanisms [29].

This stage ensures that autonomous operation is complemented by continuous operator oversight, enhancing system safety and reliability during deployment.

4.4.3.8 Mission Termination and System Shutdown

Upon completion of the assigned mission, the UAV transitions into the mission termination phase. This process is managed through Mission Planner [35], where the operator confirms that mission objectives have been completed and that the UAV is ready for safe recovery. Depending on the mission configuration, the UAV either executes an automatic return-to-launch (RTL) procedure or performs a controlled landing at a predefined location, both managed by ArduPilot [29].

During this phase, the Pixhawk flight controller [20] manages descent and landing operations using onboard sensor data [25] to ensure stability and safety. The operator continuously monitors MAVLink telemetry data [39] including altitude, battery status, and flight mode to confirm proper execution of the landing sequence. In case of any irregularities, manual intervention can be performed through Mission Planner to ensure a safe landing.

After the UAV has landed successfully, the system is disarmed to stop all motor activity. The NVIDIA Jetson Orin Nano [19] is then safely terminated, including the YOLOv8s perception pipeline [11] and HTTPS streaming service [48]. MAVLink telemetry links and HTTPS streaming channels are closed in an orderly manner.

Finally, all mission logs, telemetry records, and YOLOv8s detection outputs are retained locally at the GCS for post-mission analysis, system validation, and iterative performance improvement.

4.4.3.9 Final System Deployment and Hardware Overview

This subsection presents the final assembled SwiftRescue UAV platform after complete hardware integration and system deployment. The developed system consists of an F450

quadcopter frame equipped with a Pixhawk flight controller [20] for autonomous navigation and stabilization under ArduPilot firmware [29], along with an NVIDIA Jetson Orin Nano [19] for onboard perception and real-time data processing. The propulsion system includes 1000KV brushless motors and 30A ESCs, powered by a 5200 mAh LiPo battery.

The IMX219 CSI camera is interfaced with the Jetson Orin Nano [19] to capture real-time video, which is processed using the fine-tuned YOLOv8s model [11] for victim detection. The annotated detection stream is transmitted to the Ground Control Station via HTTPS encryption [48], while flight control and telemetry are managed independently through the MAVLink communication link [39] via Mission Planner [35]. This integrated setup enables simultaneous autonomous navigation, real-time perception, and operator monitoring during mission execution.

The final assembled UAV demonstrates the practical realization of the SwiftRescue system, combining hardware components, onboard AI, and dual-channel communication infrastructure into a unified operational platform. The system is designed to support controlled autonomous missions with real-time YOLOv8s-based victim detection [11] and continuous operator supervision through the GCS.



Figure 21: Final Assembled SwiftRescue UAV Platform

4.5 Testing Details

This section presents the testing methodology and results for the SwiftRescue UAV Swarm System, covering both simulation-based validation and hardware-oriented testing. Testing is organized into two complementary phases that together provide comprehensive evaluation of system functionality, reliability, and readiness for real-world deployment. The first phase focuses on simulation-based testing conducted within the Gazebo Harmonic environment [26], where the complete system stack — including ArduPilot SITL [30], ROS 2 Jazzy [27], the YOLOv8s perception pipeline [11], and Mission Planner GCS [35] — is evaluated under controlled and repeatable conditions. The second phase addresses hardware-oriented testing of the physical UAV platform, validating power systems, sensor calibration, propulsion, communication, and flight behavior on real hardware. Together, these two testing phases ensure that the system meets the functional and non-functional requirements defined in Chapter 2 and that the physical platform is prepared for controlled autonomous operation.

4.5.1 Simulation-Based Testing and Validation

Simulation-based testing forms the primary validation phase of the SwiftRescue development process. It provides a safe, controlled, and repeatable environment to evaluate system functionality, component integration, multi-UAV coordination, communication reliability, and operator interaction without risk to hardware or human safety [26, 3]. Testing is structured across seven categories — functional, integration, multi-UAV swarm, communication reliability, failure and safety, performance observation, and user interaction — each targeting a distinct aspect of system behavior. All simulation tests were executed using the Gazebo Harmonic physics simulation [26], ArduPilot SITL [30], ROS 2 Jazzy middleware [27], the fine-tuned YOLOv8s perception pipeline [11], and Mission Planner as the Ground Control Station [35]. Results from this phase directly informed hardware integration decisions described in Section 4.5.2.

4.5.1.1 Functional Testing

Functional testing was performed to verify that the core functionalities of the SwiftRescue UAV Swarm System operate correctly within the simulation environment [26]. These tests validated mission configuration through Mission Planner [35], autonomous waypoint navigation using ArduPilot SITL [30], execution of the fine-tuned YOLOv8s perception pipeline [11] through ROS 2 nodes [27], and flight telemetry handling via MAVLink communication [39]. All tests were conducted in Gazebo-based simulation [26], where ROS 2 was used as middleware for internal sensor data exchange and YOLOv8s perception processing, to confirm correct system logic and compliance with the functional requirements defined in Chapter 2.

Table 2: Functional Testing: Mission Configuration (TC-FUNC-01)

Test Case ID	TC-FUNC-01
Test Objective	Verify successful mission configuration and initialization through Mission Planner GCS.
Preconditions	Mission Planner GCS is operational, Gazebo simulation is running, ROS 2 nodes are active.
Test Steps	• Open Mission Planner mission configuration interface. • Define search area boundaries and assign independent waypoint plans per UAV. • Save configuration and initialize mission.
Expected Result	Mission parameters are validated, independent UAV plans are saved, and mission initializes successfully.
Actual Result	Mission initialized successfully in Mission Planner with correct parameters and independent UAV waypoint plans confirmed.
Pass / Fail	Pass

Table 3: Functional Testing: Autonomous Navigation (TC-FUNC-02)

Test Case ID	TC-FUNC-02
Test Objective	Verify autonomous UAV waypoint navigation within mission boundaries using ArduPilot SITL.
Preconditions	Mission initialized in Mission Planner, UAVs spawned in Gazebo, ArduPilot SITL running.
Test Steps	• Issue launch command from Mission Planner GCS. • Observe UAV takeoff and waypoint-following behavior in Gazebo. • Verify UAV remains within assigned mission boundaries.
Expected Result	UAV follows assigned waypoints autonomously and remains within mission boundaries.
Actual Result	UAV followed waypoint sequence correctly within assigned mission area under ArduPilot control.
Pass / Fail	Pass

Table 4: Functional Testing: YOLOv8s Victim Detection Pipeline (TC-FUNC-03)

Test Case ID	TC-FUNC-03
Test Objective	Verify execution of the YOLOv8s victim detection pipeline using simulated Gazebo camera input.
Preconditions	UAV camera active in Gazebo, YOLOv8s perception node running via ROS 2, mission active.
Test Steps	• Allow UAV to capture image frames from the Gazebo simulated camera. • Process image frames through the YOLOv8s detection model via ROS 2 node. • Forward detection output to the GCS detection dashboard.
Expected Result	Victims are detected by the YOLOv8s model and detection results with bounding boxes are generated and transmitted to the GCS.
Actual Result	YOLOv8s detection results generated correctly and transmitted to the GCS detection dashboard with bounding boxes and confidence scores visible.
Pass / Fail	Pass

Table 5: Functional Testing: MAVLink Telemetry Transmission (TC-FUNC-04)

Test Case ID	TC-FUNC-04
Test Objective	Verify real-time MAVLink telemetry transmission from UAVs to Mission Planner GCS.
Preconditions	MAVLink communication active between ArduPilot SITL and Mission Planner, ROS 2 nodes running for internal sensor data exchange.
Test Steps	• Start UAV autonomous flight in Gazebo. • Monitor telemetry data (position, altitude, battery, flight mode) on the Mission Planner GCS dashboard.
Expected Result	UAV position, altitude, battery status, and flight mode are updated continuously in real time on Mission Planner.
Actual Result	MAVLink telemetry data displayed correctly and continuously on Mission Planner GCS with consistent update rate.
Pass / Fail	Pass

Table 6: Functional Testing: Mission Termination (TC-FUNC-05)

Test Case ID	TC-FUNC-05
Test Objective	Verify safe mission termination via abort command issued through Mission Planner GCS.
Preconditions	Mission active, UAVs airborne in Gazebo simulation, Mission Planner GCS operational.
Test Steps	• Issue mission abort command from Mission Planner. • Observe UAV response and transition to safe state in Gazebo.
Expected Result	UAVs stop mission execution and transition to safe state (RTL or hover) via ArduPilot failsafe.
Actual Result	Mission aborted successfully; UAVs transitioned to safe state via ArduPilot failsafe mechanisms as expected.
Pass / Fail	Pass

4.5.1.2 Integration Testing

Integration testing was conducted to verify that individual subsystems of the SwiftRescue UAV Swarm System interact correctly when combined within the simulation environment [26, 27]. These tests focused on validating data flow, command exchange, and coordinated behavior across Gazebo, ROS 2 middleware, ArduPilot SITL, YOLOv8s perception modules, GCS-based mission coordination, and Mission Planner [35]. In this architecture, MAVLink [39] handles flight control commands and telemetry exchange between ArduPilot SITL and Mission Planner, while ROS 2 [27] facilitates internal sensor data transmission and YOLOv8s perception processing within the simulation. Integration testing ensures that independently validated components operate reliably as a unified system during mission execution.

Table 7: Integration Testing: Gazebo–ROS 2 Sensor Integration (TC-INT-01)

Test Case ID	TC-INT-01
Test Objective	Verify correct transmission of simulated sensor data from Gazebo to ROS 2 topics via the ROS–Gazebo bridge.
Preconditions	Gazebo Harmonic simulation running, ROS 2 Jazzy nodes active, UAV model with camera, GPS, and IMU sensors enabled.
Test Steps	• Launch Gazebo simulation with multi-sensor UAV model. • Monitor ROS 2 topics for camera image, GPS position, and IMU data publication.
Expected Result	Sensor data from Gazebo is published correctly on corresponding ROS 2 topics at expected rates.
Actual Result	Camera, GPS, and IMU sensor data received successfully by ROS 2 nodes via the Gazebo–ROS 2 bridge at consistent rates.
Pass / Fail	Pass

Table 8: Integration Testing: ROS 2 and ArduPilot SITL Communication (TC-INT-02)

Test Case ID	TC-INT-02
Test Objective	Verify MAVLink-based command and telemetry exchange between ArduPilot SITL and Mission Planner via the ROS 2–MAVLink bridge.
Preconditions	ArduPilot SITL running, MAVLink bridge active between ROS 2 and ArduPilot SITL, Mission Planner connected.
Test Steps	• Upload waypoint commands to ArduPilot SITL via MAVLink from Mission Planner. • Monitor telemetry feedback from ArduPilot SITL on Mission Planner GCS.
Expected Result	ArduPilot SITL executes uploaded waypoints correctly and publishes telemetry back to Mission Planner via MAVLink.
Actual Result	Waypoint commands executed by ArduPilot SITL and telemetry received correctly in Mission Planner without communication errors.
Pass / Fail	Pass

Table 9: Integration Testing: YOLOv8s Perception Module and GCS Detection Dashboard (TC-INT-03)

Test Case ID	TC-INT-03
Test Objective	Verify transmission and visualization of YOLOv8s victim detection results from perception module to the GCS detection dashboard.
Preconditions	YOLOv8s perception node active via ROS 2, UAV simulated camera enabled, mission running in Gazebo.
Test Steps	• Run YOLOv8s victim detection pipeline during simulated flight over victim models. • Observe detection results and bounding boxes on the GCS detection dashboard.
Expected Result	YOLOv8s detection results are forwarded and visualized correctly with bounding boxes and confidence scores on the GCS dashboard.
Actual Result	Detection results and annotated bounding boxes displayed accurately on the GCS detection dashboard with correct confidence values.
Pass / Fail	Pass

Table 10: Integration Testing: Multi-UAV Telemetry Aggregation at GCS (TC-INT-04)

Test Case ID	TC-INT-04
Test Objective	Verify aggregation of independent MAVLink telemetry and YOLOv8s detection data from multiple UAVs at Mission Planner GCS.
Preconditions	Multiple UAVs spawned with unique MAVLink system IDs, Mission Planner GCS connected to all UAVs.
Test Steps	• Launch multiple UAVs executing independently assigned mission plans. • Monitor combined telemetry and detection data for all UAVs on Mission Planner GCS.
Expected Result	Mission Planner GCS displays aggregated telemetry for all active UAVs, each identified by unique system ID.
Actual Result	Telemetry from all UAVs aggregated and displayed correctly in Mission Planner, with each UAV identifiable by its unique MAVLink system ID.
Pass / Fail	Pass

Table 11: Integration Testing: End-to-End Mission Execution (TC-INT-05)

Test Case ID	TC-INT-05
Test Objective	Verify complete mission workflow from configuration through autonomous execution to safe termination.
Preconditions	All system components active: Gazebo, ArduPilot SITL, ROS 2, YOLOv8s perception node, Mission Planner GCS.
Test Steps	• Configure and upload independent mission plans via Mission Planner. • Launch UAVs and monitor autonomous execution in Gazebo. • Observe YOLOv8s detection outputs on GCS dashboard. • Terminate mission safely via Mission Planner abort command.
Expected Result	Complete mission workflow executes correctly with consistent telemetry, detection, and safe termination.
Actual Result	End-to-end mission completed without errors; all components interacted correctly throughout execution and termination.
Pass / Fail	Pass

4.5.1.3 Multi-UAV Swarm Testing

Multi-UAV swarm testing was performed to validate coordinated operation of multiple UAVs within the same simulated disaster environment [26, 8]. These tests focused on verifying correct deployment, independent control, distributed area coverage, and aggregation of telemetry and detection data across the swarm. Swarm coordination in simulation is achieved through GCS-based mission planning via Mission Planner [35], where each UAV is assigned an independent waypoint-based mission plan, rather than through decentralized inter-UAV communication. All tests were conducted in the Gazebo-based simulation using ROS 2 [27] for internal sensor data exchange and ArduPilot SITL [30] for autonomous flight control.

Table 12: Multi-UAV Swarm Testing: Multi-UAV Deployment (TC-SWARM-01)

Test Case ID	TC-SWARM-01
Test Objective	Verify successful deployment of multiple UAVs within the same Gazebo simulation environment with unique identifiers.
Preconditions	Gazebo Harmonic environment running, Mission Planner GCS connected, ArduPilot SITL instances active for each UAV.
Test Steps	• Configure a mission with multiple UAVs in Mission Planner, each with a unique MAVLink system ID. • Launch the mission from Mission Planner GCS. • Observe UAV spawning, initialization, and identification in Gazebo.
Expected Result	All UAVs spawn successfully in Gazebo with unique namespaces and unique MAVLink system IDs, each independently connected to Mission Planner.
Actual Result	Multiple UAVs deployed correctly in Gazebo with unique namespaces; each UAV independently identified and monitored in Mission Planner.
Pass / Fail	Pass

Table 13: Multi-UAV Swarm Testing: Independent UAV Control (TC-SWARM-02)

Test Case ID	TC-SWARM-02
Test Objective	Verify that each UAV executes its independently assigned mission plan without interfering with others.
Preconditions	Multiple UAVs launched in Gazebo, each with unique MAVLink system ID and independently assigned waypoint plan via Mission Planner.
Test Steps	• Observe individual UAV waypoint-following behavior during concurrent mission execution in Gazebo. • Verify that ROS 2 control topics and MAVLink telemetry topics remain separated per UAV namespace.
Expected Result	Each UAV navigates its assigned waypoints autonomously without interference from or dependency on other UAVs.
Actual Result	UAVs operated independently with correct namespace isolation; no cross-UAV interference observed in ROS 2 topics or MAVLink streams.
Pass / Fail	Pass

Table 14: Multi-UAV Swarm Testing: Distributed Area Coverage (TC-SWARM-03)

Test Case ID	TC-SWARM-03
Test Objective	Verify distributed and non-overlapping area coverage by the UAV swarm through independent mission plan assignment.
Preconditions	GCS-based mission planning active, multiple UAVs in flight with non-overlapping coverage zones assigned per UAV.
Test Steps	• Execute a mission where each UAV covers a distinct non-overlapping zone of the disaster area. • Observe UAV flight paths, coverage distribution, and separation in Gazebo.
Expected Result	UAVs distribute coverage across distinct non-overlapping zones without collision, collectively covering the full search area.
Actual Result	Distributed coverage achieved across assigned zones; collision-free operation confirmed through visual observation of Gazebo flight paths.
Pass / Fail	Pass

Table 15: Multi-UAV Swarm Testing: Multi-UAV Telemetry Aggregation (TC-SWARM-04)

Test Case ID	TC-SWARM-04
Test Objective	Verify aggregation of independent MAVLink telemetry streams from all active UAVs at the Mission Planner GCS.
Preconditions	Mission Planner GCS running, multiple UAV independent MAVLink telemetry streams active.
Test Steps	• Monitor telemetry data for all UAVs simultaneously during multi-UAV mission execution. • Verify that all UAV states are correctly displayed and distinguishable on the GCS map.
Expected Result	Mission Planner GCS displays aggregated and distinguishable telemetry for all active UAVs without data loss or stream confusion.
Actual Result	Telemetry from all UAVs aggregated and displayed correctly in Mission Planner; each UAV distinguishable by unique MAVLink system ID with no stream loss observed.
Pass / Fail	Pass

Table 16: Multi-UAV Swarm Testing: Swarm Scalability Observation (TC-SWARM-05)

Test Case ID	TC-SWARM-05
Test Objective	Observe system behavior and GCS responsiveness under varying swarm sizes.
Preconditions	Gazebo simulation configured for multiple swarm sizes; Mission Planner GCS connected to all UAV instances.
Test Steps	• Execute missions with 2 and 3 UAVs in separate runs. • Monitor GCS responsiveness, MAVLink telemetry stability, and Gazebo simulation performance as UAV count increases.
Expected Result	System remains stable with acceptable GCS responsiveness and telemetry clarity as swarm size increases.
Actual Result	Stable operation observed for all tested swarm sizes; minor increase in simulation computational load noted with 3 UAVs but system remained operational throughout.
Pass / Fail	Pass

4.5.1.4 Communication Reliability Testing

Communication reliability testing was conducted to evaluate the consistency and stability of MAVLink telemetry, YOLOv8s detection data, and operator command exchange within the SwiftRescue simulation environment [39, 26]. These tests focused on verifying continuous MAVLink telemetry streaming to Mission Planner [35], responsiveness to operator-issued commands, and reliable detection data delivery during multi-UAV missions. All tests were performed using ROS 2 middleware [27] for internal sensor exchange and MAVLink-based communication [39] between ArduPilot SITL and Mission Planner in Gazebo simulation, emphasizing observation of communication behavior rather than physical network characteristics.

Table 17: Communication Reliability: Continuous MAVLink Telemetry Streaming (TC-COMM-01)

Test Case ID	TC-COMM-01
Test Objective	Verify continuous MAVLink telemetry transmission from ArduPilot SITL to Mission Planner GCS during mission execution.
Preconditions	UAVs in autonomous flight in Gazebo, MAVLink link established, Mission Planner GCS running.
Test Steps	• Launch a mission with one or more UAVs in Gazebo. • Monitor MAVLink telemetry data on Mission Planner GCS over an extended period.
Expected Result	MAVLink telemetry updates continuously without interruption or data gaps throughout mission execution.
Actual Result	MAVLink telemetry stream remained stable throughout the entire mission; no connection drops or telemetry gaps observed.
Pass / Fail	Pass

Table 18: Communication Reliability: YOLOv8s Detection Data Transmission (TC-COMM-02)

Test Case ID	TC-COMM-02
Test Objective	Verify reliable transmission of YOLOv8s detection results from the perception pipeline to the GCS detection dashboard.
Preconditions	YOLOv8s perception node active via ROS 2, mission running, victim models present in Gazebo environment.
Test Steps	• Execute YOLOv8s victim detection during simulated flight over victim models. • Observe detection result updates and annotated frames on the GCS detection dashboard.
Expected Result	YOLOv8s detection results are delivered to and visualized on the GCS detection dashboard without data loss.
Actual Result	Detection results received and displayed on the GCS detection dashboard without loss; bounding boxes and confidence scores updated correctly per detection event.
Pass / Fail	Pass

Table 19: Communication Reliability: Operator Command Responsiveness via MAVLink (TC-COMM-03)

Test Case ID	TC-COMM-03
Test Objective	Verify UAV response to high-level operator commands issued from Mission Planner via MAVLink.
Preconditions	UAV in autonomous flight in Gazebo, MAVLink command link active between Mission Planner and ArduPilot SITL.
Test Steps	• Issue navigation or flight mode change command from Mission Planner via MAVLink. • Observe UAV response and state transition in Gazebo simulation.
Expected Result	UAV responds promptly to operator-issued MAVLink commands without noticeable delay.
Actual Result	UAV executed MAVLink commands from Mission Planner without noticeable delay; correct flight mode transitions observed in Gazebo.
Pass / Fail	Pass

Table 20: Communication Reliability: Multi-UAV MAVLink Telemetry Stability (TC-COMM-04)

Test Case ID	TC-COMM-04
Test Objective	Verify stability and integrity of independent MAVLink telemetry streams during simultaneous multi-UAV operation.
Preconditions	Multiple UAVs active in Gazebo with unique MAVLink system IDs, Mission Planner GCS monitoring all UAVs.
Test Steps	• Launch multiple UAVs executing independent mission plans simultaneously. • Monitor independent MAVLink telemetry streams for all UAVs on Mission Planner GCS.
Expected Result	Independent MAVLink telemetry streams for all UAVs remain stable and correctly associated with their respective system IDs.
Actual Result	No telemetry drop or stream cross-contamination observed during simultaneous multi-UAV operation; all telemetry streams remained stable and correctly associated throughout the mission.
Pass / Fail	Pass

Table 21: Communication Reliability: Extended Mission Duration Stability (TC-COMM-05)

Test Case ID	TC-COMM-05
Test Objective	Observe MAVLink telemetry and detection data communication stability during long-duration simulated missions.
Preconditions	Extended mission configured in Mission Planner, all Gazebo, ROS 2, and ArduPilot SITL nodes active.
Test Steps	• Execute an extended mission in Gazebo simulation. • Monitor MAVLink telemetry and YOLOv8s detection streams for stability over the full mission duration.
Expected Result	Both MAVLink telemetry and detection data communication remain stable for the full duration of the extended mission.
Actual Result	Stable MAVLink telemetry and detection data communication observed throughout extended simulation run; no stream drops or connection failures recorded.
Pass / Fail	Pass

4.5.1.5 Failure and Safety Testing

Failure and safety testing was conducted to evaluate the robustness of the SwiftRescue UAV Swarm System under abnormal and fault conditions within the simulation environment [29, 26]. These tests focused on verifying safe mission termination via ArduPilot failsafe mechanisms [29], system behavior during partial UAV failures, and correct handling of emergency commands issued through Mission Planner [35]. All tests were executed in Gazebo-based simulation using ROS 2 [27] and ArduPilot SITL [30], emphasizing system-level safety behavior rather than physical hardware protection.

Table 22: Failure and Safety Testing: Mission Abort Command (TC-SAFE-01)

Test Case ID	TC-SAFE-01
Test Objective	Verify safe mission termination upon abort command issued via Mission Planner.
Preconditions	Mission active, UAVs in autonomous flight in Gazebo, Mission Planner GCS operational with active MAVLink link.
Test Steps	• Start an active autonomous mission in Gazebo. • Issue abort command from Mission Planner via MAVLink. • Observe UAV response and transition in Gazebo.
Expected Result	UAVs stop mission execution and enter a safe state (RTL or hover) via ArduPilot failsafe mechanisms.
Actual Result	Mission aborted successfully; UAVs transitioned to safe state via ArduPilot failsafe as expected with no instability observed in Gazebo.
Pass / Fail	Pass

Table 23: Failure and Safety Testing: Partial UAV Failure (TC-SAFE-02)

Test Case ID	TC-SAFE-02
Test Objective	Observe system behavior when one UAV stops responding during a multi-UAV mission.
Preconditions	Multiple UAVs executing independent mission plans in Gazebo, Mission Planner GCS monitoring all UAVs.
Test Steps	• Disable or pause one UAV instance during active multi-UAV mission execution. • Observe behavior of remaining UAVs and Mission Planner GCS response.
Expected Result	Remaining UAVs continue executing their independently assigned mission plans without system failure.
Actual Result	Mission continued with remaining UAVs completing their assigned plans independently; Mission Planner GCS showed telemetry loss for the disabled UAV only.
Pass / Fail	Pass

Table 24: Failure and Safety Testing: Communication Interruption Recovery (TC-SAFE-03)

Test Case ID	TC-SAFE-03
Test Objective	Verify system recovery after temporary MAVLink communication interruption.
Preconditions	Mission running in Gazebo, MAVLink telemetry active between ArduPilot SITL and Mission Planner.
Test Steps	• Temporarily interrupt the MAVLink telemetry link to Mission Planner. • Restore the communication link and observe system recovery.
Expected Result	System resumes normal MAVLink telemetry and mission operation after communication link is restored.
Actual Result	MAVLink communication recovered successfully after link restoration; mission operation continued and telemetry resumed without requiring restart.
Pass / Fail	Pass

Table 25: Failure and Safety Testing: ArduPilot Emergency Mode Switching (TC-SAFE-04)

Test Case ID	TC-SAFE-04
Test Objective	Verify UAV response to emergency flight mode switching via ArduPilot during simulated mission.
Preconditions	UAV in autonomous flight in Gazebo, ArduPilot SITL active, Mission Planner connected via MAVLink.
Test Steps	• Trigger emergency flight mode change (e.g., from AUTO to LAND) via Mission Planner during active mission. • Observe UAV flight mode transition behavior in Gazebo.
Expected Result	UAV transitions to the commanded emergency flight mode safely without instability or unexpected behavior.
Actual Result	Safe and stable flight mode transition observed in Gazebo; ArduPilot executed mode change correctly with no instability.
Pass / Fail	Pass

Table 26: Failure and Safety Testing: Mission Termination Event Logging (TC-SAFE-05)

Test Case ID	TC-SAFE-05
Test Objective	Verify that mission termination and failure events are correctly logged at the GCS.
Preconditions	Mission Planner GCS data logging enabled, mission active in Gazebo simulation.
Test Steps	• Terminate mission using both normal completion and abort command conditions. • Review Mission Planner GCS logs for recorded termination events.
Expected Result	Mission termination events including abort triggers, flight mode changes, and landing events are logged correctly at the GCS.
Actual Result	Mission Planner GCS logs recorded all termination events correctly including abort command timestamp, mode transitions, and final UAV state.
Pass / Fail	Pass

4.5.1.6 Performance Observation Testing

Performance observation testing was conducted to qualitatively evaluate the runtime behavior and stability of the SwiftRescue UAV Swarm System during simulation-based mission execution [26]. Since the system is validated primarily in a simulated environment, these tests focus on observing MAVLink telemetry responsiveness, YOLOv8s detection output latency, Mission Planner visualization smoothness, and overall system stability rather than precise hardware-level performance metrics. This approach is adopted because simulation environments do not accurately reflect real-world hardware constraints such as physical network latency and edge processing limitations. The goal of this testing is to ensure that the system operates smoothly under normal mission conditions and remains stable during extended operation.

Table 27: Performance Observation: MAVLink Telemetry Update Rate (TC-PERF-01)

Test Case ID	TC-PERF-01
Test Objective	Observe continuity and update rate of MAVLink telemetry from ArduPilot SITL on Mission Planner GCS.
Preconditions	UAV autonomous mission active in Gazebo, MAVLink telemetry streaming to Mission Planner GCS.
Test Steps	• Launch UAV autonomous mission in Gazebo. • Observe live position and status update rate on Mission Planner GCS throughout the mission.
Expected Result	MAVLink telemetry updates appear continuous and responsive at acceptable rate during mission execution.
Actual Result	MAVLink telemetry updates displayed smoothly and continuously on Mission Planner GCS without noticeable interruption or lag throughout the mission.
Pass / Fail	Pass

Table 28: Performance Observation: UAV Position Visualization Smoothness (TC-PERF-02)

Test Case ID	TC-PERF-02
Test Objective	Observe smoothness of UAV position and trajectory visualization on the Mission Planner GCS map interface.
Preconditions	UAVs navigating autonomously in Gazebo, Mission Planner GCS map visualization active.
Test Steps	• Monitor UAV trajectories on the Mission Planner GCS map view during autonomous flight. • Observe rendering quality during turns and waypoint-to-waypoint transitions.
Expected Result	UAV position updates and trajectory visualization appear smooth without abrupt jumps or rendering freezes.
Actual Result	UAV motion visualized smoothly on Mission Planner map throughout mission; no rendering freezes or position jumps observed.
Pass / Fail	Pass

Table 29: Performance Observation: YOLOv8s Detection Output Latency (TC-PERF-03)

Test Case ID	TC-PERF-03
Test Objective	Observe the observable delay between YOLOv8s detection event occurrence and visualization on the GCS detection dashboard.
Preconditions	YOLOv8s perception pipeline active via ROS 2, victim models present in Gazebo, GCS detection dashboard running.
Test Steps	• Fly UAV directly over victim models in Gazebo. • Observe time elapsed until YOLOv8s detection result appears on the GCS detection dashboard.
Expected Result	YOLOv8s detection outputs appear on the GCS dashboard within acceptable simulation response time with bounding boxes visible.
Actual Result	YOLOv8s detection results displayed on GCS detection dashboard with minimal observable latency; bounding boxes and confidence scores updated promptly upon victim detection.
Pass / Fail	Pass

Table 30: Performance Observation: Multi-UAV Simulation System Stability (TC-PERF-04)

Test Case ID	TC-PERF-04
Test Objective	Observe system stability across Gazebo, ROS 2 nodes, ArduPilot SITL, and Mission Planner during extended multi-UAV mission execution.
Preconditions	Multiple UAVs active in Gazebo, all ROS 2 nodes and ArduPilot SITL instances running, Mission Planner GCS monitoring.
Test Steps	• Run a multi-UAV mission for an extended simulation duration. • Observe stability of Gazebo physics simulation, ROS 2 node execution, ArduPilot SITL, and Mission Planner GCS throughout.
Expected Result	All system components remain stable without crashes, node failures, or unexpected state changes during extended operation.
Actual Result	System remained stable throughout extended multi-UAV simulation; no component crashes or unexpected state transitions observed, though minor computational load increase noted with higher UAV counts.
Pass / Fail	Pass

Table 31: Performance Observation: Operator Command Response Time via MAVLink (TC-PERF-05)

Test Case ID	TC-PERF-05
Test Objective	Observe UAV response time to operator-issued high-level commands during active mission.
Preconditions	UAV autonomous mission active in Gazebo, Mission Planner GCS control enabled, MAVLink command link active.
Test Steps	• Issue high-level commands (flight mode change, waypoint update) from Mission Planner via MAVLink during mission. • Observe UAV response behavior and state transition in Gazebo.
Expected Result	UAVs respond promptly to Mission Planner MAVLink commands with observable state changes within acceptable simulation response time.
Actual Result	UAVs responded consistently and promptly to all Mission Planner commands; correct state transitions observed in Gazebo without delay.
Pass / Fail	Pass

4.5.1.7 User Interaction and Usability Testing

User interaction and usability testing was conducted to evaluate how effectively a human operator can interact with the SwiftRescue UAV Swarm System through the dual-interface Ground Control Station during simulation-based missions [35, 3]. These tests focus on clarity of Mission Planner telemetry visualization, ease of mission configuration

and control, interpretability of YOLOv8s detection results [11] on the GCS detection dashboard, and the operator's ability to supervise autonomous UAV behavior without excessive manual intervention. Since SwiftRescue is designed as a human-in-the-loop system, usability testing ensures that autonomy is complemented by intuitive and reliable operator interaction.

Table 32: User Interaction Testing: Mission Configuration via Mission Planner GCS (TC-UI-01)

Test Case ID	TC-UI-01
Test Objective	Verify ease of mission configuration and independent waypoint plan assignment using Mission Planner GCS interface.
Preconditions	Mission Planner GCS running, Gazebo simulation environment initialized, ArduPilot SITL active.
Test Steps	• Open Mission Planner mission configuration panel. • Define search area, partition coverage zones, and assign independent waypoint plans per UAV. • Initialize and upload mission plans.
Expected Result	Operator can configure search area, partition coverage, and initialize independent mission plans per UAV without ambiguity.
Actual Result	Mission configuration completed clearly and successfully in Mission Planner; independent waypoint plans assigned and uploaded to each UAV without difficulty.
Pass / Fail	Pass

Table 33: User Interaction Testing: Mission Planner Telemetry Clarity (TC-UI-02)

Test Case ID	TC-UI-02
Test Objective	Evaluate clarity and interpretability of UAV telemetry and position visualization on Mission Planner GCS.
Preconditions	UAVs active in Gazebo, MAVLink telemetry streaming to Mission Planner GCS.
Test Steps	• Observe live UAV positions on Mission Planner map and status panel indicators. • Monitor mission progress and telemetry data (altitude, battery, flight mode) during autonomous flight.
Expected Result	Telemetry data and UAV positions are clearly and intuitively presented without requiring interpretation effort.
Actual Result	UAV status indicators and positions displayed clearly in Mission Planner; operators could identify each UAV and interpret telemetry without difficulty.
Pass / Fail	Pass

Table 34: User Interaction Testing: YOLOv8s Detection Result Interpretation (TC-UI-03)

Test Case ID	TC-UI-03
Test Objective	Verify ease of interpreting YOLOv8s victim detection results on the GCS detection dashboard during mission.
Preconditions	YOLOv8s detection pipeline active via ROS 2, victim models present in Gazebo, GCS detection dashboard running.
Test Steps	• Observe YOLOv8s detection markers, bounding boxes, and confidence scores on the GCS detection dashboard during mission execution. • Correlate detection locations with corresponding UAV positions shown in Mission Planner.
Expected Result	YOLOv8s detection information including bounding boxes and confidence scores is clearly and intuitively presented on the GCS dashboard.
Actual Result	YOLOv8s detection markers and bounding boxes displayed clearly on GCS dashboard; operators could correlate detections with UAV positions in Mission Planner without difficulty.
Pass / Fail	Pass

Table 35: User Interaction Testing: Operator Intervention During Autonomous Mission (TC-UI-04)

Test Case ID	TC-UI-04
Test Objective	Verify operator ability to issue intervention commands via Mission Planner without disrupting ongoing autonomous operation.
Preconditions	Autonomous mission active in Gazebo, Mission Planner GCS control enabled and MAVLink link active.
Test Steps	• Issue a high-level intervention command (e.g., flight mode change or waypoint update) from Mission Planner via MAVLink during active autonomous mission. • Observe UAV response behavior and mission continuation in Gazebo.
Expected Result	Operator commands are executed correctly via MAVLink without disrupting the overall autonomous operation of other UAVs.
Actual Result	UAVs responded correctly to operator intervention commands; other UAVs continued their independent mission plans without disruption.
Pass / Fail	Pass

Table 36: User Interaction Testing: Overall Dual-Interface GCS Usability (TC-UI-05)

Test Case ID	TC-UI-05
Test Objective	Assess overall usability of the dual-interface GCS (Mission Planner + detection dashboard) during complete mission supervision.
Preconditions	Complete mission executed in Gazebo simulation; both Mission Planner and GCS detection dashboard active simultaneously.
Test Steps	• Supervise a complete multi-UAV mission from configuration through execution to safe termination. • Observe cognitive load, clarity of simultaneous telemetry and detection information, and ease of interaction with both GCS interfaces.
Expected Result	Operator can supervise the complete mission using both GCS interfaces with minimal cognitive load and without requiring context switching.
Actual Result	Dual-interface GCS enabled effective mission supervision; operators could monitor Mission Planner telemetry and GCS detection dashboard simultaneously without significant cognitive overhead.
Pass / Fail	Pass

4.5.2 Hardware-Oriented Testing Considerations and System Readiness

Hardware-oriented testing was conducted to validate the physical reliability, electrical stability, and operational readiness of the SwiftRescue UAV platform prior to deployment [20, 29]. Unlike simulation testing, which focuses on software behavior and system logic, hardware testing emphasizes real-world performance under practical constraints such as power fluctuations, sensor accuracy, communication reliability, and flight stability. Testing was organized across seven categories covering power systems, flight controller calibration, propulsion, communication, flight stability, autonomous flight modes, and system safety validation.

These tests were performed in a controlled indoor environment to ensure safe operation while verifying that all critical components — including the power system, Pixhawk flight controller [20], propulsion subsystem, NVIDIA Jetson Orin Nano [19], and MAVLink communication link [39] — function correctly under expected operating conditions. Particular attention was given to ensuring stable power delivery to both the Pixhawk and the Jetson Orin Nano, verifying safe motor operation, and confirming communication consistency under varying load conditions.

The objective of this testing phase is to confirm that the UAV system is electrically stable, mechanically sound, and capable of supporting controlled autonomous operation. Results are reported using three outcome categories: **Pass** for tests meeting all expected criteria, **Pass with Observation** for tests that passed but revealed minor deviations warranting attention, and **Conditional Pass** for tests that passed within acceptable limits but identified areas requiring further tuning or environmental consideration before full-scale deployment.

4.5.2.1 Power System and Electrical Testing

Power system and electrical testing was performed to ensure stable and reliable energy distribution across all UAV components, including the Pixhawk flight controller [20], ESCs, brushless motors, and the NVIDIA Jetson Orin Nano [19]. These tests verify that the system can operate safely under varying load conditions without critical voltage instability or power interruption during autonomous mission execution [29].

Table 37: Power System Test Case TC-UAV-PS-01: Battery Voltage Verification

Test Case ID	TC-UAV-PS-01
Test Objective	Verify that the 3S LiPo battery provides correct nominal voltage output at full charge.
Preconditions	Battery fully charged (3S LiPo, 5200 mAh) and connected to the UAV power distribution system.
Test Steps	• Measure battery terminal voltage using a calibrated multimeter. • Connect battery to the UAV power distribution board and verify voltage at input.
Expected Result	Battery voltage within nominal 3S LiPo fully-charged range of 11.1V–12.6V.
Actual Result	Measured voltage consistently between 12.4V–12.6V at initialization, confirming full charge and correct cell configuration.
Pass / Fail	**Pass**

Table 38: Power System Test Case TC-UAV-PS-02: Voltage Stability Under Motor Load

Test Case ID	TC-UAV-PS-02
Test Objective	Evaluate voltage stability and load-induced drop during motor operation.
Preconditions	UAV fully powered, motors armed via Mission Planner [35], propellers removed for safety.
Test Steps	• Gradually increase throttle to approximately 50% via Mission Planner. • Monitor battery voltage continuously during motor operation using Mission Planner power telemetry.
Expected Result	Voltage drop under load remains within safe limits (less than 1.0V drop from resting voltage).
Actual Result	Voltage dropped from 12.5V to approximately 11.7V under 50% throttle load (0.8V drop). Minor switching ripple of approximately ±0.1V observed due to ESC PWM switching, consistent with expected behavior for unregulated distribution.
Pass / Fail	**Pass with Observation** – voltage drop within limits; ripple expected and acceptable but noted for awareness.

Table 39: Power System Test Case TC-UAV-PS-03: Power Distribution Verification

Test Case ID	TC-UAV-PS-03
Test Objective	Verify stable power delivery to all system components via the power distribution board.
Preconditions	Full system assembled and powered; Pixhawk, ESCs, and Jetson Orin Nano [19] all connected.
Test Steps	• Measure voltage at Pixhawk power input, ESC power rails, and Jetson Orin Nano regulated input. • Verify wiring connections and confirm stable readings across all modules.
Expected Result	All components receive stable and consistent regulated voltage within their specified input ranges.
Actual Result	Stable voltage confirmed across all modules with minor variation of less than 0.2V. Jetson Orin Nano [19] received stable regulated input through dedicated voltage regulator module, with no observed instability.
Pass / Fail	**Pass**

Table 40: Power System Test Case TC-UAV-PS-04: ESC Electrical Response Testing

Test Case ID	TC-UAV-PS-04
Test Objective	Validate ESC electrical response and startup behavior across the throttle range.
Preconditions	ESCs calibrated and connected to Pixhawk [20]; propellers removed for safety.
Test Steps	• Arm UAV via Mission Planner and gradually increase throttle from 0% to 60%. • Observe motor startup behavior, timing, and speed synchronization across all four ESCs.
Expected Result	Smooth and synchronized motor startup and speed increase across all ESCs.
Actual Result	Motors responded with a startup delay of approximately 0.2–0.3 seconds, consistent with ESC initialization behavior. Speed synchronization was maintained at throttle levels above 15%. One ESC showed marginally longer startup delay but stabilized within one throttle cycle.
Pass / Fail	**Pass with Observation** – minor startup timing variation in one ESC noted; recommend re-verifying ESC calibration if delay persists under load.

Table 41: Power System Test Case TC-UAV-PS-05: Jetson Orin Nano Power Integration

Test Case ID	TC-UAV-PS-05
Test Objective	Ensure stable and uninterrupted power supply to the NVIDIA Jetson Orin Nano during active detection workload.
Preconditions	Jetson Orin Nano [19] powered via regulated supply module from LiPo battery; YOLOv8s detection pipeline [11] ready to run.
Test Steps	• Power on the Jetson Orin Nano and allow full system boot. • Launch the YOLOv8s detection pipeline and HTTPS streaming service under active inference load. • Monitor system stability for 10–15 minutes under continuous inference.
Expected Result	Continuous operation without reboot, shutdown, or power instability during active YOLOv8s inference.
Actual Result	Stable operation confirmed over 10–15 minute runtime with no shutdown or reboot events. GPU utilization reached approximately 70–80% during peak inference. Brief thermal throttling observed after 12 minutes of continuous operation, causing a temporary FPS reduction from approximately 18 to 14 FPS, recovering within 60 seconds. Recommend monitoring thermal performance during extended outdoor deployments.
Pass / Fail	**Pass with Observation** – stable operation confirmed; thermal throttling under sustained load noted as consideration for long-duration missions.

4.5.2.2 Flight Controller and Sensor Calibration Testing

Flight controller and sensor calibration testing was performed to ensure accurate state estimation, stable flight behavior, and reliable sensor integration within the UAV system [25]. The Pixhawk flight controller [20] relies on correctly calibrated sensors — including the accelerometer, gyroscope, compass, and GPS — to maintain precise navigation and control under ArduPilot [29]. These tests were conducted using Mission Planner [35] in a controlled indoor and semi-open environment. Minor variations due to environmental factors such as magnetic interference from onboard electronics and surface leveling inaccuracies were observed and are reported below.

Table 42: Flight Controller Test Case TC-UAV-FC-01: IMU Calibration Verification

Test Case ID	TC-UAV-FC-01
Test Objective	Calibrate accelerometer and gyroscope sensors for stable attitude and orientation estimation.
Preconditions	Pixhawk [20] connected to Mission Planner [35] and placed on a level surface away from vibration sources.
Test Steps	• Initiate IMU calibration routine in Mission Planner. • Place UAV in all six required orientations as prompted by the calibration interface. • Complete and save calibration parameters.
Expected Result	Accurate orientation readings with minimal gyroscope drift during idle state.
Actual Result	Stable IMU readings confirmed after calibration. Residual gyroscope drift of less than 0.5° observed during static idle state, within acceptable ArduPilot operational limits [29]. Calibration accepted successfully on first attempt.
Pass / Fail	**Pass**

Table 43: Flight Controller Test Case TC-UAV-FC-02: Compass Calibration

Test Case ID	TC-UAV-FC-02
Test Objective	Establish accurate heading estimation through compass calibration, accounting for onboard magnetic interference.
Preconditions	UAV placed in a semi-open environment with reduced magnetic interference; Pixhawk connected to Mission Planner [35].
Test Steps	• Start compass calibration routine in Mission Planner. • Rotate UAV through all required axes as directed by the calibration interface. • Save calibration parameters on successful completion.
Expected Result	Compass heading error within acceptable operational range ($\leq 5^\circ$ deviation from reference).
Actual Result	Compass readings showed deviation of approximately 2°–4° from reference direction after calibration, attributed to residual magnetic interference from onboard ESC wiring and the power distribution board. Deviation remained within ArduPilot [29] operational tolerance. Outdoor re-calibration away from electronics is recommended before field deployment.
Pass / Fail	**Pass with Observation** – heading error within tolerance; recommend outdoor calibration before deployment for improved accuracy.

Table 44: Flight Controller Test Case TC-UAV-FC-03: GPS Signal Acquisition and Sensor Validation

Test Case ID	TC-UAV-FC-03
Test Objective	Verify GPS signal acquisition, satellite lock, and consistency of multi-sensor telemetry in an outdoor environment.
Preconditions	UAV powered on and placed in an outdoor or open-sky environment with clear view of the sky; Pixhawk connected to Mission Planner [35].
Test Steps	• Monitor GPS lock acquisition status and satellite count in Mission Planner. • Verify position data accuracy and consistency with known reference location. • Observe concurrent IMU and compass sensor outputs during GPS lock.
Expected Result	Stable GPS lock achieved with at least 8 satellites and consistent position and sensor telemetry.
Actual Result	GPS lock achieved with 9–11 satellites in outdoor environment after approximately 45–90 seconds from cold start. Consistent position telemetry confirmed in Mission Planner [25]. Note: GPS acquisition was not possible in indoor testing environment; all GPS-dependent tests required outdoor relocation. This is an inherent hardware constraint for deployment planning.
Pass / Fail	**Pass with Observation** – GPS performance confirmed outdoors; indoor GPS unavailability noted as deployment constraint requiring open-sky operating conditions.

4.5.2.3 Motor, ESC, and Propulsion Testing

Motor, ESC, and propulsion testing was conducted to verify correct motor rotation direction, ESC throttle response, and overall thrust generation behavior [22, 21]. These tests ensure that the UAV generates stable and balanced thrust, which is essential for safe takeoff, hovering, and maneuvering during autonomous missions. Proper CW/CCW motor configuration and ESC calibration are critical for preventing yaw instability or unintended rotation. All tests involving motors at elevated throttle were performed with propellers removed where safety required.

Table 45: Propulsion Test Case TC-UAV-MP-01: Motor Rotation Direction Verification

Test Case ID	TC-UAV-MP-01
Test Objective	Verify correct CW and CCW motor rotation direction for the quadrotor X-configuration.
Preconditions	Motors connected to ESCs and Pixhawk [20]; propellers removed; UAV armed via Mission Planner [35].
Test Steps	• Arm UAV via Mission Planner and gradually increase throttle to 10–20%. • Observe and verify rotation direction of each motor against the ArduPilot motor mapping diagram [29].
Expected Result	All four motors rotate in correct CW/CCW configuration matching the ArduPilot quadrotor X-frame layout.
Actual Result	Three motors confirmed correct rotation direction immediately. One motor required direction correction via ESC reconfiguration due to incorrect initial wiring. After correction, all four motors confirmed correct CW/CCW configuration. Startup delay of approximately 0.2 seconds observed consistently across all motors.
Pass / Fail	**Pass with Observation** – one motor required direction correction before test passed; wiring verification during assembly is recommended as a pre-deployment checklist item.

Table 46: Propulsion Test Case TC-UAV-MP-02: ESC Calibration and Throttle Response

Test Case ID	TC-UAV-MP-02
Test Objective	Verify ESC calibration accuracy and uniform throttle response across all motors.
Preconditions	ESC calibration mode enabled via Pixhawk [20]; propellers removed; all four ESCs connected.
Test Steps	• Perform ESC calibration procedure via Pixhawk using Mission Planner. • Gradually increase throttle from 0% to 60% and observe motor speed response and uniformity.
Expected Result	Smooth, synchronized, and uniform motor speed increase across all four ESCs throughout throttle range.
Actual Result	ESCs responded uniformly at throttle levels above 15%. Minor speed variation observed below 10% throttle (approximately 5–8% RPM difference between motors), attributed to ESC dead-band at very low throttle. Synchronization was consistent at mid and high throttle levels used during flight operations [22].
Pass / Fail	**Pass with Observation** – low-throttle variation is within acceptable ESC characteristics; all motors synchronized correctly at operational throttle levels.

Table 47: Propulsion Test Case TC-UAV-MP-03: Thrust Generation and Hover Readiness

Test Case ID	TC-UAV-MP-03
Test Objective	Verify sufficient and balanced thrust generation for stable lift-off and initial hover readiness.
Preconditions	Propellers attached and direction-verified; UAV secured to test stand or held safely in controlled indoor area.
Test Steps	• Arm UAV and gradually increase throttle while monitoring frame response. • Observe lift tendency, frame balance, and vibration characteristics.
Expected Result	UAV generates balanced upward thrust without lateral imbalance or excessive vibration.
Actual Result	Balanced upward thrust confirmed. Slight mechanical vibration noted at 40–50% throttle, attributed to minor propeller imbalance on two propellers. Vibration did not cause control instability but was detectable in IMU telemetry. Propeller balancing using a static balancer is recommended before field deployment to reduce vibration-induced sensor noise [21].
Pass / Fail	**Pass with Observation** – thrust generation sufficient; propeller balancing recommended to reduce IMU vibration noise before deployment.

4.5.2.4 Communication and Telemetry Testing

Communication and telemetry testing was performed to ensure reliable data exchange between the UAV and the Ground Control Station. These tests validate the MAVLink-based [39] telemetry link with Mission Planner [35] and the HTTPS-encrypted detection stream [48] from the NVIDIA Jetson Orin Nano [19]. Testing was conducted over a local WiFi network under controlled indoor conditions to evaluate latency, data consistency, and system responsiveness.

Table 48: Communication Test Case TC-UAV-CM-01: MAVLink Telemetry Link Verification

Test Case ID	TC-UAV-CM-01
Test Objective	Verify stable and continuous MAVLink telemetry communication between Pixhawk and Mission Planner GCS.
Preconditions	Telemetry radio or USB MAVLink link established between Pixhawk [20] and Mission Planner [35].
Test Steps	• Connect UAV to Mission Planner via MAVLink link. • Monitor live telemetry data including position, battery voltage, and system status.
Expected Result	Continuous and accurate MAVLink telemetry updates with consistent update rate and minimal latency.
Actual Result	Stable MAVLink telemetry link confirmed with update rate of approximately 5–10 Hz and round-trip latency of approximately 100–200 ms over the test link, consistent with expected MAVLink performance [39]. No connection drops observed during the test duration.
Pass / Fail	**Pass**

Table 49: Communication Test Case TC-UAV-CM-02: MAVLink Command Transmission Testing

Test Case ID	TC-UAV-CM-02
Test Objective	Validate reliable command transmission from Mission Planner GCS to UAV via MAVLink and confirm UAV response within acceptable delay.
Preconditions	UAV connected to Mission Planner [35] via active MAVLink link [39].
Test Steps	• Issue arm and disarm commands from Mission Planner. • Issue flight mode change commands and observe UAV response time.
Expected Result	UAV responds to all Mission Planner commands within acceptable round-trip delay.
Actual Result	All arm, disarm, and flight mode commands executed successfully. Response delay measured at approximately 150–300 ms, consistent with MAVLink protocol characteristics over the test link [39]. No missed or rejected commands observed during the test sequence.
Pass / Fail	**Pass**

Table 50: Communication Test Case TC-UAV-CM-03: YOLOv8s Detection Stream Transmission via HTTPS

Test Case ID	TC-UAV-CM-03
Test Objective	Verify real-time HTTPS-encrypted YOLOv8s detection stream transmission from Jetson Orin Nano to the GCS detection dashboard.
Preconditions	Jetson Orin Nano [19] connected to local WiFi network; YOLOv8s detection pipeline [11] and HTTPS streaming server [48] running.
Test Steps	• Start HTTPS detection streaming service on the Jetson Orin Nano. • Access encrypted stream via web browser on the GCS machine. • Observe stream quality, detection overlays, and latency under active inference load.
Expected Result	Live HTTPS-encrypted video stream with YOLOv8s detection overlays accessible with minimal latency.
Actual Result	HTTPS detection stream received successfully at the GCS browser. Stream latency measured at approximately 200–400 ms under active YOLOv8s inference load, with frame rate of approximately 15–20 FPS. Latency is acceptable for search and rescue monitoring where real-time video is used for victim identification rather than time-critical control. Self-signed SSL certificate warning appeared on first browser connection, requiring manual exception as expected for self-signed certificates [48].
Pass / Fail	**Pass with Observation** – stream latency of 200–400 ms acceptable for rescue monitoring; self-signed certificate management should be addressed before operational deployment.

4.5.2.5 Flight Stability and Hover Testing

Flight stability and hover testing was conducted to evaluate the UAV's ability to maintain controlled and balanced flight under basic operating conditions [21]. These tests verify stability during hover, attitude responsiveness, and short-duration flight behavior. All tests were performed at low altitude (approximately 0.5–1.0 m) in a controlled indoor environment using ArduPilot stabilized flight modes [29]. Minor variations due to indoor airflow disturbances, absence of GPS position hold, and sensor noise were anticipated and are documented in the results.

Table 51: Flight Test Case TC-UAV-FS-01: Initial Hover Stability Assessment

Test Case ID	TC-UAV-FS-01
Test Objective	Assess hover stability and positional drift under indoor conditions without GPS position hold.
Preconditions	UAV fully calibrated; operating in STABILIZE mode indoors at approximately 0.5–1.0 m altitude; operator ready for manual intervention.
Test Steps	• Arm UAV and gradually increase throttle to achieve stable hover. • Maintain hover for approximately 20–30 seconds and observe lateral drift and altitude stability.
Expected Result	UAV maintains stable hover position with minimal lateral drift in STABILIZE mode.
Actual Result	Hover achieved at target altitude. Lateral drift of approximately 0.3–0.6 m observed over 20–30 seconds, attributed to absence of GPS position hold in indoor STABILIZE mode and indoor airflow disturbances. Altitude was maintained manually with reasonable consistency. Note: this level of positional drift exceeds the precision required for confined rescue environments and highlights the need for outdoor GPS-enabled Loiter mode for operational deployment. PID gain tuning is recommended to improve STABILIZE mode response.
Pass / Fail	**Conditional Pass** – hover achieved successfully; lateral drift of 0.3–0.6 m is acceptable for open-area deployment but requires GPS-enabled Loiter mode and PID tuning refinement for confined or precision rescue operations.

Table 52: Flight Test Case TC-UAV-FS-02: Attitude Stability and Recovery Response

Test Case ID	TC-UAV-FS-02
Test Objective	Evaluate ArduPilot attitude stabilization response to manual control inputs and recovery behavior.
Preconditions	UAV in stable hover in STABILIZE mode; operator applying small manual inputs via RC transmitter.
Test Steps	• Apply small deliberate pitch, roll, and yaw inputs via RC transmitter. • Release inputs and observe ArduPilot attitude recovery behavior and settling time.
Expected Result	UAV returns smoothly to level attitude after input release with minimal overshoot.
Actual Result	Pitch and roll recovery observed within approximately 1–2 seconds with slight underdamped overshoot of approximately 3–5° before settling. Yaw axis showed slightly longer recovery of approximately 1.5–2.5 seconds due to lower yaw authority in the current PID configuration. Behavior is consistent with default ArduPilot PID settings [29] and expected to improve with gain tuning.
Pass / Fail	**Pass with Observation** – attitude recovery functional; yaw axis overshoot and settling time identified as targets for PID tuning before deployment.

Table 53: Flight Test Case TC-UAV-FS-03: Low-Altitude Short-Duration Stability

Test Case ID	TC-UAV-FS-03
Test Objective	Validate overall flight stability during sustained low-altitude hover in a controlled environment.
Preconditions	UAV operating in STABILIZE mode at approximately 0.5–1.0 m altitude; controlled indoor test area.
Test Steps	• Maintain controlled hover for 30–60 seconds with minimal manual input. • Observe altitude variation, lateral drift, oscillation, and control consistency throughout.
Expected Result	Stable sustained flight with minimal oscillation and acceptable altitude variation.
Actual Result	Consistent hover maintained for full duration. Minor altitude variation of ±0.2–0.3 m observed, attributed to throttle sensitivity in STABILIZE mode without barometric hold. No significant oscillation detected. Frame and motor behavior remained consistent throughout the test duration.
Pass / Fail	**Pass with Observation** – altitude variation within manual control limits; altitude hold mode recommended for operational missions to reduce pilot workload.

4.5.2.6 Autonomous Flight Mode Testing

Autonomous flight mode testing was performed to verify the correct functioning of ArduPilot flight modes [29] required for SwiftRescue operations, including STABILIZE, ALTITUDE HOLD, and LOITER. These tests confirm that the UAV can transition between modes, execute predefined autonomous behaviors, and maintain stability under mode-specific control. All tests were conducted at low altitude in controlled environments to ensure safety while evaluating responsiveness, stability, and GPS-dependent behavior.

Table 54: Flight Mode Test Case TC-UAV-AF-01: STABILIZE Mode Verification

Test Case ID	TC-UAV-AF-01
Test Objective	Verify correct manual control behavior and attitude stabilization assistance in STABILIZE mode.
Preconditions	UAV armed and hovering in controlled indoor environment; STABILIZE mode selected via RC transmitter.
Test Steps	• Switch to STABILIZE mode via RC transmitter. • Apply deliberate pitch, roll, and yaw inputs and observe ArduPilot stabilization response.
Expected Result	UAV responds correctly to manual RC inputs with ArduPilot stabilization assistance preventing uncontrolled attitude drift.
Actual Result	Responsive and stable control confirmed in STABILIZE mode. Minor input-to-response delay of approximately 100–150 ms observed, consistent with ArduPilot control loop frequency [29]. Stabilization assistance prevented attitude divergence during all applied inputs.
Pass / Fail	**Pass**

Table 55: Flight Mode Test Case TC-UAV-AF-02: ALTITUDE HOLD Mode Testing

Test Case ID	TC-UAV-AF-02
Test Objective	Verify automatic altitude maintenance behavior in ALTITUDE HOLD mode using barometric sensor.
Preconditions	UAV in stable hover with barometric altitude reference established; ALTITUDE HOLD mode ready.
Test Steps	• Switch to ALTITUDE HOLD mode and release throttle input. • Observe altitude maintenance behavior and variation over 30 seconds.
Expected Result	UAV autonomously maintains the reference altitude with minimal variation.
Actual Result	Altitude hold engaged successfully. Altitude variation of approximately ± 0.2–0.4 m observed, attributed to barometric sensor noise in the indoor environment and minor air pressure fluctuations. Performance is acceptable for outdoor operational conditions where barometric readings are more stable. No uncontrolled altitude excursions occurred [29].
Pass / Fail	**Pass with Observation** – altitude hold functional; ± 0.4 m variation in indoor conditions expected to improve in outdoor deployment where barometric environment is more stable.

Table 56: Flight Mode Test Case TC-UAV-AF-03: LOITER Mode (GPS Position Hold) Testing

Test Case ID	TC-UAV-AF-03
Test Objective	Verify GPS-based position hold accuracy and drift behavior in LOITER mode during outdoor testing.
Preconditions	GPS lock achieved with a minimum of 8 satellites in outdoor environment [25]; UAV airborne in LOITER mode.
Test Steps	• Switch to LOITER mode in outdoor environment with confirmed GPS lock. • Observe UAV position hold accuracy and drift over 20–30 seconds.
Expected Result	UAV maintains GPS-referenced position with minimal horizontal drift.
Actual Result	Position hold activated with GPS lock of 9–11 satellites. Horizontal position drift of approximately 0.5–1.0 m observed during the test period, consistent with the expected positional accuracy of the u-blox NEO-M8N GPS module under standard conditions [25]. Drift is acceptable for open-area search and rescue operations but would limit precision in narrow or confined deployment zones. GPS-assisted position hold represents a significant improvement over indoor STABILIZE-only drift observed in TC-UAV-FS-01.
Pass / Fail	**Conditional Pass** – GPS position hold functional and represents operational improvement over STABILIZE mode; 0.5–1.0 m drift acceptable for open-area missions but may limit precision in confined environments. RTK-GPS upgrade recommended for precision deployment scenarios.

4.5.2.7 System Readiness and Safety Validation

System readiness and safety validation testing was conducted to confirm that the SwiftRescue UAV platform meets safety requirements and is operationally ready for controlled autonomous missions [29]. These tests verify the completeness of pre-flight safety checks in Mission Planner [35], correct ArduPilot failsafe mechanism behavior under simulated fault conditions, and reliable emergency disarm functionality. The objective is to confirm that all safety layers function correctly before field deployment.

Table 57: Safety Test Case TC-UAV-SF-01: Pre-Flight Safety Check Verification

Test Case ID	TC-UAV-SF-01
Test Objective	Verify that all Mission Planner pre-flight safety checks are correctly performed and enforced before arming.
Preconditions	UAV powered and connected to Mission Planner [35]; GPS lock available in outdoor environment.
Test Steps	• Verify GPS lock status and satellite count ($\geq$8 satellites) in Mission Planner. • Verify battery level above minimum operational threshold. • Confirm sensor health indicators and system readiness status in Mission Planner. • Attempt UAV arming and confirm pre-arm check enforcement.
Expected Result	UAV passes all pre-flight safety checks and ArduPilot allows arming only when all checks are satisfied.
Actual Result	All pre-flight safety checks validated successfully. GPS lock acquisition required approximately 45–90 seconds in outdoor environment; arming was correctly blocked until GPS lock was confirmed by ArduPilot [29]. Battery and sensor health checks passed without issues. Pre-arm enforcement functioned correctly, preventing arming under incomplete conditions.
Pass / Fail	**Pass with Observation** – pre-flight checks enforced correctly; GPS cold-start acquisition time of 45–90 seconds should be factored into operational pre-mission planning.

Table 58: Safety Test Case TC-UAV-SF-02: ArduPilot Failsafe Response Verification

Test Case ID	TC-UAV-SF-02
Test Objective	Validate correct ArduPilot failsafe response behavior under simulated low-battery and telemetry signal loss conditions.
Preconditions	UAV powered and connected to Mission Planner [35]; failsafe thresholds configured in ArduPilot [29].
Test Steps	• Simulate low battery condition by adjusting battery failsafe voltage threshold in Mission Planner. • Simulate telemetry signal interruption by disconnecting the MAVLink link briefly. • Observe ArduPilot failsafe response behavior in both scenarios.
Expected Result	ArduPilot triggers configured failsafe action (RTL or controlled landing) correctly under both fault conditions.
Actual Result	Battery failsafe triggered correctly with a response delay of approximately 1–2 seconds from threshold crossing, transitioning the system to the configured safe action as expected. RC signal loss failsafe also triggered correctly within the same delay window. Response delay is within ArduPilot's documented failsafe detection latency [29]. Both failsafe scenarios resolved without system instability.
Pass / Fail	**Pass with Observation** – failsafe mechanisms functional; 1–2 second detection-to-response latency should be considered when defining minimum safe operating altitudes for field missions.

Table 59: Safety Test Case TC-UAV-SF-03: Emergency Stop and Disarm Testing

Test Case ID	TC-UAV-SF-03
Test Objective	Verify immediate and reliable motor shutdown via emergency disarm command under controlled conditions.
Preconditions	UAV armed on ground in controlled environment with propellers attached; Mission Planner and RC transmitter both available as disarm input sources.
Test Steps	• Trigger emergency disarm command via Mission Planner GCS. • Separately trigger emergency disarm via RC transmitter disarm sequence. • Observe motor shutdown response time and system disarm state in both scenarios.
Expected Result	Motors stop completely and system disarms within acceptably short response time from both GCS and RC inputs.
Actual Result	Immediate disarm confirmed from both Mission Planner and RC transmitter inputs within approximately 0.5 seconds. Motors stopped without any instability or delayed shutdown. System correctly transitioned to disarmed state in Mission Planner status indicators after both tests [29]. Both disarm input paths functioned reliably and consistently.
Pass / Fail	**Pass**

Chapter 5

Conclusions and Future Work

5.1 Conclusion

This project presented the design, implementation, and validation of **SwiftRescue**, an AI-enabled multi-UAV system developed to support search and rescue operations in disaster-affected environments [5, 3]. The work addresses key challenges associated with traditional single-UAV approaches, including limited area coverage, delayed situational awareness, and lack of coordinated multi-agent operation, by introducing a scalable and modular framework for distributed UAV deployment under unified Ground Control Station supervision [8].

A simulation-driven development methodology was adopted using Gazebo Harmonic [26], ROS 2 Jazzy [27], and ArduPilot SITL [30] to enable safe and systematic validation of system components before hardware deployment. Realistic flood disaster environments were modeled in Gazebo to evaluate autonomous UAV navigation, coordinated multi-UAV operation, and real-time communication behavior under controlled conditions. A fine-tuned YOLOv8s perception model [11], trained on a custom dataset of 472 flood-related aerial images across two training phases, was successfully deployed to perform victim detection from UAV camera feeds. The baseline model achieved a mean Average Precision of 0.893 at mAP@0.5, with precision of 0.836 and recall of 0.810. The UAV-optimized deployment model, trained for 80 epochs at 960px resolution with aerial-specific augmentation strategies, was selected for deployment due to its improved generalization under real-world aerial conditions, validating the feasibility of deploying AI-based vision models within UAV-assisted rescue workflows [12].

The system architecture was designed with clear separation between perception, navigation, communication, and user interaction layers, ensuring modularity and extensibility. Reliable dual-channel data exchange was achieved using MAVLink [39] for flight telemetry and HTTPS-encrypted streaming [48] for YOLOv8s detection outputs, enabling real-time situational awareness through the dual-interface Ground Control Station combining Mission Planner [35] with a web-based detection dashboard. Multi-UAV coordination was realized through GCS-based mission planning, where each UAV executes an independently assigned waypoint-based mission plan under unified operator supervision, enabling systematic and non-overlapping area coverage without requiring inter-UAV communication.

Comprehensive validation was conducted through both simulation-based and hardware-oriented testing. Thirty-five simulation test cases confirmed correct execution of mission workflows, stable ArduPilot autonomous navigation, functional YOLOv8s perception pipeline integration, reliable MAVLink communication, and effective operator interaction through Mission Planner. Hardware-oriented testing across 23 test cases demonstrated stable power delivery to the Pixhawk flight controller [20] and NVIDIA Jetson Orin Nano [19], correct sensor calibration, reliable propulsion behavior, and effective ArduPilot failsafe mechanisms [29]. Seven tests passed cleanly, fourteen passed with documented observations — including thermal throttling under sustained YOLOv8s in-

ference load and ESC startup timing variation — and two received conditional passes related to hover drift and GPS positional accuracy, identifying specific areas for pre-deployment refinement. These results collectively confirm that the system is capable of supporting controlled autonomous operation on physical hardware.

It is important to acknowledge that the current implementation was validated primarily in simulation and controlled indoor testing environments. Real-world outdoor deployment introduces additional challenges including wind disturbances, GPS multipath effects, varying lighting conditions for perception, and communication variability, which will require further evaluation [1]. Furthermore, the current coordination approach relies on GCS-managed mission planning rather than decentralized swarm intelligence, which limits real-time adaptability when mission conditions change dynamically during execution. The hover positional drift observed in hardware testing also indicates that PID tuning refinement and GPS-enabled Loiter mode are prerequisites for precision deployment in confined rescue environments.

Despite these limitations, SwiftRescue establishes a strong and technically grounded foundation for UAV-based disaster response by successfully integrating autonomous navigation, AI-driven perception using the fine-tuned YOLOv8s model [11], HTTPS-secured data streaming [48], and real-time GCS monitoring within a cohesive and tested framework. The project demonstrates not only functional system implementation across simulation and hardware but also the practical feasibility of combining autonomous flight control, onboard deep learning inference on the NVIDIA Jetson Orin Nano [19], and operator-supervised multi-UAV coordination into a unified operational system for time-critical search and rescue applications.

Overall, this work contributes a structured, modular, and extensible approach toward the development of intelligent multi-UAV systems, highlighting the potential of coordinated aerial platforms to enhance efficiency, coverage, and responsiveness in disaster management applications [4, 3].

5.2 Future Work

While the SwiftRescue system demonstrates a functional and validated framework for UAV-based search and rescue, several enhancements can be explored to further improve system autonomy, robustness, and real-world applicability.

The primary direction for future work is the transition from controlled indoor hardware validation to full outdoor real-world deployment. This involves extended field testing of the physical UAV platform in operational outdoor environments, where factors such as wind disturbances, GPS multipath inaccuracies, varying illumination conditions affecting YOLOv8s detection performance [11], and wireless communication variability must be systematically characterized and addressed. PID gain tuning for improved hover stability and RTK-GPS integration for sub-meter positional accuracy are immediate prerequisites identified from hardware testing results.

Another important extension is the development of decentralized swarm coordination. The current system achieves coordinated coverage through GCS-based mission planning with independently assigned waypoint plans per UAV [35]. Future implementations can incorporate distributed decision-making mechanisms such as consensus-based task allocation and dynamic area repartitioning, enabling UAVs to adapt mission assignments in real time when environmental conditions change or when a UAV becomes unavailable

during deployment [8].

The YOLOv8s perception module [11] can be further enhanced by expanding the training dataset beyond the current 472-image collection to include more diverse aerial perspectives, weather conditions, and victim appearances. Integration of additional sensing modalities such as thermal infrared imaging would significantly improve detection capability in low-visibility scenarios including nighttime operations and smoke-obscured environments, which are common in real disaster contexts [5].

Communication reliability represents another critical area for improvement. The current architecture uses independent MAVLink [39] and HTTPS [48] channels between each UAV and the GCS. Future work can explore mesh-based communication between UAVs to enable peer-to-peer data exchange and detection sharing without requiring continuous direct GCS connectivity, improving system robustness in large-scale or remote deployments [51].

Mission planning capabilities can be enhanced through intelligent path planning algorithms that dynamically optimize waypoint sequences based on terrain features, victim detection confidence maps, and remaining battery capacity. Reinforcement learning-based approaches [36] represent a promising direction for training adaptive mission planning policies that can respond to dynamic disaster environments, though significant work is required to ensure safety and reliability before such methods can be deployed on physical hardware platforms.

Security and system resilience can also be strengthened beyond the current self-signed HTTPS certificate implementation [48]. Future work should incorporate properly issued TLS certificates, MAVLink message authentication, and automated failsafe recovery strategies that go beyond the current return-to-launch behavior, including intelligent mission repartitioning when a UAV exits the swarm during operation.

Overall, these enhancements aim to transition SwiftRescue from a simulation-validated and hardware-tested prototype toward a fully autonomous, resilient, and operationally deployable UAV swarm system capable of supporting real-world disaster response in Pakistan and similar disaster-prone regions [1, 2].

References

[1] A. Malik and T. Khan, "Drone technology for flood response in Pakistan," *International Journal of Disaster Risk Reduction*, vol. 92, p. 103800, 2023.

[2] United Nations Office for Disaster Risk Reduction (UNDRR), "Pakistan flood response 2022–2023: Damage and needs assessment," [Online]. Available: `https://www.undrr.org`, 2024.

[3] J. P. Queralta, J. Taipalmaa, B. C. Pullinen, V. K. Sarker, T. N. Gia, H. Tenhunen, M. Gabbouj, J. Raitoharju, and T. Westerlund, "Collaborative multi-robot search and rescue: Planning, coordination, perception, and active vision," *IEEE Access*, vol. 8, pp. 191617–191643, 2020.

[4] M. Ahmad and M. Waseem, "AI-driven UAV swarm systems for disaster response: A review," *Journal of Intelligent & Robotic Systems*, vol. 105, no. 4, pp. 1–18, 2022.

[5] M. Erdelj, E. Natalizio, K. R. Chowdhury, and I. F. Akyildiz, "Help from the sky: Leveraging UAVs for disaster management," *IEEE Pervasive Computing*, vol. 16, no. 1, pp. 24–32, 2017.

[6] D. Floreano and R. J. Wood, "Science, technology and the future of small autonomous drones," *Nature*, vol. 521, no. 7553, pp. 460–466, 2015.

[7] A. Koubaa, Ed., *Robot Operating System (ROS): The Complete Reference*, vol. 4, Springer, 2019.

[8] C. Zhang, L. Wang, and H. Liu, "Swarm intelligence in UAVs: Algorithms and applications," *IEEE Transactions on Cybernetics*, vol. 51, no. 10, pp. 5063–5077, 2021.

[9] G. Chmaj and H. Selvaraj, "Distributed processing applications for UAV/drones: A survey," *Progress in Systems Engineering*, Springer, pp. 449–454, 2015.

[10] M. Silvagni, A. Tonoli, E. Zenerino, and M. Chiaberge, "Multipurpose UAV for search and rescue operations in mountain avalanche events," *Geomatics, Natural Hazards and Risk*, vol. 8, no. 1, pp. 18–33, 2017.

[11] Ultralytics, "YOLOv8: A new state-of-the-art model for real-time object detection," [Online]. Available: `https://github.com/ultralytics/ultralytics`, 2023.

[12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.

[13] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[14] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and efficient object detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 10781–10790, 2020.

[15] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017.

[16] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.

[17] S. Mittal, "A survey of techniques for optimizing deep learning on embedded platforms," *Journal of Systems Architecture*, vol. 97, pp. 1–15, 2019.

[18] X. Zhu, S. Lyu, X. Wang, and Q. Zhao, "TPH-YOLOv5: Improved YOLOv5 based on transformer prediction head for object detection on drone-captured scenarios," in *Proc. IEEE/CVF Int. Conf. Computer Vision Workshops (ICCVW)*, pp. 2778–2788, 2021.

[19] NVIDIA Corporation, "Jetson Orin Nano Developer Kit documentation," [Online]. Available: `https://developer.nvidia.com/embedded/jetson-orin-nano-developer-kit`, 2024.

[20] L. Meier, D. Honegger, and M. Pollefeys, "Pixhawk: A micro aerial vehicle design for autonomous flight using open-source hardware," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, pp. 3861–3868, 2014.

[21] R. Mahony, V. Kumar, and P. Corke, "Multirotor aerial vehicles: Modeling, estimation, and control," *IEEE Robotics & Automation Magazine*, vol. 19, no. 3, pp. 20–32, 2012.

[22] P. Pounds, R. Mahony, and P. Corke, "Modelling and control of a quad-rotor robot," in *Proc. Australasian Conf. Robotics and Automation*, 2006.

[23] A. Bouabdallah, P. Murrieri, and R. Siegwart, "Design and control of an indoor micro quadrotor," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, pp. 4393–4398, 2004.

[24] D. Floreano, R. J. Wood, and others, "Autonomous flying robots: UAV platforms and onboard intelligence," in *Springer Handbook of Robotics*, Springer, 2016.

[25] P. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, 2nd ed., Norwood, MA, USA: Artech House, 2013.

[26] Open Source Robotics Foundation (OSRF), "Gazebo Harmonic simulation documentation," [Online]. Available: `https://gazebosim.org/libs/sim/`, 2025.

[27] ROS-Industrial Consortium, "ROS 2 Jazzy documentation," [Online]. Available: `https://docs.ros.org/en/jazzy/index.html`, 2024.

[28] ROS-Industrial Consortium, "ROS 2 and Gazebo integration tutorial (Jazzy)," [Online]. Available: `https://docs.ros.org/en/jazzy/Tutorials/Advanced/Simulators/Gazebo/Simulation-Gazebo.html`, 2024.

[29] ArduPilot Development Team, "ArduPilot autopilot documentation," [Online]. Available: `https://ardupilot.org/`, 2025.

[30] ArduPilot Development Team, "ArduPilot software-in-the-loop (SITL) simulation guide," [Online]. Available: https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html, 2025.

[31] ArduPilot Development Team, "Using SITL with Gazebo: ArduPilot Gazebo plugin and tutorial," [Online]. Available: https://ardupilot.org/dev/docs/sitl-with-gazebo.html, 2025.

[32] ArduPilot Development Team, "ROS 2 and Gazebo integration with ArduPilot," [Online]. Available: https://ardupilot.org/dev/docs/ros2-gazebo.html, 2025.

[33] H. R. M. Sardinha, M. Dragone, and P. A. Vargas, "Closing the gap in swarm robotics simulations: An extended ArduPilot/Gazebo plugin," *arXiv preprint arXiv:1811.06948*, 2018.

[34] PX4 Autopilot Development Team, "PX4 Gazebo simulation (SITL) guide," [Online]. Available: https://docs.px4.io/main/en/sim_gazebo_gz/index.html, 2025.

[35] ArduPilot Development Team, "Mission Planner ground control station," [Online]. Available: https://ardupilot.org/planner/, 2025.

[36] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., Cambridge, MA, USA: MIT Press, 2018.

[37] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, and others, "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.

[38] OpenAI and DLR-RM, "Stable-Baselines3: Reinforcement learning framework," [Online]. Available: https://github.com/DLR-RM/stable-baselines3, 2024.

[39] MAVLink Development Team, "MAVLink micro air vehicle communication protocol," [Online]. Available: https://mavlink.io/en/, 2024.

[40] Object Management Group (OMG), "Data Distribution Service (DDS) specification," [Online]. Available: https://www.omg.org/spec/DDS/, 2023.

[41] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[42] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Technical Report TR 98-11, Iowa State University, 1998.

[43] T. Vicsek, A. Czirók, E. Ben-Jacob, I. Cohen, and O. Shochet, "Novel type of phase transition in a system of self-driven particles," *Physical Review Letters*, vol. 75, no. 6, pp. 1226–1229, 1995.

[44] L. E. Parker, "ALLIANCE: An architecture for fault tolerant multirobot cooperation," *IEEE Transactions on Robotics and Automation*, vol. 14, no. 2, pp. 220–240, 1998.

[45] F. Schilling, J. Lecoeur, F. Schiano, and D. Floreano, "Learning vision-based flight in drone swarms by imitation," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 4523–4530, 2019.

[46] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, pp. 740–755, 2014.

[47] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL visual object classes (VOC) challenge," *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.

[48] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed., Pearson, 2017.

[49] H. Shakhatreh, A. H. Sawalmeh, A. Al-Fuqaha, Z. Dou, E. Almaita, I. Khalil, N. S. Othman, A. Khreishah, and M. Guizani, "Unmanned aerial vehicles (UAVs): A survey on civil applications and key research challenges," *IEEE Access*, vol. 7, pp. 48572–48634, 2019.

[50] C. Yan, X. Xiang, and C. Wang, "Towards real-time path planning through deep reinforcement learning for a UAV in dynamic environments," *Journal of Intelligent & Robotic Systems*, vol. 98, pp. 297–309, 2020.

[51] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, "A tutorial on UAVs for wireless networks: Applications, challenges, and open problems," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2334–2360, 2019.